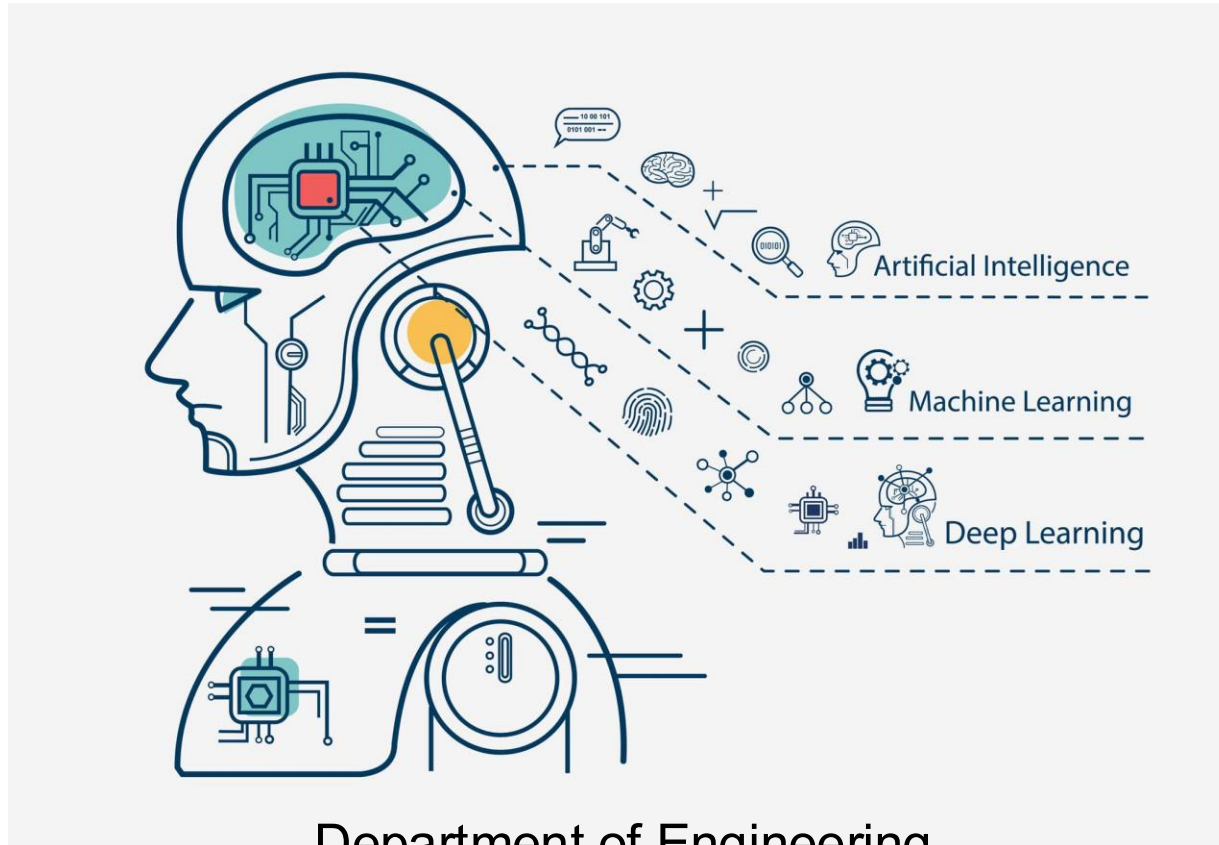


# Lecture 1

## Introduction & Fundamentals



Department of Engineering  
Drexel University  
Summer 2025

# A bit about your Instructor

- Somayeh Keshavarz
- Research Interests
  - Image and Video Processing
  - Computer Vision
  - Deep Learning and Machine Learning Algorithms
  - Biomedical Applications
- Education
  - PhD Candidate in Computer Science, Temple University
  - MS in Computer Science, University of Central Florida
  - MS in Electrical Engineering (Signal and Image Processing)
- Other courses I teach: Computer Vision, Image Processing

# Purpose of this course

The primary objective of this course is to

- teach you how to write python code that makes use of **modern machine learning software libraries**,
  - primarily Keras (a deep learning API) on TensorFlow2,
  - some PyTorch w/ transformers, diffusers, etc.
- to rapidly prototype and test machine learning techniques on data.

*Means we will focus on how to use existing implementations of learning algorithms and models, and best practices for training and evaluating them on data.*

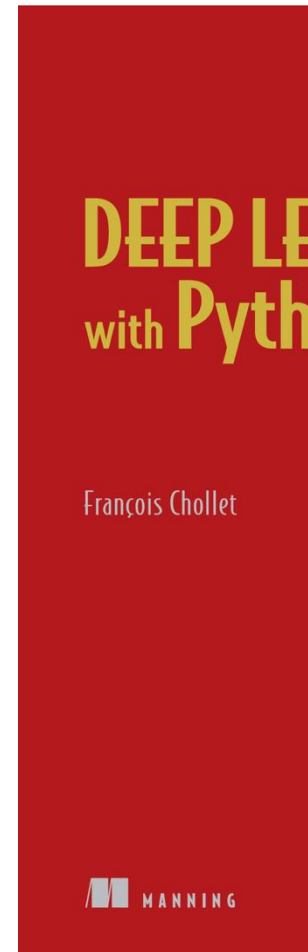
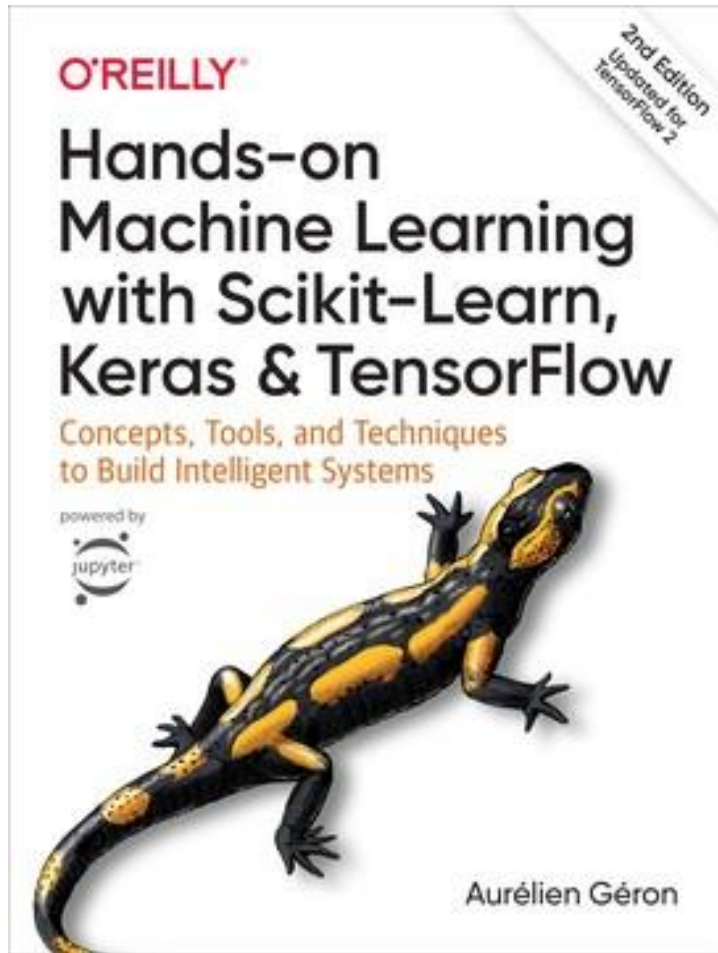
# Machine Learning is a Large Hairy Beast

Modern machine learning is really the nexus of several traditional ECE disciplines, including

- Mathematical statistics (Random variables, stochastic processes, detection & estimation)
- Optimization both numerical and analytical
- Computer Architecture (acceleration via GPU computing)
- Signal Processing & Information Theory (c.f. multimedia encoding, time series)

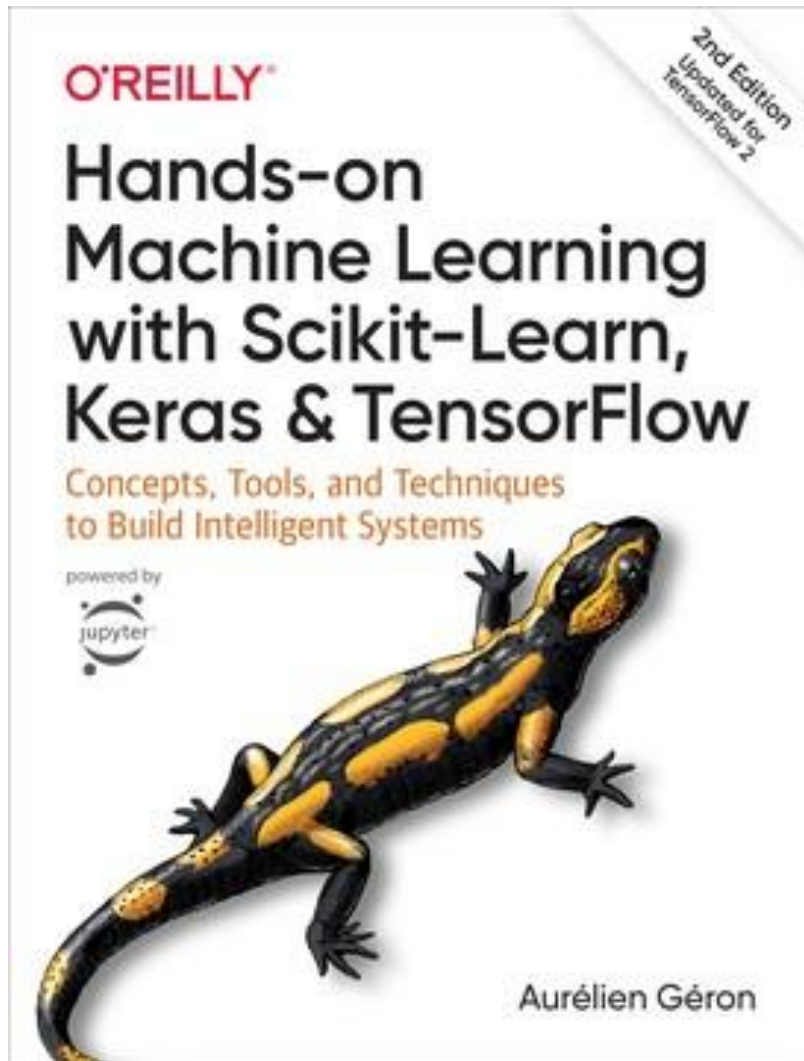
The ECE department has advanced courses, both undergraduate and graduate, in each of these topics.

# Course Textbooks



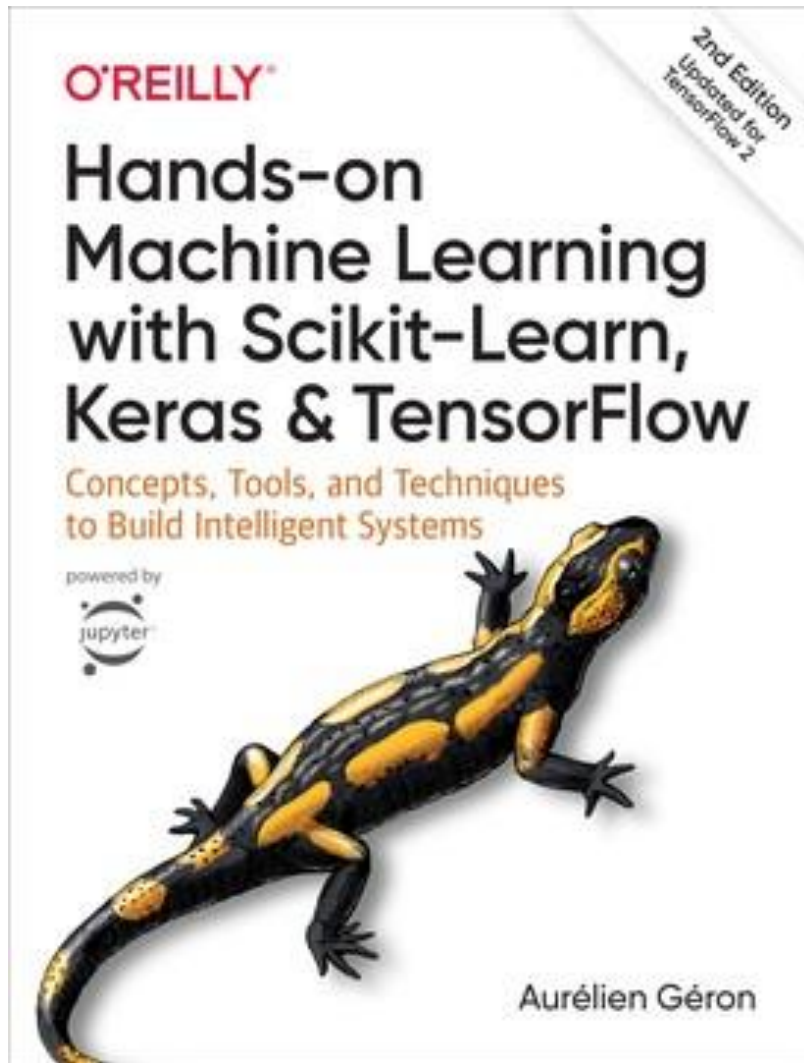
We will be following HOML closely. You could view electronically using link from syllabus.

# Why These Textbooks? – HOML



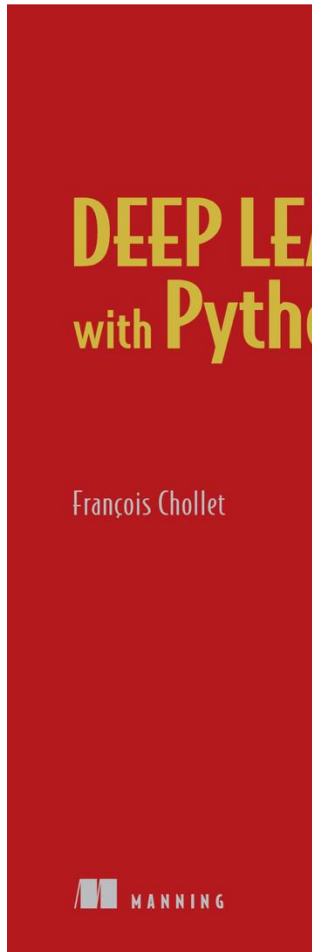
- Strengths:
  - super recent (October '22 -3rd ed!)
  - covers a unique sampling of topics
  - everything applied/coded immediately
  - 3rd ed. improves NLP coverage
- Weaknesses:
  - not a historical treatment
  - possibly over-emphasizes recent winners in ML power tug of war
  - Some important algo.s missing (graphical models/BP/variational Bayes inference/Gibbs sampling,...)
  - Wiener, LMS, adaptive filtering missing
  - occasional bad footnotes (e.g. signals & Shannon)

# Why These Textbooks? – HOML



- Summary: use to learn how to rapidly prototype in scikit-learn and TensorFlow, but do not use to understand the historical development of ML, deep math details, or who the important figures are/were, ethical issues, etc.
- weekly reading on syllabus (ch 1 & 2 this week, ch. 3-4 next)

# Why These Textbooks? – Chollet



- Chollet is the author of Keras API
- Keras API enables one to describe, train, test deep NN w/ very short code
- API can be run on several NN backends (theano, mxn, tensorflow)
- TensorFlow has extended it to include additional features
- No required reading from this text, supplemental. Getting quite dated in '24.
- can get electronically via library using link on syllabus



# Course Syllabus

Go over course syllabus emphasizing

- office hours, emails
- Grading policy (5 biweekly coding projects, drop one)
  - Compare your results with a friend and discuss why one result might be better than the other.
- required reading
- assignment deadlines

# coding projects

## 1- Basic Keras Model Workflow

- Converting a “real” raw dataset into a tfrecord format
- Use keras preprocessing layers
- Build, tune, and save a small keras model

## 2- Transfer Learning for Image Classification

Use transfer learning on the type of vision problems

## 3- Generative Modeling

- Use generative models to produce convincing newly generated images

## 4- Learning to Control with TensorFlow Agents

Your mission is to train machine learning algorithms to play two Atari video games

## 5- Working with Text

Generate sentences for a specific purpose.

# Perceptrons

Slides adapted from Yaser Abu-Mostafa and Mohamed Batouche

# Roadmap

- Linear methods (lines, planes, and hyperplanes)
  - Benefits
    - Simple algorithms
    - (Human) interpretable model parameters
  - Drawbacks
    - *Most real-world problems are not linear*

# Creditworthiness



- Banks would like to decide whether or not to extend credit to new customers
  - Good customers pay back loans
  - Bad customers default
- Problem: Predict creditworthiness based on:
  - Salary, Years in residence, Current debt, Age, etc.

$$\vec{x} = (x_1, x_2, \dots, x_d)$$

Diagram showing arrows pointing from the text "Salary", "Years in residence", "Current debt", and "Age, etc." to the components  $x_1$ ,  $x_2$ ,  $x_d$ , and the ellipsis in the vector  $\vec{x}$  respectively.

# Perceptron

- For a customer,  $\vec{x} = (x_1, x_2, \dots, x_d)$ 
  - Approve credit if  $\sum_{i=1}^d w_i x_i > \text{threshold},$
  - Deny credit if  $\sum_{i=1}^d w_i x_i < \text{threshold}.$
- This linear formula  $h \in \mathcal{H}$  can be written as:

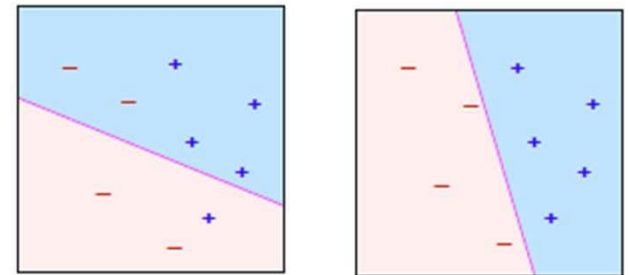
$$h(\mathbf{x}) = \text{sign} \left( \left( \sum_{i=1}^d w_i x_i \right) - \text{threshold} \right)$$

# Perceptron (cont'd),

$$h(\mathbf{x}) = \text{sign} \left( \left( \sum_{i=1}^d w_i x_i \right) + w_0 \right)$$

- Introduce an artificial coordinate  $x_0 = 1$ 
  - Called the *bias* term

$$h(\mathbf{x}) = \text{sign} \left( \sum_{i=0}^d w_i x_i \right)$$



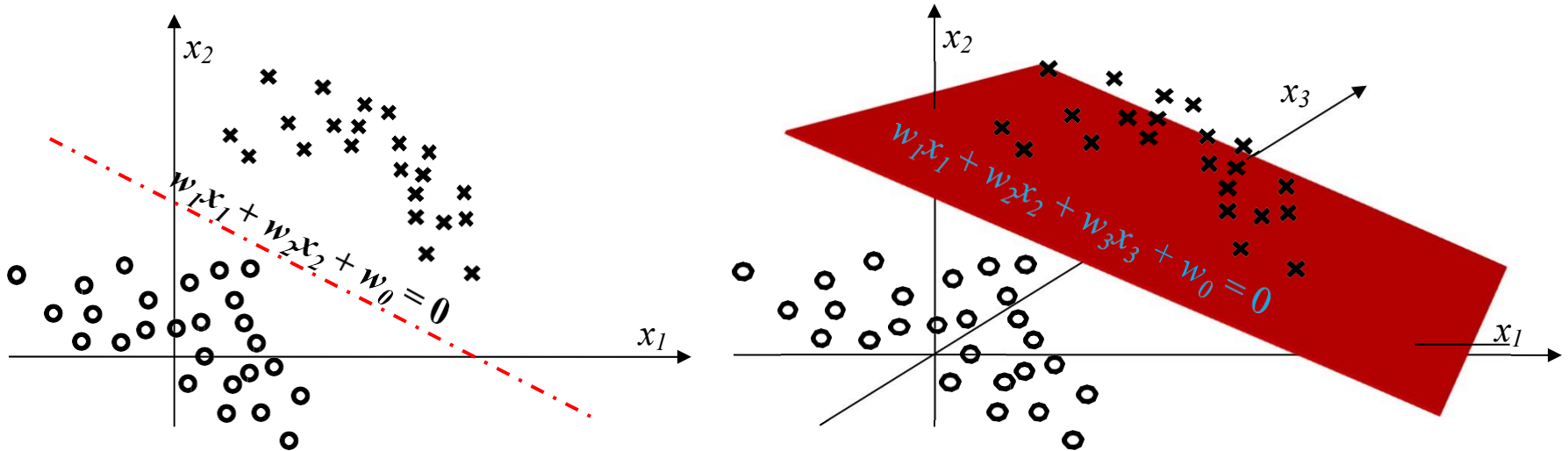
'linearly separable' data

- So, perceptron implements:

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$$

# Perceptron (Geometrically)

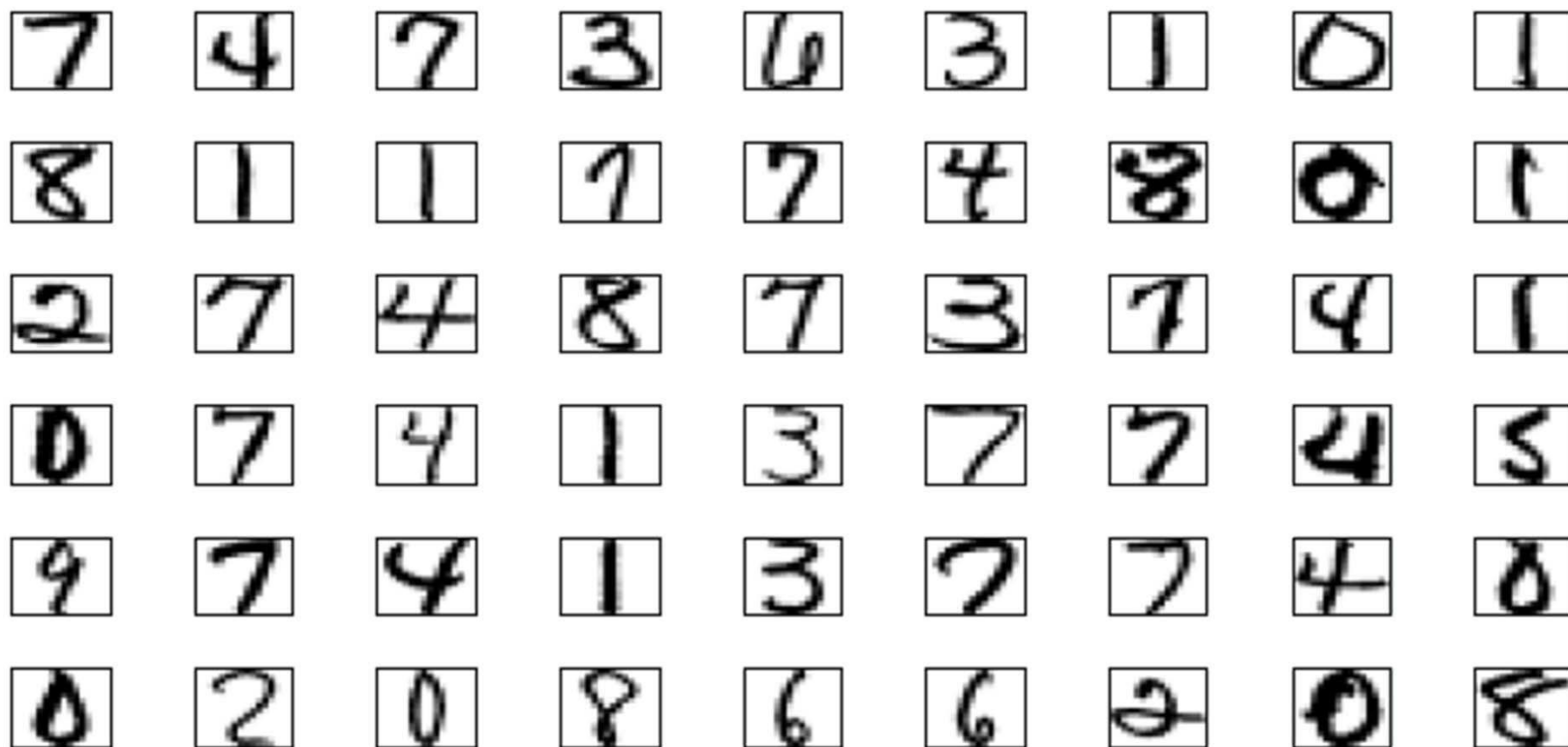
$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$$



- Weights are the coefficients of hyperplane
- $\mathbf{w}^T \mathbf{x}$  is the (signed) distance of point  $\mathbf{x}$  to hyperplane  $\mathbf{w}$ 
  - Example: [http://mathinsight.org/distance\\_point\\_plane](http://mathinsight.org/distance_point_plane)



# Example Linear Classification



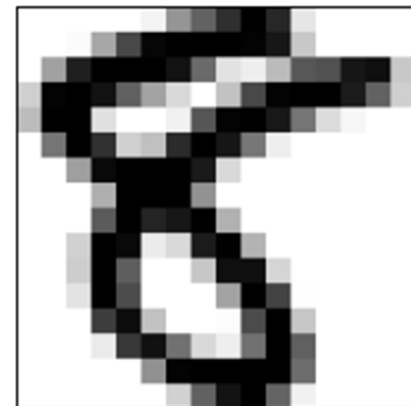
# Input Representation

- Raw input: 16\*16 image = 256 pixels

$$\mathbf{x} = (x_0, x_1, x_2, \dots, x_{256})$$

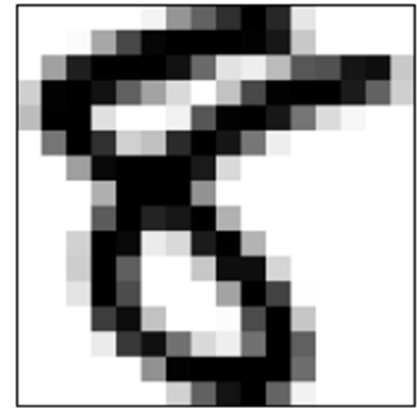
- Linear model:

$$\mathbf{w} = (w_0, w_1, w_2, \dots, w_{256})$$



- Features: Extract useful information from raw data
  - Intensity and symmetry  $\rightarrow \mathbf{x} = (x_0, x_1, x_2)$
  - Linear model  $\rightarrow \mathbf{w} = (w_0, w_1, w_2)$

# Feature Space

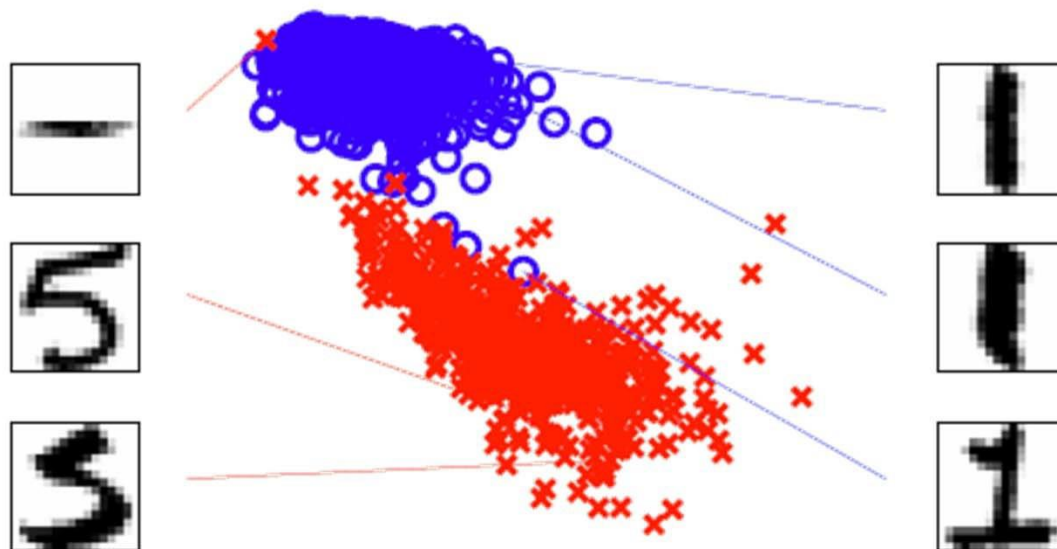


- Consider a binary classification task
  - “Is  $\mathbf{x}$  a ‘1’ (positive) or a ‘5’ (negative)?”

$$\mathbf{x} = (x_0, x_1, x_2)$$

$x_1$ : intensity

$x_2$ : symmetry



Not linearly separable

# Linear Regression

- **Classification:** Credit approval (yes / no)
- **Regression:** Credit line (dollar amount)

- Same input,  $x =$

|                    |          |
|--------------------|----------|
| age                | 23 years |
| annual salary      | \$30,000 |
| years in residence | 1 year   |
| years in job       | 1 year   |
| current debt       | \$15,000 |
| ...                | ...      |

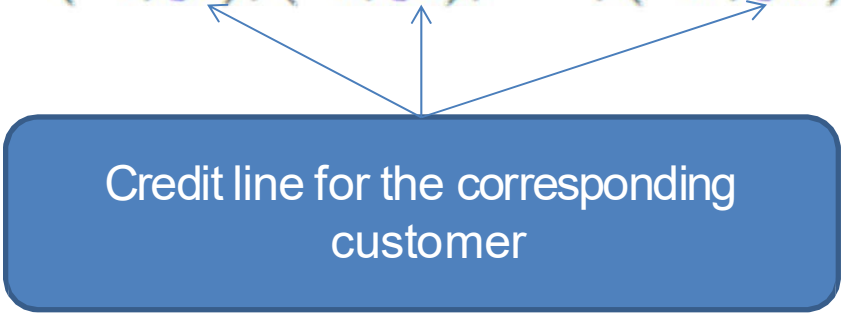
- Linear regression output:

$$h(\mathbf{x}) = \sum_{i=0}^d w_i x_i = \mathbf{w}^T \mathbf{x}$$

# Training Data

- Credit officers decide on credit lines:

$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$



Credit line for the corresponding customer

- Linear regression attempts to replicate this

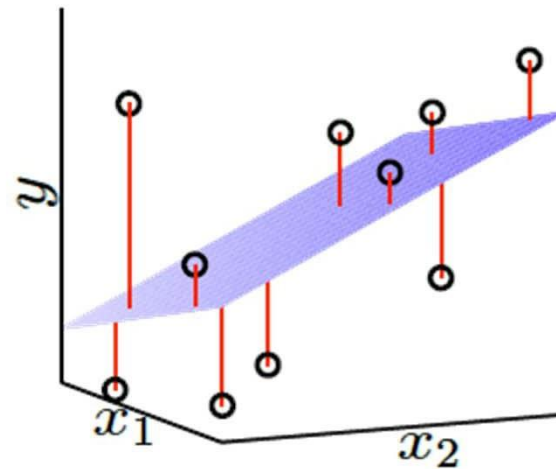
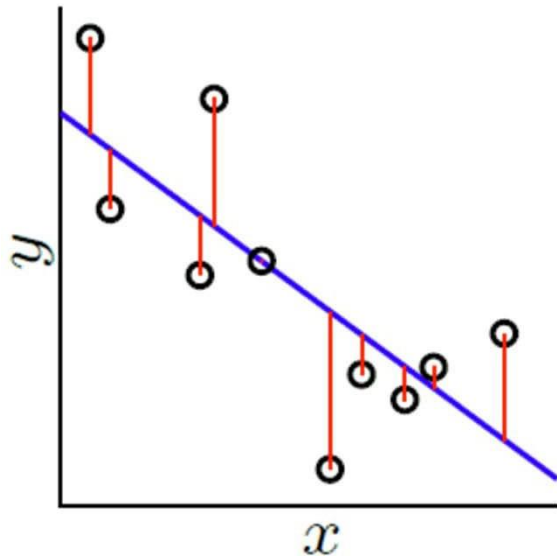
# Error Measure

- Classification error (right or wrong) won't work here
  - With regression, you can be “close” or “far”
- How well does  $h(x) = w^T x$  approximate  $f(x)$ ?
- We use **squared error**:  $(h(x) - f(x))^2$

in-sample error: 
$$E_{\text{in}}(h) = \frac{1}{N} \sum_{n=1}^N (h(\mathbf{x}_n) - y_n)^2$$

# Linear Regression Visually

- Circles are **training examples**
- Line/Plane represent the **hypothesis**
- Red lines represent **in-sample error**



# In-Sample Error, $E_{\text{in}}$

$$\begin{aligned} E_{\text{in}}(\mathbf{w}) &= \frac{1}{N} \sum_{n=1}^N (\mathbf{w}^T \mathbf{x}_n - y_n)^2 \\ &= \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 \end{aligned}$$

where

$$\mathbf{X} = \begin{bmatrix} -\mathbf{x}_1^T- \\ -\mathbf{x}_2^T- \\ \vdots \\ -\mathbf{x}_N^T- \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$$



# Minimizing $E_{\text{in}}$

$$E_{\text{in}}(\mathbf{w}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$$

$$\nabla E_{\text{in}}(\mathbf{w}) = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0}$$

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

$$\mathbf{w} = \mathbf{X}^\dagger \mathbf{y} \quad \text{where} \quad \mathbf{X}^\dagger = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$$

$\mathbf{X}^\dagger$  is the 'pseudo-inverse' of  $\mathbf{X}$

# Linear Regression Algorithm

- 1: Construct the matrix  $\mathbf{X}$  and the vector  $\mathbf{y}$  from the data set  $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)$  as follows

$$\underbrace{\mathbf{X} = \begin{bmatrix} -\mathbf{x}_1^\top - \\ -\mathbf{x}_2^\top - \\ \vdots \\ -\mathbf{x}_N^\top - \end{bmatrix}}_{\text{input data matrix}}, \quad \underbrace{\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}}_{\text{target vector}}.$$

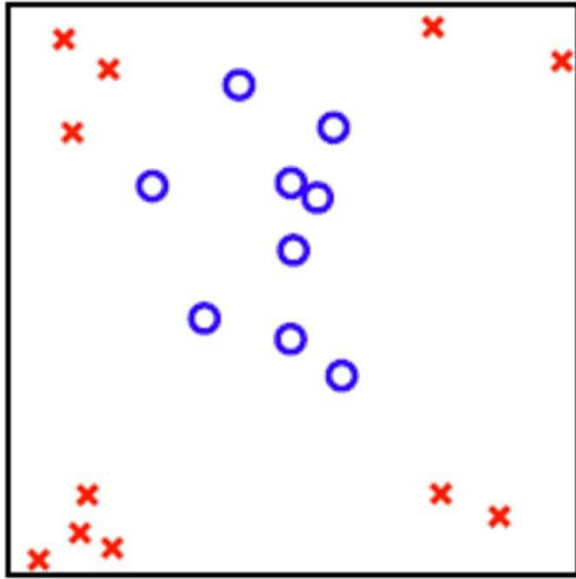
- 2: Compute the pseudo-inverse  $\mathbf{X}^\dagger = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ .
- 3: Return  $\mathbf{w} = \mathbf{X}^\dagger \mathbf{y}$ .

# Back to Credit Example

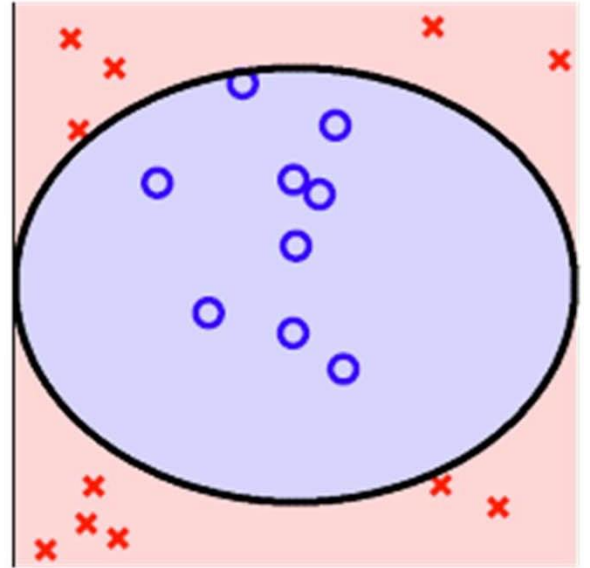
- Credit line is affected by 'years in residence'
  - But perhaps not linearly
- Perhaps credit officers use a different model
  - Is 'years in residence' less than 1?
  - Is 'years in residence' greater than 5?
- Can we do that with linear models?

# Linear?

Data:

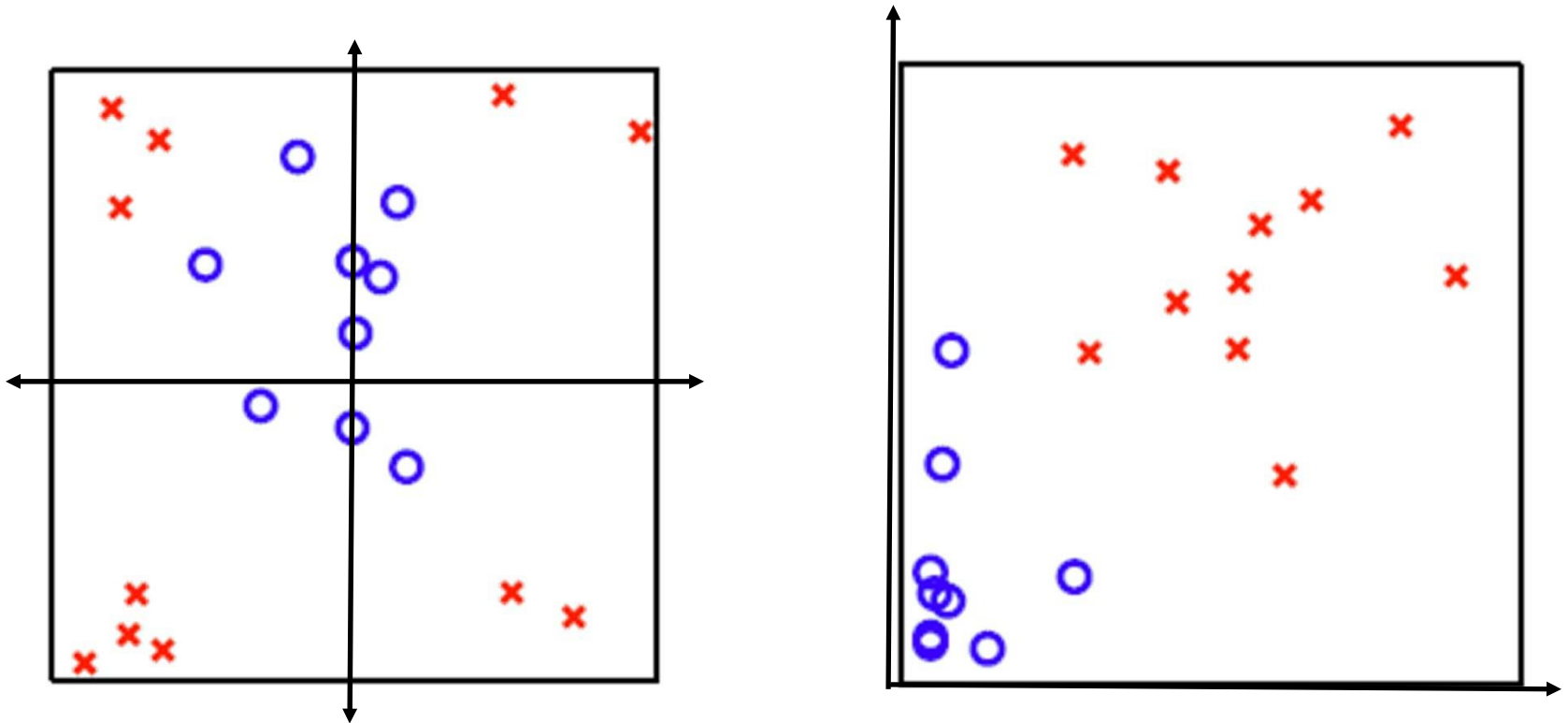


Hypothesis:



# Nonlinear Transformations

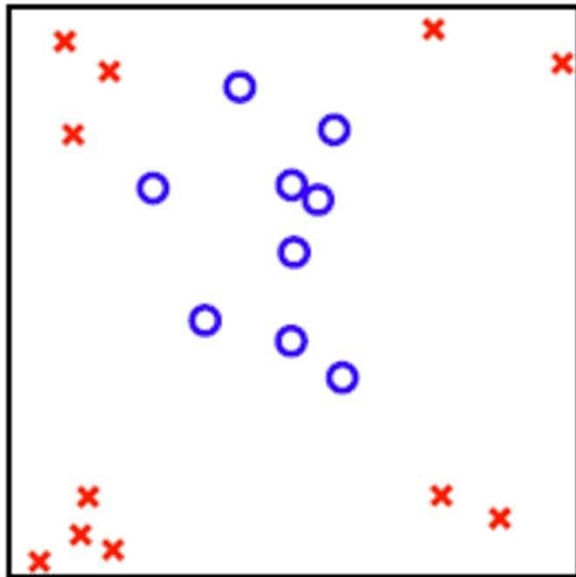
$$(x_1, x_2) \xrightarrow{\Phi} (x_1^2, x_2^2)$$



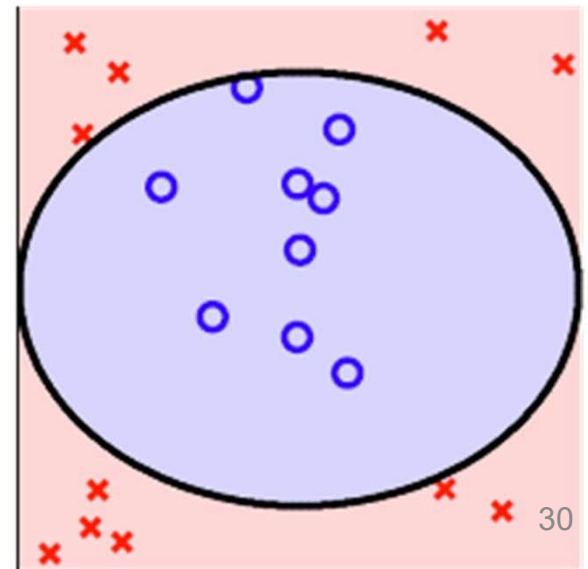
# Beyond Perceptrons

- Two main approaches
  - Combining perceptrons: neural networks
  - Efficient data transforms: kernel methods

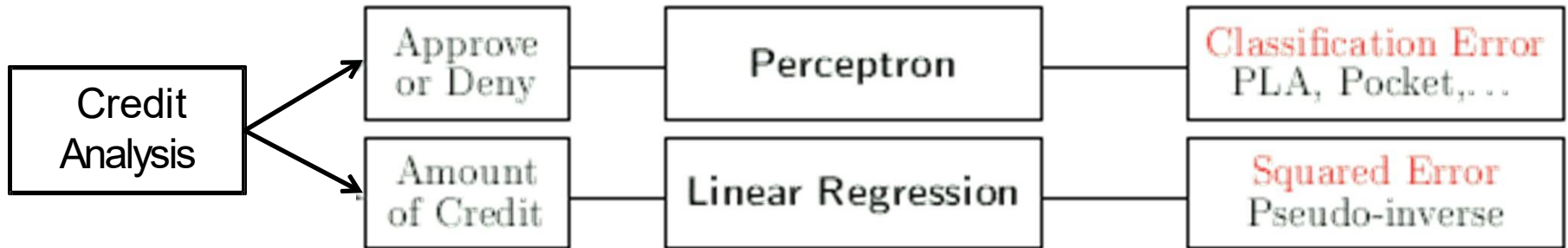
Data:



Hypothesis:



# Summary



- Benefits of linear approaches
  - Simple algorithms
  - (Human) interpretable model parameters
- Drawbacks
  - *Most real-world problems are not linear*
    - *Nonlinear transformations can help*

# Gradient Descent & Regularization

Slides adapted from David Kauchak, Michael T. Brannick, Ethem Alpaydin, and Yaser Abu-Mostafa

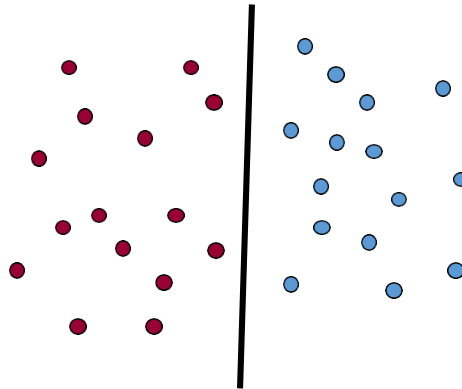


# Linear models

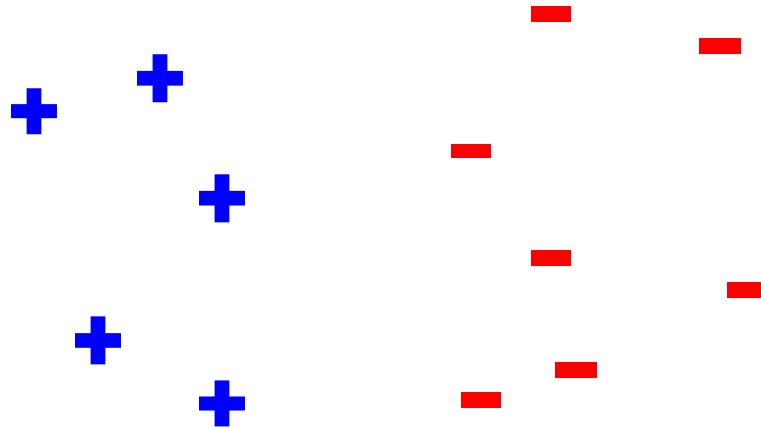
A strong high-bias assumption is *linear separability*:

- in 2 dimensions, can separate classes by a line
- in higher dimensions, need hyperplanes

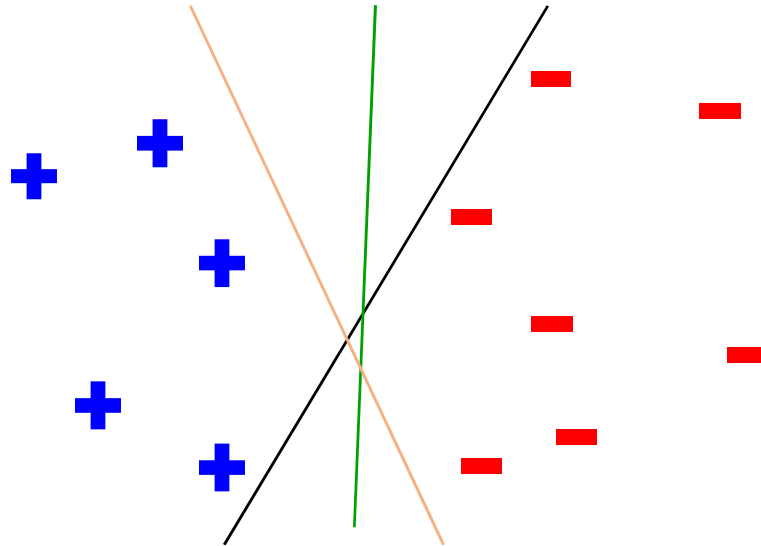
A *linear model* assumes the data is linearly separable



# Which line will it find?



# Which line will it find?



Only guaranteed to find *some*  
line that separates the data

# Linear models

- Perceptron algorithm is one example of a linear classifier
- Many, many other algorithms that learn a line (i.e. a setting of a linear combination of weights)
- Goals:
  - Explore a number of linear training algorithms
  - Understand *why these algorithms work*

# Model-based machine learning

1. pick a model
  - e.g. a hyperplane, a decision tree,...
  - A model is defined by a collection of parameters
2. pick a criterion to optimize (aka objective function)
  - e.g. training error
3. develop a learning algorithm
  - the algorithm should try and minimize the criteria
  - sometimes in a heuristic way (i.e. non-optimally), sometimes exactly

# Linear Models

1. pick a model

$$0 = \underbrace{b} + \sum_{j=1}^m \underbrace{w_j} f_j$$

These are the parameters we want to learn

2. pick a criteria to optimize (aka objective function)

$$\sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$

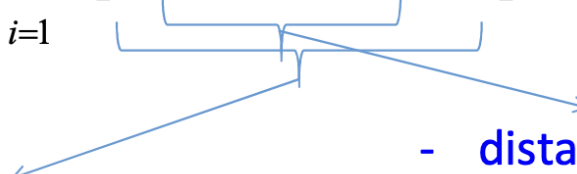
Indicator Function

$$1[x] = \begin{cases} 1 & \text{if } x = \text{True} \\ 0 & \text{if } x = \text{False} \end{cases}$$

3. develop a learning algorithm

*Find  $w$  and  $b$  that minimize the 0/1 loss (i.e. training error)*

# 0/1 loss function

$$\sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$


whether or not the  
prediction and label agree,  
true if ***they don't***

- distance from hyperplane
- sign is prediction

total number of mistakes,  
aka 0/1 loss

# Minimizing 0/1 loss

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$

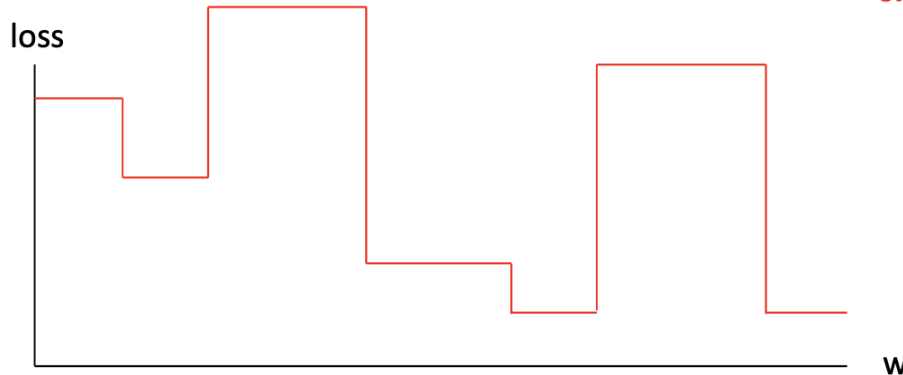
Find  $w$  and  $b$  that  
minimize the 0/1 loss



# Minimizing 0/1 in one dimension

$$\sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$

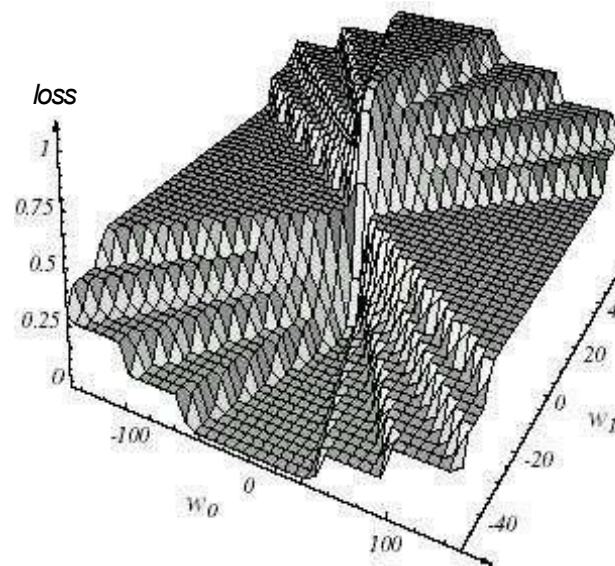
Why is it hard to minimize this function?



Each time we change w such that the example is right/wrong the loss will increase/decrease

# Minimizing 0/1 over all $w$

$$\sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$



Each feature (i.e. weight) represents another dimension to this space!

# Minimizing 0/1 loss

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$

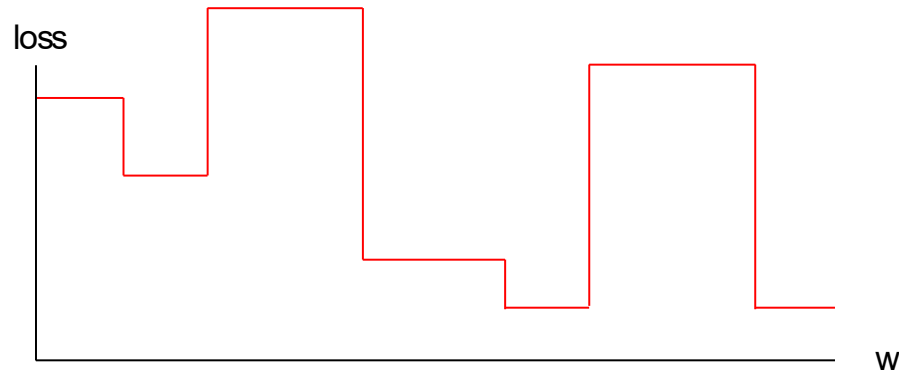
Find  $w$  and  $b$  that  
minimize the 0/1 loss

This turns out to be hard (in fact, NP-HARD  $\square$ )

Challenges:

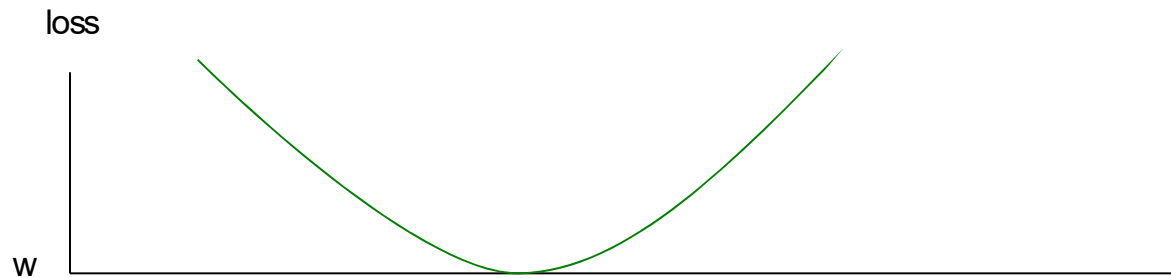
- small changes in some  $w$  can have large effect on loss
- large changes in some  $w$  can have no effect on loss
- there can be many, many local minima
- at any given point, not much information to lead to any minima

# More manageable loss functions



What property/properties do we want from our loss function?

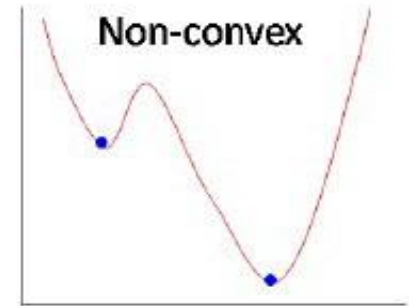
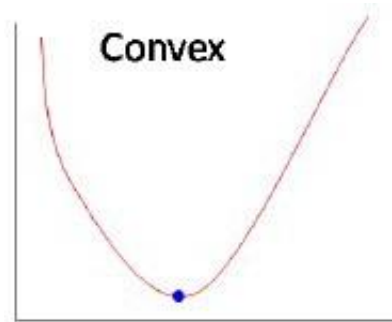
# More manageable loss functions



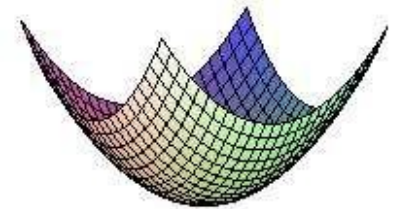
- Ideally, continuous (i.e. differentiable) so we get an indication of direction of minimization
- Only one minima

# Convex functions

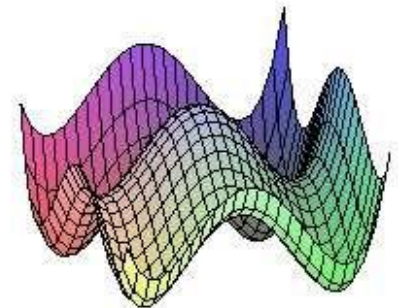
- Smooth, continuous
- Local = global minimum
- Line segment between any two points on the function
  - is above the function
  - Stays “inside” function



Convex



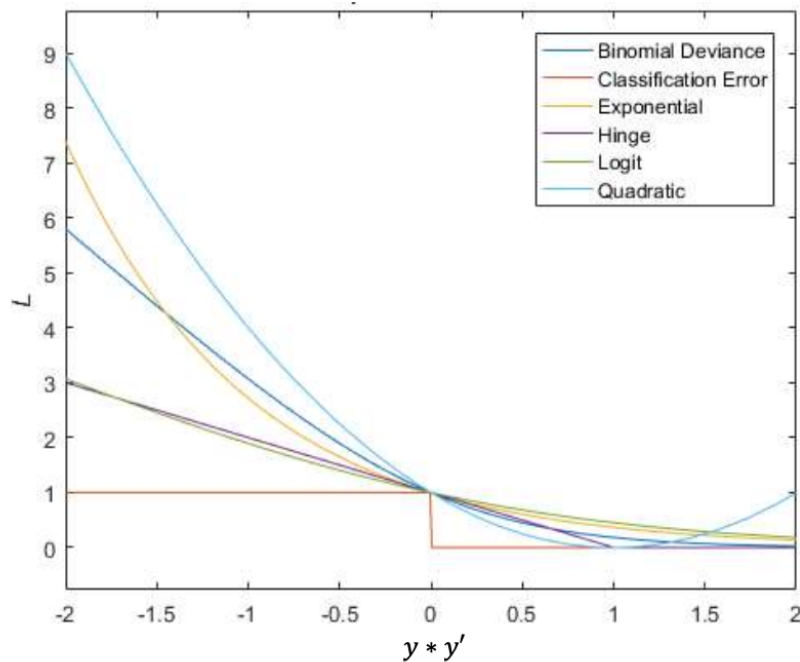
Non-convex



# loss functions

0/1 (classification) loss:  $l(y, y') = 1[y y' \leq 0]$       Hinge:  $l(y, y') = \max(0, 1 - y y')$

Squared (Quadratic) loss:  $l(y, y') = (y - y')^2$       Exponential:  $l(y, y') = \exp(-y y')$



# Linear Model Learning

1. pick a model

$$0 = b + \sum_{j=1}^m w_j f_j$$

2. pick a criteria to optimize (aka objective function)

$$\sum_{i=1}^n \exp(-y_i (w \cdot x_i + b))$$

use a convex loss function  
(e.g., exponential)

3. develop a learning algorithm

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i (w \cdot x_i + b))$$

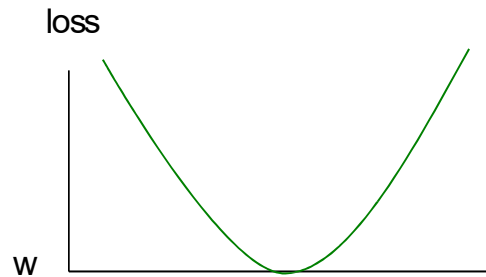
Find  $w$  and  $b$  that  
minimize loss



# Minimizing convex loss function

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b))$$

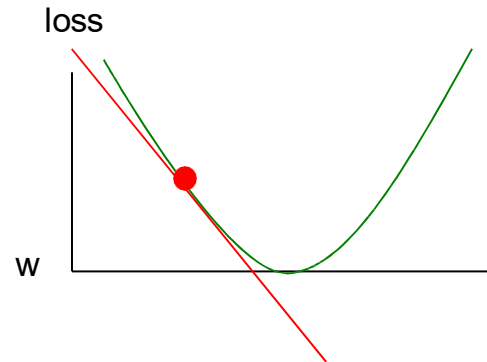
Find  $w$  and  $b$  that  
minimize the  
surrogate loss



How do we do this now?

# One approach: gradient descent

Partial derivatives give us the slope (i.e. direction to move) in that dimension

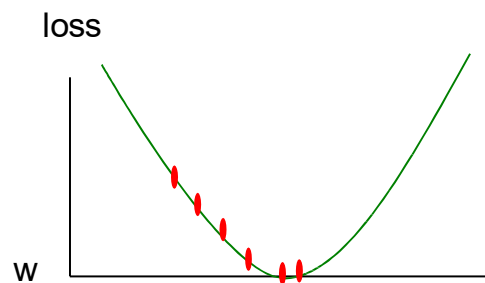


# One approach: gradient descent

Partial derivatives give us the slope (i.e. direction to move) in that dimension

Approach:

- pick a starting point ( $w$ )
- repeat:
  - pick a dimension
  - move a small amount in that dimension towards decreasing loss (using the derivative)



# Gradient descent

- pick a starting point ( $w$ )
- repeat until loss doesn't decrease in any dimension:
  - pick a dimension
  - move a small amount in that dimension towards decreasing loss (using the derivative)

$$w_j = w_j - \eta \frac{d}{dw_j} \text{loss}(w)$$


learning rate (how much we want to move in the error direction, often this will change over time)

# Gradient Descent w/ Exponential Loss

$$\begin{aligned}\frac{d}{dw_j} \text{loss} &= \frac{d}{dw_j} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) \\ &= \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) \frac{d}{dw_j} (-y_i(w \cdot x_i + b)) \\ &= \sum_{i=1}^n -y_i x_{ij} \exp(-y_i(w \cdot x_i + b))\end{aligned}$$

$$\frac{d e^{u(x)}}{dx} = e^{u(x)} \cdot \frac{du(x)}{dx}$$

# Gradient descent

- pick a starting point ( $w$ )
- repeat until loss doesn't decrease in any dimension:
  - pick a dimension
  - move a small amount in that dimension towards decreasing loss (using the derivative)

$$w_j = w_j + \eta \sum_{i=1}^n y_i x_{ij} \exp(-y_i (w \cdot x_i + b))$$

# The “new” constant

$$c = \eta \exp(-y_i(w \cdot x_i + b))$$

learning rate      label      prediction

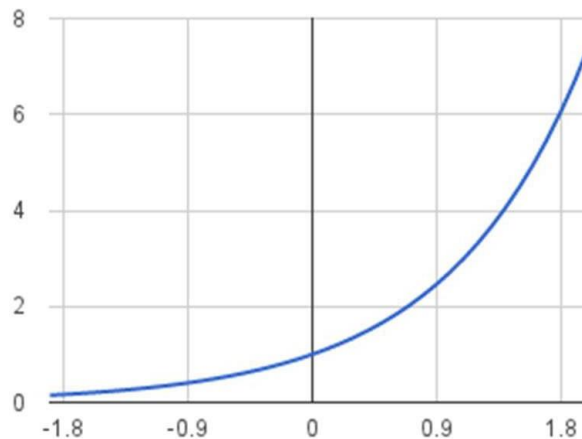
When is this large/small?

# The “new” constant

$$c = \eta \exp(-y_i(w \cdot x_i + b))$$

label

prediction



If prediction is correct (same sign),  
as the prediction gets larger, the  
update gets smaller

If the prediction is incorrect  
(different sign), the more different  
they are, the bigger the update



# Recap

Model-based machine learning:

- define a model, objective function (i.e. loss function), minimization algorithm

Gradient descent minimization algorithm

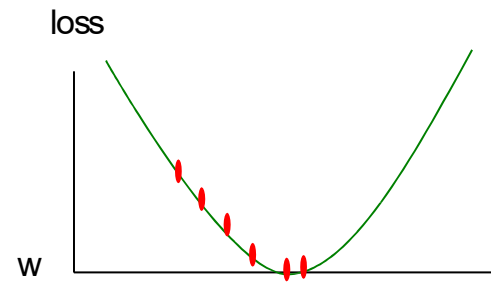
- requires that our loss function is convex
- make small updates towards lower losses

Perceptron learning algorithm:

- gradient descent
- exponential loss function (modulo a learning rate)

# Overfitting?

- We're calculating min weights on the **training set**
- We still need to be careful about overfitting!



# Overfitting : Regularization

- A **regularizer** is an additional criterion to the loss function to avoid overfitting
- Called a *regularizer* since it tries to keep the parameters regular (i.e., normal)
- It is a bias on the model that forces the learning to prefer certain types of weights over others

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \operatorname{loss}(y y') + \lambda \operatorname{regularizer}(w, b)$$

# Regularizers

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \operatorname{loss}(yy') + \lambda \operatorname{regularizer}(w,b)$$

□ □ □

- What should be our preference for weights?
  - Larger? Smaller?
- What makes for a “simpler” model for a linear model?

# Regularizers

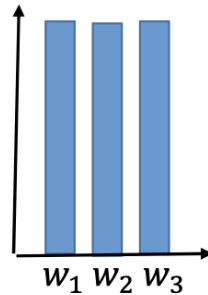
$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \operatorname{loss}(yy') + \lambda \operatorname{regularizer}(w,b)$$

- Generally, we don't want very large weights
- With large weights, small change in a feature value can result in a large change in the prediction
- Large weights may also give too much weight to certain features
- Might also prefer weights of 0 for features that aren't useful
- How do we encourage small weights? or penalize large weights?

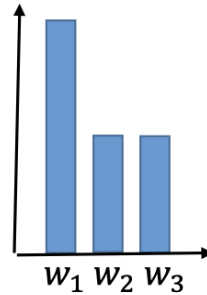
# Common regularizers

$$r(w, b) = \sum_{w_j} |w_j|$$

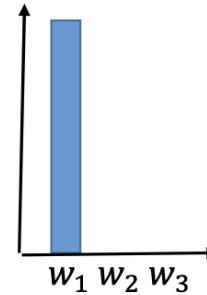
sum of the weights



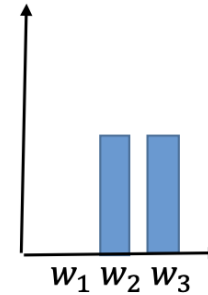
12



8



4



4

$$r(w, b) = \sqrt{\sum_{w_j} |w_j|^2}$$

sum of the squared weights

6.9

4.9

4

2.8

Squared weights penalizes large values more

# Linear Models w/ Regularization

## Linear Models w/ Regularization

1. pick a model

$$0 = b + \sum_{j=1}^n w_j f_j$$

2. pick a criteria to optimize (aka objective function)

$$\sum_{i=1}^n \text{loss}(y y') + \lambda \text{regularizer}(w)$$

3. develop a learning algorithm

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \text{loss}(y y') + \lambda \text{regularizer}(w)$$

Find  $w$  and  $b$   
that minimize

# Minimizing with a Regularizer

We know how to solve convex minimization problems using gradient descent:

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \operatorname{loss}(yy')$$

If we can ensure that the loss + regularizer is convex then we could still use gradient descent:

$$\operatorname{argmin}_{w,b} \underbrace{\sum_{i=1}^n \operatorname{loss}(yy') + \lambda \operatorname{regularizer}(w)}_{\text{make convex}}$$

If  $f$  and  $g$  are convex functions then so is  $f+g$



# Minimizing with a Regularizer

We know how to solve convex minimization problems using gradient descent:

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \operatorname{loss}(yy')$$

If we can ensure that the loss + regularizer is convex then we could still use gradient descent:

$$\operatorname{argmin}_{w,b} \underbrace{\sum_{i=1}^n \operatorname{loss}(yy') + \lambda \operatorname{regularizer}(w)}_{\text{convex as long as both loss and regularizer are convex}}$$

p-norms are convex  
for  $p \geq 1$

# Linear Models + Surrogate Loss + Regularizer

1. pick a model

$$0 = b + \sum_{j=1}^n w_j f_j$$

2. pick a criteria to optimize (aka objective function)

$$\sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2} \|w\|^2$$

3. develop a learning algorithm

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2} \|w\|^2$$

Find  $w$  and  $b$   
that minimize

# Gradient descent (same as before)

- pick a starting point ( $w$ )
- repeat until loss doesn't decrease in any dimension:
  - pick a dimension
  - move a small amount in that dimension towards decreasing loss (using the derivative)

$$w_j = w_j - \eta \frac{d}{dw_j} (\text{loss}(w) + \text{regularizer}(w, b))$$

---

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2} \|w\|^2$$

# Gradient descent

- pick a starting point ( $w$ )
- repeat until loss doesn't decrease in any dimension:
  - pick a dimension
  - move a small amount in that dimension towards decreasing loss (using the derivative)

$$w_j = w_j - \eta \frac{d}{dw_j} (\text{loss}(w) + \text{regularizer}(w, b))$$

---

$$w_j = w_j + \eta \sum_{i=1}^n y_i x_{ij} \exp(-y_i (w \cdot x_i + b)) - \eta \lambda w_j$$

# Regularizers summarized

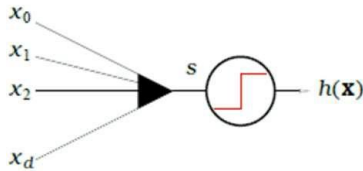
- L1 is popular because it tends to result in sparse solutions (i.e. lots of zero weights)
  - However, it is not differentiable, so it only works for gradient descent solvers
- L2 is also popular because for some loss functions, it can be solved directly (no gradient descent required)

# Logistic Regression

$$s = \sum_{i=0}^d w_i x_i$$

linear classification

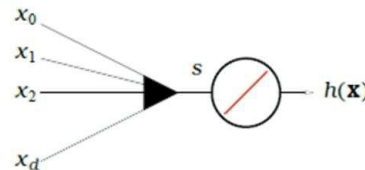
$$h(\mathbf{x}) = \text{sign}(s)$$



“hard” decision

linear regression

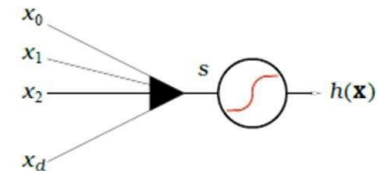
$$h(\mathbf{x}) = s$$



no decision

logistic regression

$$h(\mathbf{x}) = \theta(s)$$



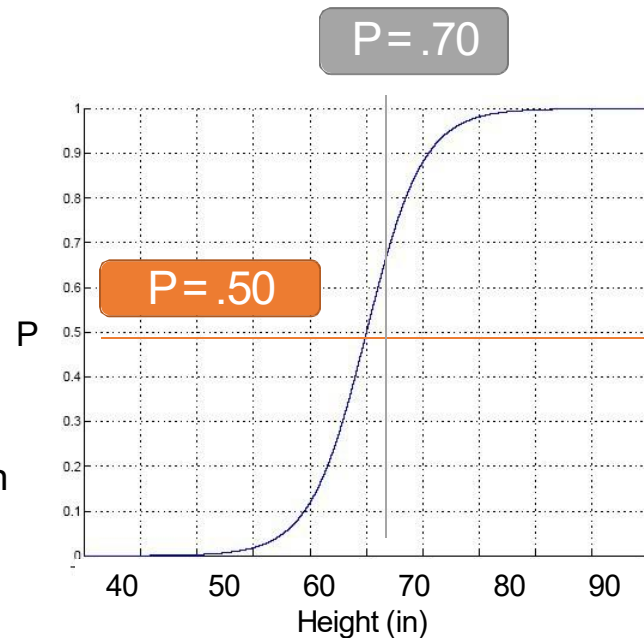
“soft” decision

# Probability Interpretation

- $h(\mathbf{x}) = \theta(s)$  is interpreted as a probability
- Example: Prediction of heart attacks
  - Input  $\mathbf{x}$ : cholesterol, age, weight, etc.
  - The signal,  $s = \mathbf{w}^T \mathbf{x}$  is “risk score”
    - Linear regression
  - $\theta(s)$  is “probability of a heart attack”
    - Logistic regression

# Simple (1D) Example

- Gender Classification
  - Input: height
  - Output: male (1) or female (0)
- Details...
  - Regression line is a rolling average
    - Similar to linear regression
  - Regression line is nonlinear
  - Data doesn't actually fall on regression line
    - Each sample is male (1) or female (0)



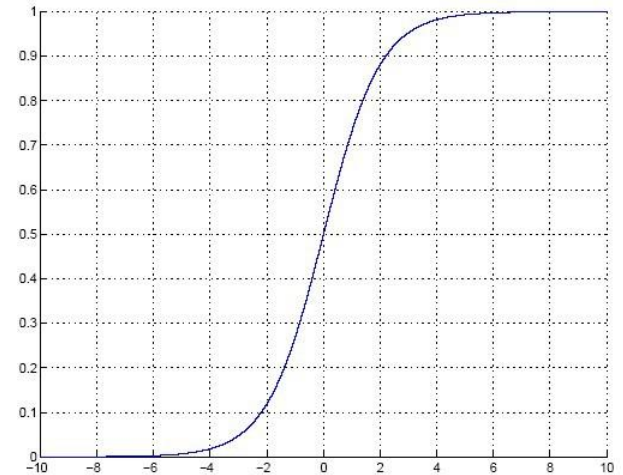


# Logistic Curve

- Relates the input,  $X$ , to the rolling mean  $P$

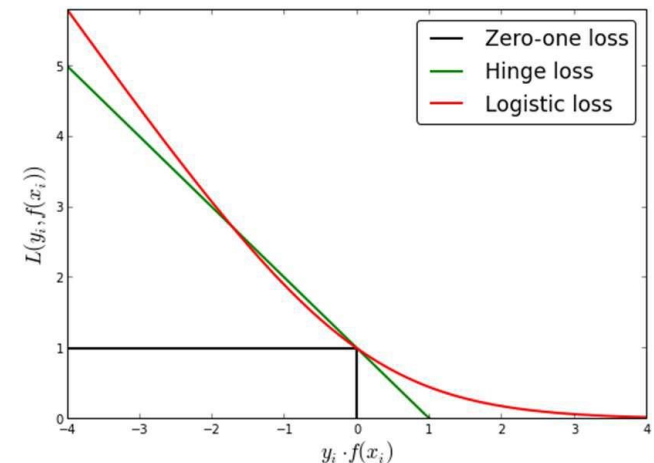
$$P = \frac{e^{\mathbf{w}^T \mathbf{x}}}{1 + e^{\mathbf{w}^T \mathbf{x}}}$$

- $P$  is the probability of a 1
- $e$  is the base of the natural logarithm ( $\sim 2.718$ )
- $w$  is the vector of parameters of the model



# Logistic Regression Loss Functions

- Two common loss functions for “logistic regression”
- Logistic loss ( $y = -1/+1$ )
  - $l(y, y') = \frac{1}{\ln} \ln(1 + e^{-yy'})$
- Cross-Entropy loss ( $y = 0/1$ )
  - $l(y, y') = -y \ln y' - (1 - y) \ln(1 - y')$



# Logistic Regression Algorithm

- 1: Initialize the weights at  $t = 0$  to  $\mathbf{w}(0)$
- 2: **for**  $t = 0, 1, 2, \dots$  **do**
- 3:   Compute the gradient

$$\nabla E_{\text{in}} = -\frac{1}{N} \sum_{n=1}^N \frac{y_n \mathbf{x}_n}{1 + e^{y_n \mathbf{w}^T(t) \mathbf{x}_n}}$$

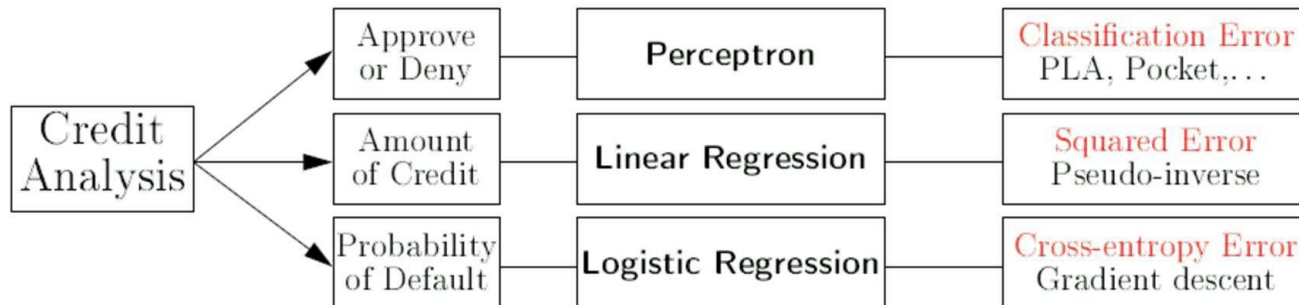
- 4:   Update the weights:  $\mathbf{w}(t + 1) = \mathbf{w}(t) - \eta \nabla E_{\text{in}}$
- 5:   Iterate to the next step until it is time to stop
- 6: Return the final weights  $\mathbf{w}$

Gradient Descent

# Logistic Regression Summary

- Straightforward extensions for:
  - Multiclass (unordered categories)
  - Ordinal (ordered categories)
- LogReg returns *maximum likelihood estimate*
  - When interpreted as probabilities

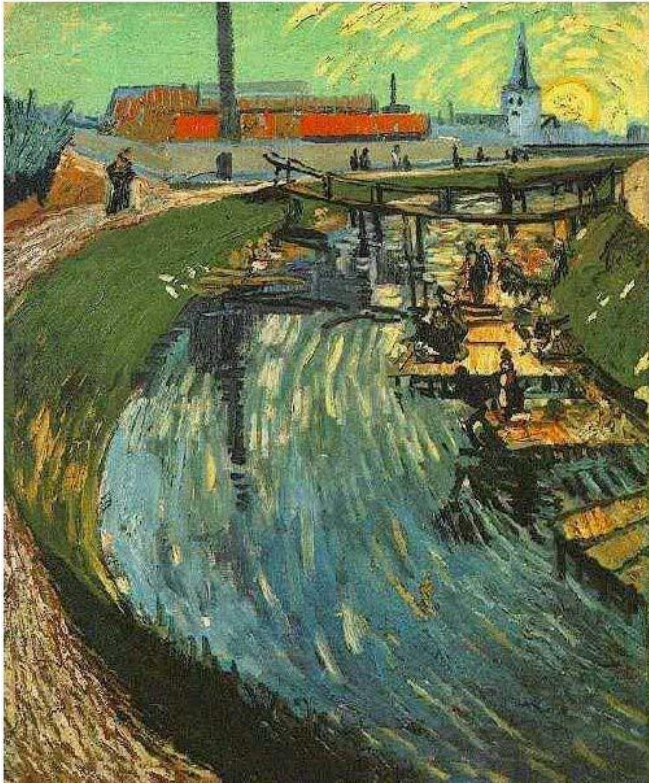
# Summary of Linear Models



# Artificial Neural Networks & Backpropagation

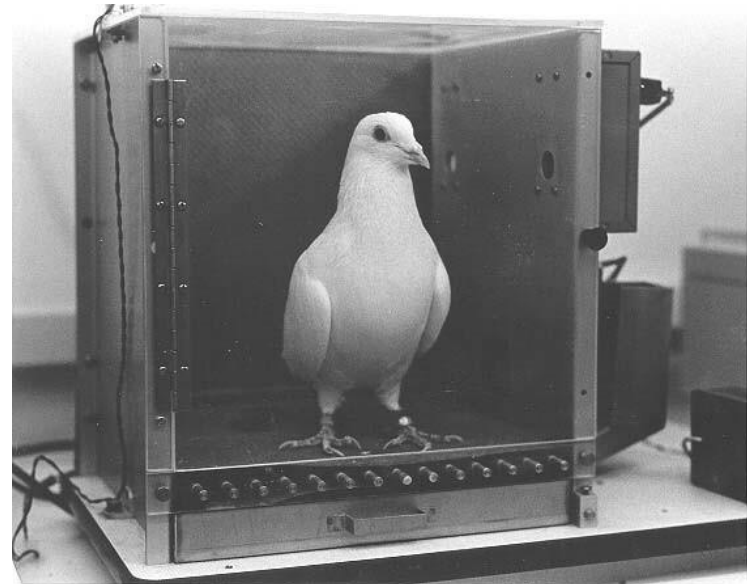
Slides adapted from Torsten Reil, Andrew Rosenberg, Fei-Fei Li, Andrej Karpathy, Justin Johnson and Etham Alpaydin

Can you name the  
artist?



# Biological Neural Nets

- Pigeons as art experts  
(Watanabe *et al.* 1995)
- Experiment:
  - Pigeon in Skinner box
  - Present paintings of two different artists  
(e.g. Chagall / Van Gogh)
  - Reward for pecking  
when presented a  
particular artist





# Pigeon-Art Experiment Results

- Pigeon in-sample error of 5%
  - presented with pictures they had been trained on
- Out of sample error of 15%
  - New, unseen paintings of Chagall / Van Gogh
- *Pigeons must be learning something (not memorization)*
  - *Extract and recognise patterns (the 'style')*
  - *Generalize from the already seen to make predictions*
- This is the idea behind neural networks (biological and artificial)

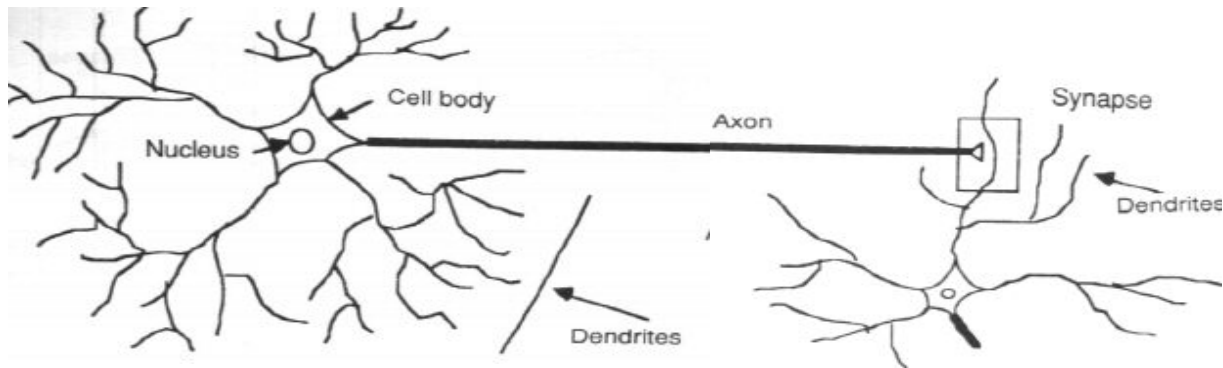
# Artificial Neural Networks (ANN)

- Popular set of learning techniques
  - Popularity ebbs and flows over time
  - Currently in a resurgence
- Common features of ANNs
  - Networks of simple elements
  - Inspired (loosely) by brain organization
  - Compact representation

# ANNs (cont'd)

- Highly parallel
  - Process information much more like the brain than a serial computer
- Simple principles → complex behaviours
- Applications
  - As powerful problem solvers
  - As biological models

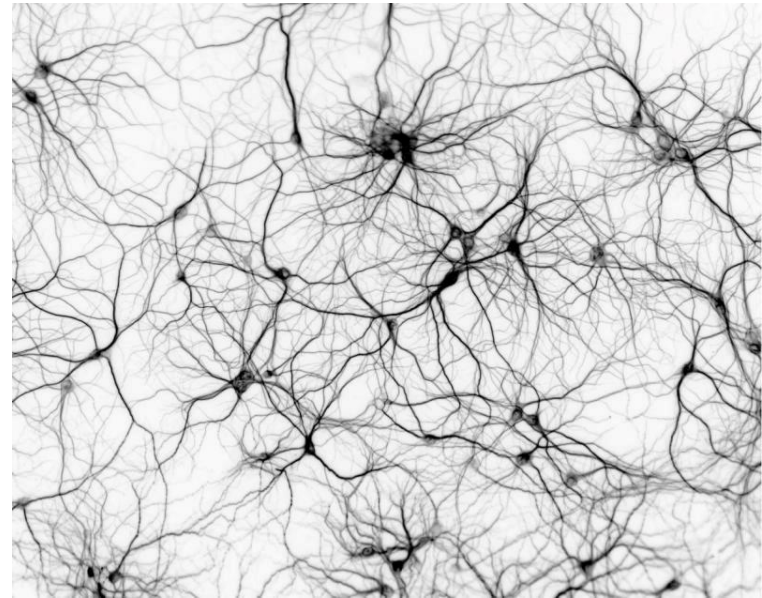
# Biological Neuron



- Signals arrive on **synapses** and enters cell through **dendrites**
  - (Relatively slow) electro-chemical process
- Raises electrical potential of cell body
  - If potential is high enough, the cell **fires** pulse along **axon**
- Process continues at synapse of another neuron

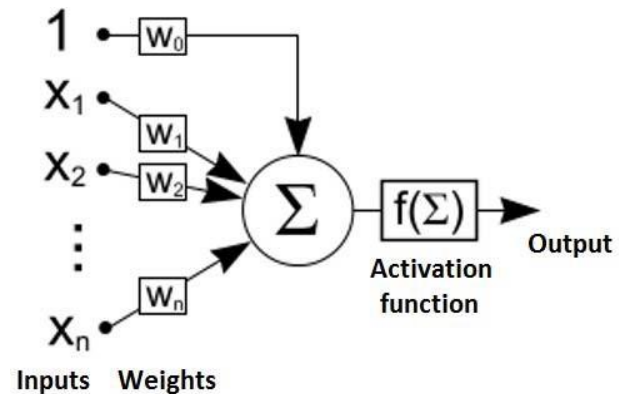
# Biological Neural Network

- Single neuron
  - Relatively simple
- Neural network
  - Many neurons
  - Complex connectivity
  - Basis for everything we do, remember, etc.



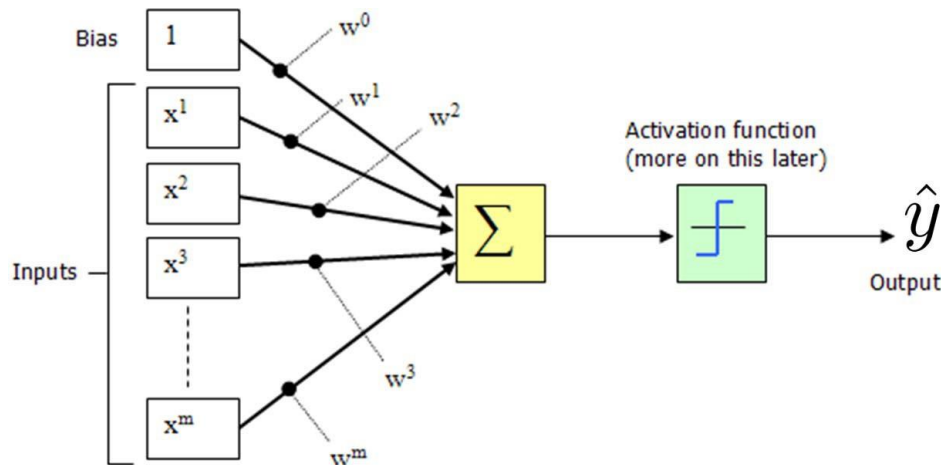
# Artificial Neuron

- McCulloch-Pitts (1943) proposed a computational model
  - Later extended to **perceptron**
- Output depends on:
  - Inputs  $\{0, 1\}$
  - Weights
  - Threshold (activation)
- Binary output
  - If (weighted sum of inputs) is greater than threshold



# Perceptron (Visually)

- m-dimensional input
- 1 output,  $\hat{y}$
- m+1 weights to learn
  - bias term + weights for each of m dimensions

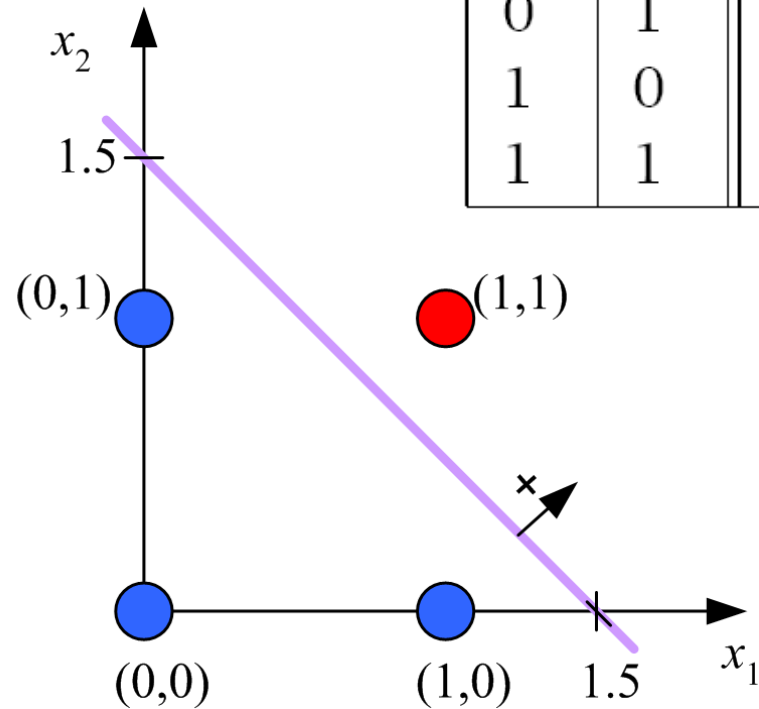
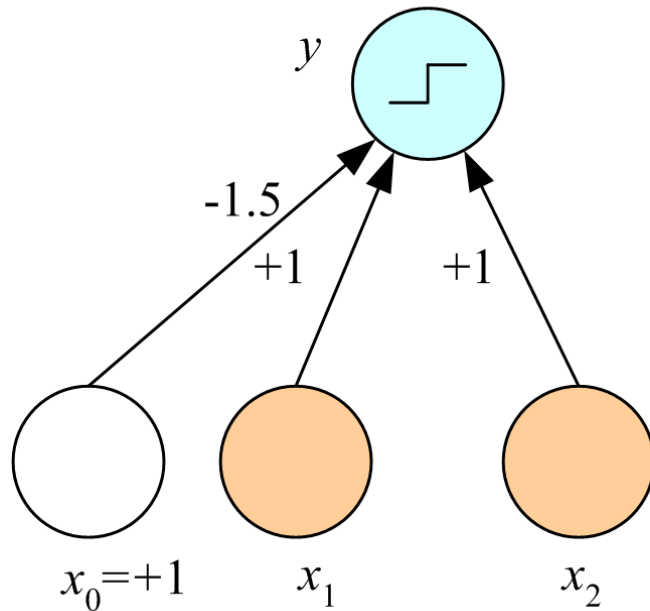


# Threshold Units

- Simple perceptrons  $f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$
- Very simple artificial neurons
- Note output is 0/1 instead of -1/+1
  - Corresponds to the notion of neuron “firing”



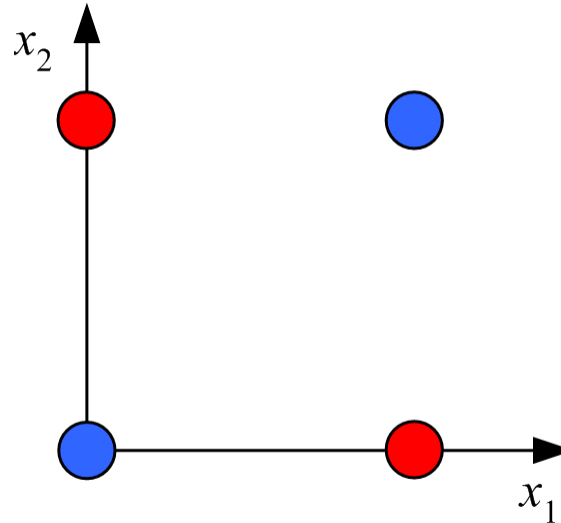
# Boolean AND (w/ Threshold unit)



| $x_1$ | $x_2$ | $r$ |
|-------|-------|-----|
| 0     | 0     | 0   |
| 0     | 1     | 0   |
| 1     | 0     | 0   |
| 1     | 1     | 1   |

# How about Boolean XOR?

| $x_1$ | $x_2$ | $r$ |
|-------|-------|-----|
| 0     | 0     | 0   |
| 0     | 1     | 1   |
| 1     | 0     | 1   |
| 1     | 1     | 0   |



- Need  $w_0, w_1, w_2$  to satisfy:

$$w_0 \leq 0$$

$$w_2 + w_0 > 0$$

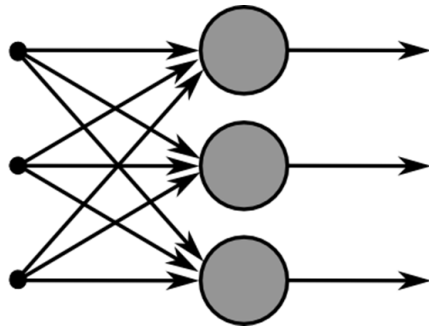
$$w_1 + w_0 > 0$$

$$w_1 + w_2 + w_0 \leq 0$$

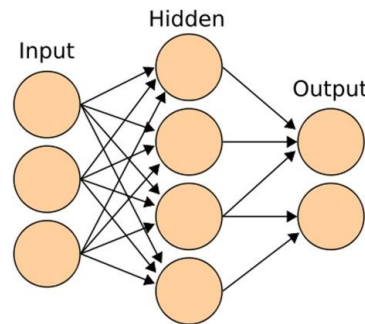
• No set of weights will satisfy these equations.

# Multi-Layer Networks

- Single perceptrons cannot compute certain simple functions, like XOR
- We can overcome this using multiple layers



One-Layer Perceptron

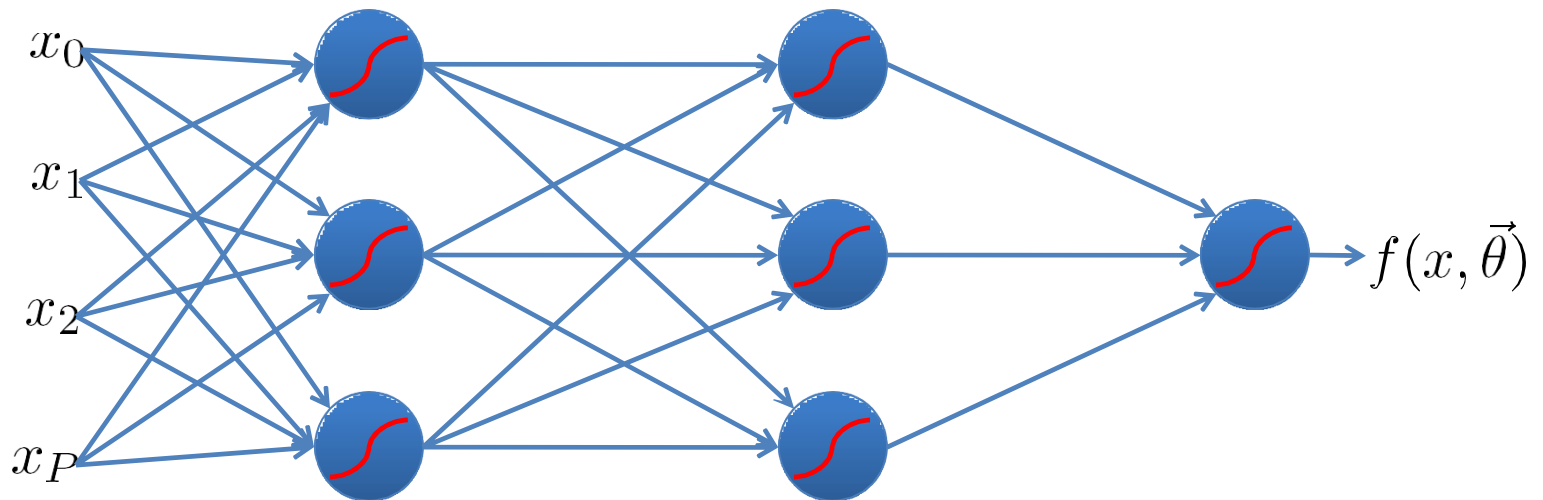


Two-Layer Perceptron

Number of layers may or may not count input layer (usually doesn't)

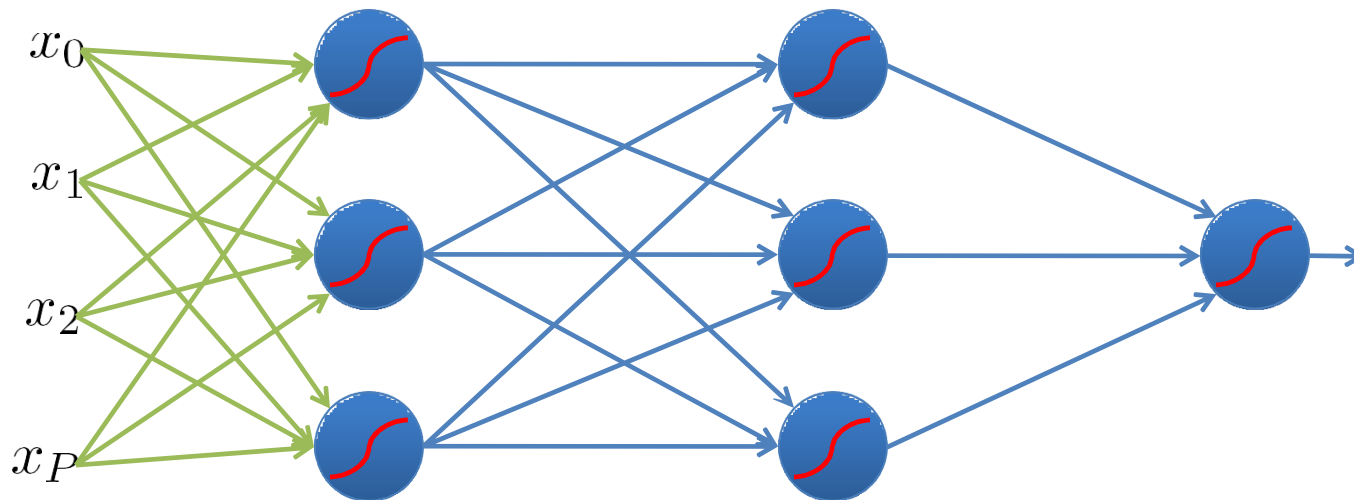
# Multilayer Networks

- Cascade Neurons together
- The output from one layer is the input to the next
- Each Layer has its own sets of weights



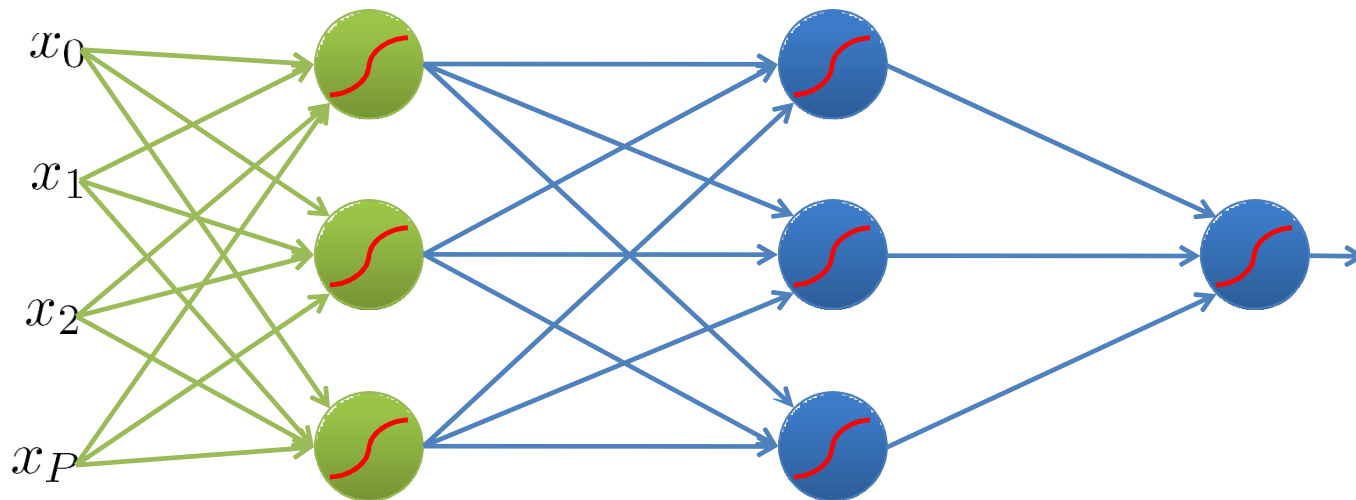
# Feed-Forward Networks

- Predictions are fed forward through the network to classify



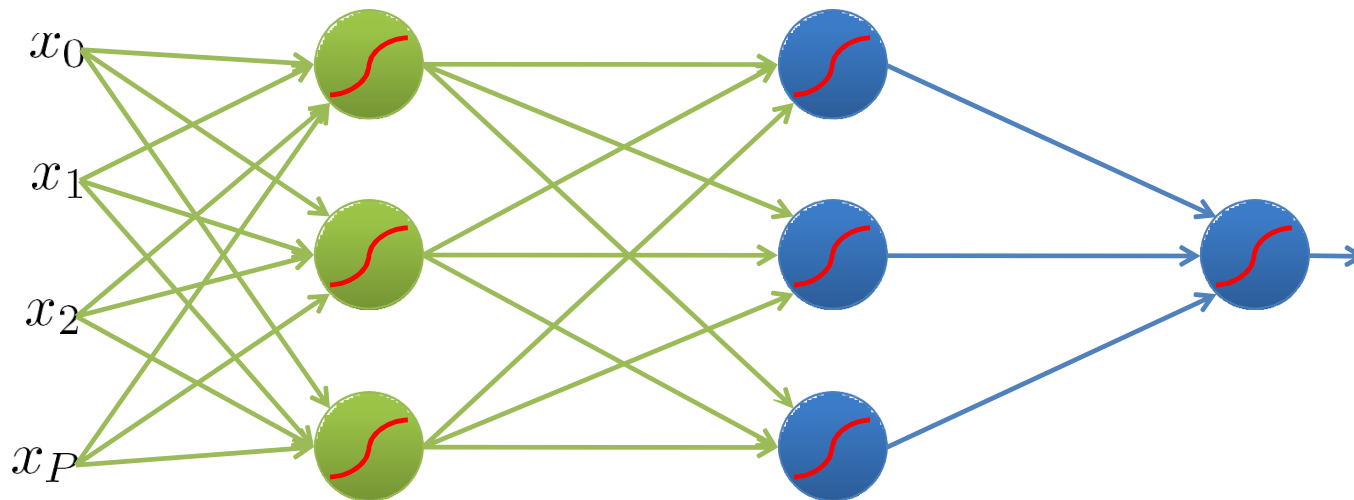
# Feed-Forward Networks

- Predictions are fed forward through the network to classify



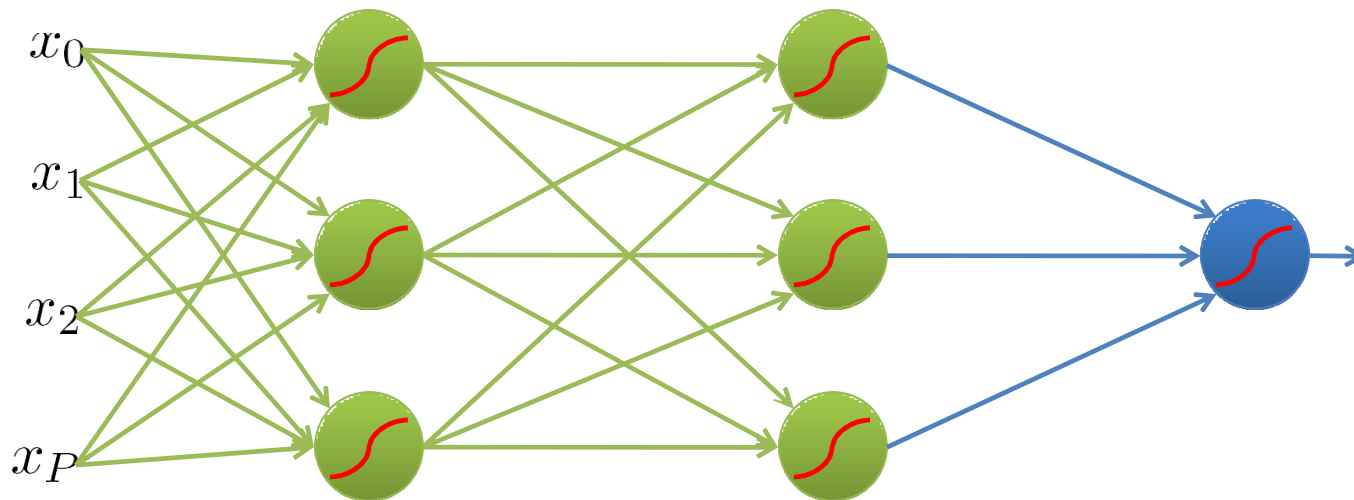
# Feed-Forward Networks

- Predictions are fed forward through the network to classify



# Feed-Forward Networks

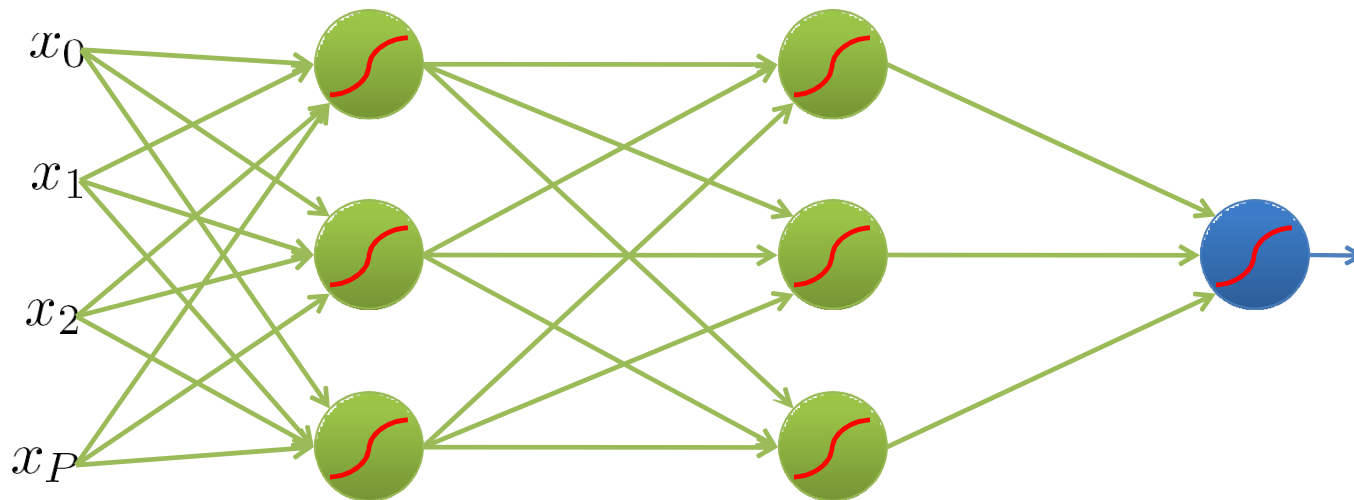
- Predictions are fed forward through the network to classify





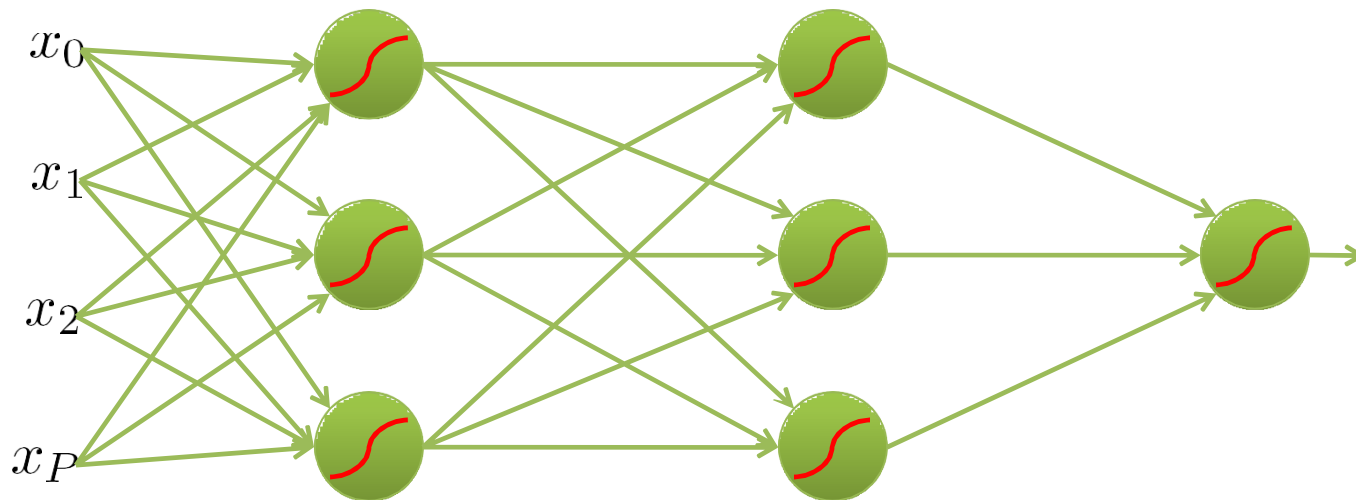
# Feed-Forward Networks

- Predictions are fed forward through the network to classify



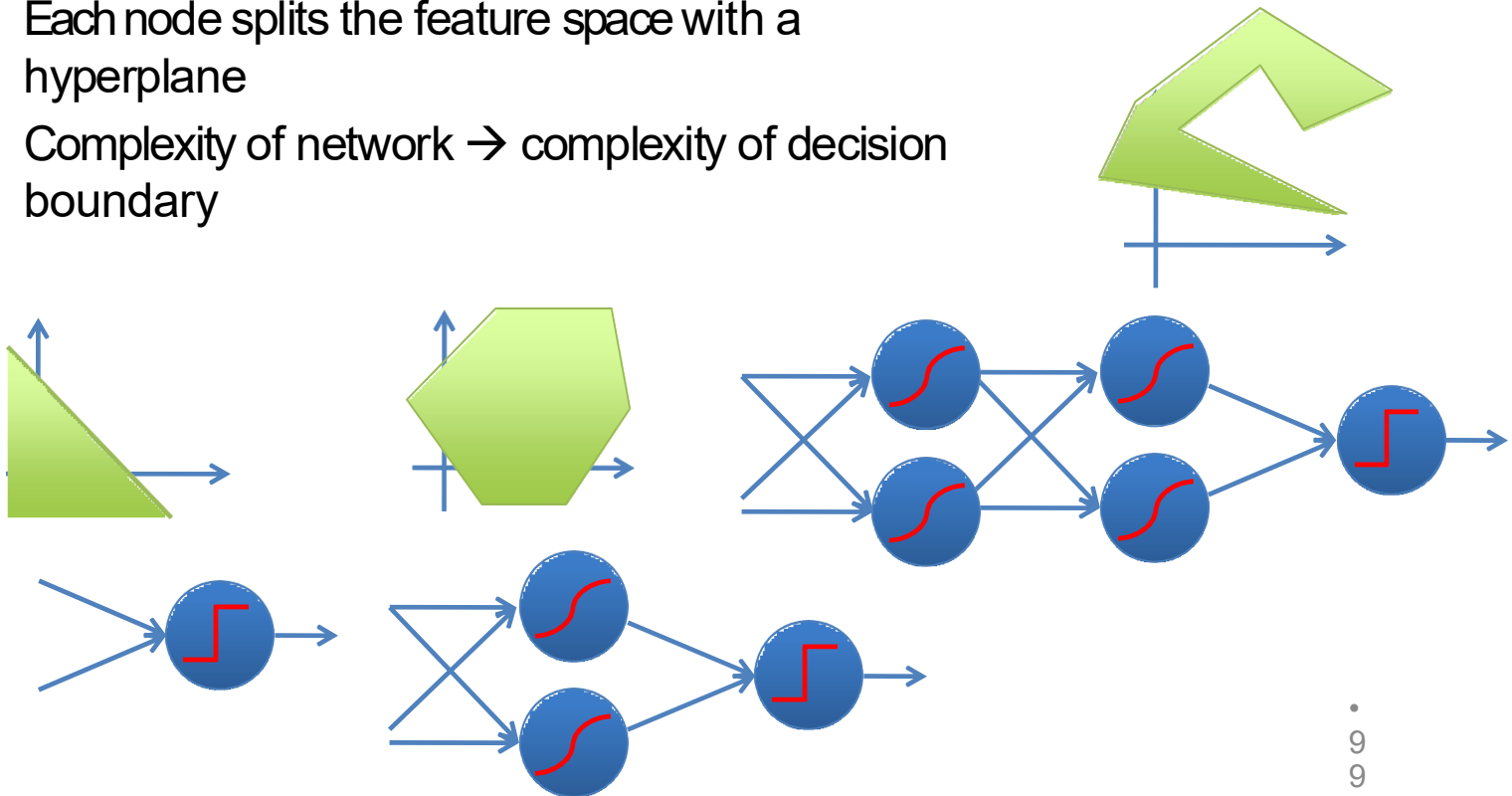
# Feed-Forward Networks

- Predictions are fed forward through the network to classify



# Multilayer Networks in Feature Space

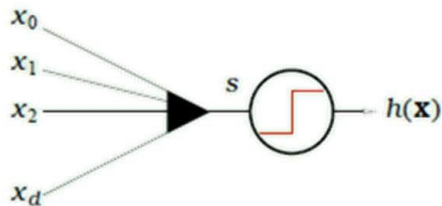
- More layers can handle more complicated spaces
  - but require more parameters
- Each node splits the feature space with a hyperplane
- Complexity of network  $\rightarrow$  complexity of decision boundary



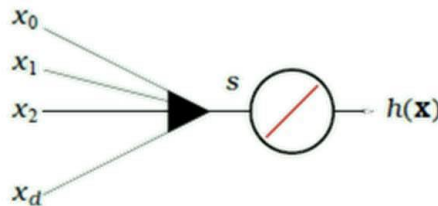
# Many Types of Units

- Perceptrons can do more than thresholding
  - Linear Regression
  - Logistic Regression
  - Many other nonlinear, differentiable functions

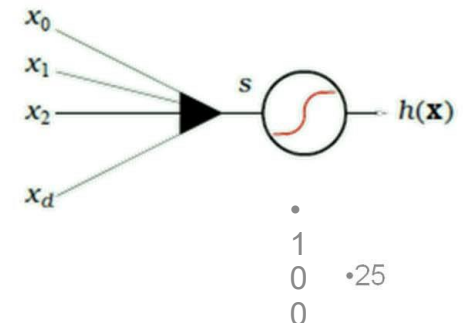
$$h(\mathbf{x}) = \text{sign}(s)$$



$$h(\mathbf{x}) = s$$



$$h(\mathbf{x}) = \theta(s)$$



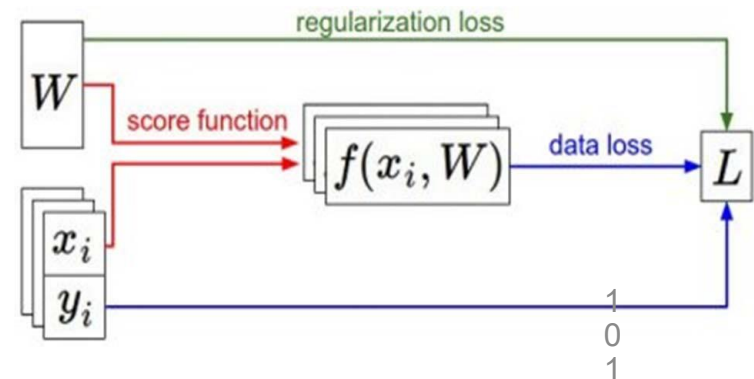
# Generic Perceptron Training

- We have some **dataset**:  $(X, Y)$
- We have a **score function**:  $s = f(x; W) \stackrel{\text{e.g.}}{=} Wx$
- We have a **loss function**
  - with (possibly) regularization
- **Gradient Descent**

Hinge:  $l(y, y') = \max(0, 1 - yy')$

Exponential:  $l(y, y') = \exp(-yy')$

Squared loss:  $l(y, y') = (y - y')^2$



# Gradient Descent

- pick a starting point ( $w$ )
- repeat until loss doesn't decrease in any dimension:
  - pick a dimension
  - move a small amount in that dimension towards decreasing loss (using the derivative)

$$w_j = w_j - \eta \frac{d}{dw_j} \text{loss}(w)$$

- Computing this derivative?

# Numerical Gradient

- In 1-dimension, the **derivative** of a function:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- In multiple dimensions, the **gradient** is the vector of partial derivatives

**current W:**

[0.34,  
-1.11,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25347**

**gradient dW:**

[?,  
?,  
?,  
?,  
?,  
?,  
?,  
?,  
?,  
?,...]



**current W:**

[0.34,  
-1.11,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25347**

**W + h (first  
dim):**

[0.34 + **0.0001**,  
-1.11,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25322**

**gradient dW:**

[?,  
?,  
?,  
?,  
?,  
?,  
?,  
?,  
?,...]

**current W:**

[0.34,  
-1.11,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25347**

**W + h (first  
dim):**

[0.34 + **0.0001**,  
-1.11,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25322**

**gradient dW:**

**[-2.5,**  
?,  
?,

$$(1.25322 - 1.25347)/0.0001 = -2.5$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?,  
?,...]

**current W:**

[0.34,  
-1.11,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25347**

**W + h (second dim):**

[0.34,  
-1.11 + **0.0001**,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25353**

**gradient dW:**

[-2.5,  
?,  
?,  
?,  
?,  
?,  
?,  
?,  
?,...]

**current W:**

[0.34,  
-1.11,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25347**

**W + h (second dim):**

[0.34,  
-1.11 + **0.0001**,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25353**

**gradient dW:**

[-2.5,  
**0.6**,  
?,  
?,

$$(1.25353 - 1.25347)/0.0001 = 0.6$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?,...]

**current W:**

[0.34,  
-1.11,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25347**

**W + h (third dim):**

[0.34,  
-1.11,  
0.78 + **0.0001**,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25347**

**gradient dW:**

[-2.5,  
0.6,  
?,  
?,  
?,  
?,  
?,  
?,  
?,...]

**current W:**

[0.34,  
-1.11,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25347**

**W + h (third dim):**

[0.34,  
-1.11,  
0.78 + **0.0001**,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25347**

**gradient dW:**

[-2.5,  
0.6,  
**0**,  
?,  
?

$$(1.25347 - 1.25347)/0.0001 = 0$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?,...]

# Evaluating the Gradient Numerically

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- approximate
- slow to evaluate
- All losses (L) are simply functions of the weights (W)
- We can compute derivative analytically

```
def eval_numerical_gradient(f, x):  
    """  
    a naive implementation of numerical gradient of f at x  
    - f should be a function that takes a single argument  
    - x is the point (numpy array) to evaluate the gradient at  
    """  
  
    fx = f(x) # evaluate function value at original point  
    grad = np.zeros(x.shape)  
    h = 0.00001  
  
    # iterate over all indexes in x  
    it = np.nditer(x, flags=['multi_index'], op_flags=['readwrite'])  
    while not it.finished:  
  
        # evaluate function at x+h  
        ix = it.multi_index  
        old_value = x[ix]  
        x[ix] = old_value + h # increment by h  
        fxh = f(x) # evaluate f(x + h)  
        x[ix] = old_value # restore to previous value (very important!)  
  
        # compute the partial derivative  
        grad[ix] = (fxh - fx) / h # the slope  
        it.iternext() # step to next dimension  
  
    return grad
```

**current W:**

[0.34,  
-1.11,  
0.78,  
0.12,  
0.55,  
2.81,  
-3.1,  
-1.5,  
0.33,...]

**loss 1.25347**

$dW = \dots$   
(some function of  
data and W)



**gradient dW:**

[-2.5,  
0.6,  
0,  
0.2,  
0.7,  
-0.5,  
1.1,  
1.3,  
-2.1,...]



# Summary

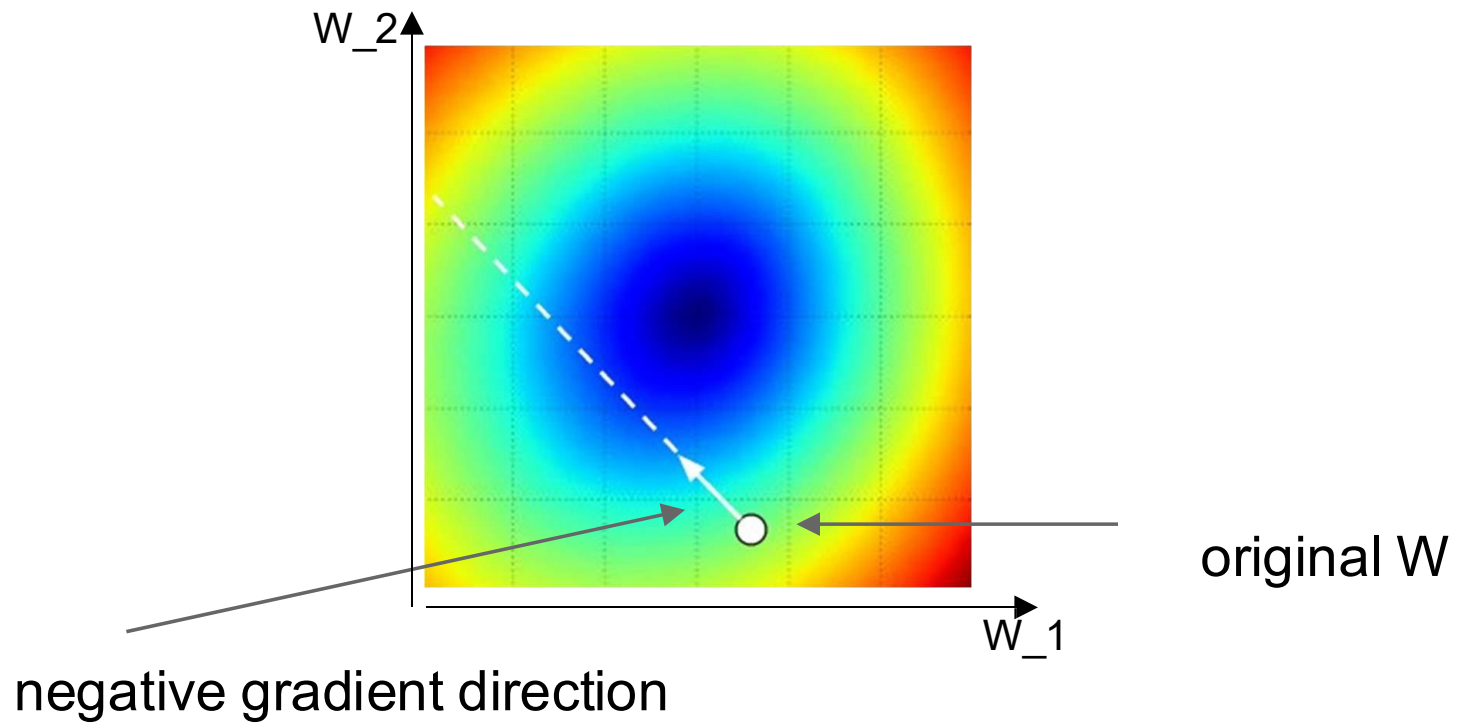
- Numerical gradient: approximate, slow, easy
- Analytic gradient: exact, fast, error-prone
- In practice: Use analytic gradient, but check correctness with numerical gradient.
  - This is called a **gradient check**

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```

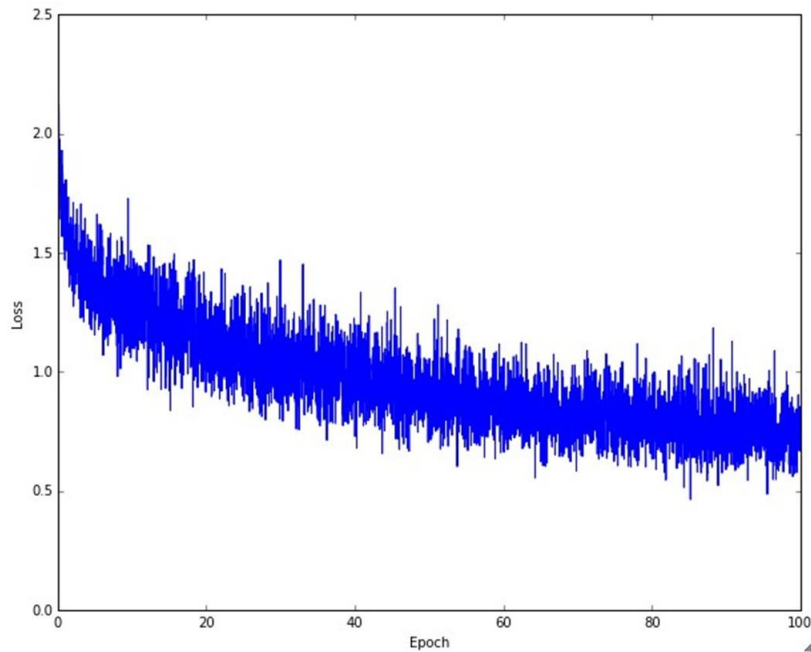


# Mini-batch Gradient Descent

- Only use a small portion of the training set to compute the gradient

```
# Vanilla Minibatch Gradient Descent  
  
while True:  
    data_batch = sample_training_data(data, 256) # sample 256 examples  
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)  
    weights += - step_size * weights_grad # perform parameter update
```

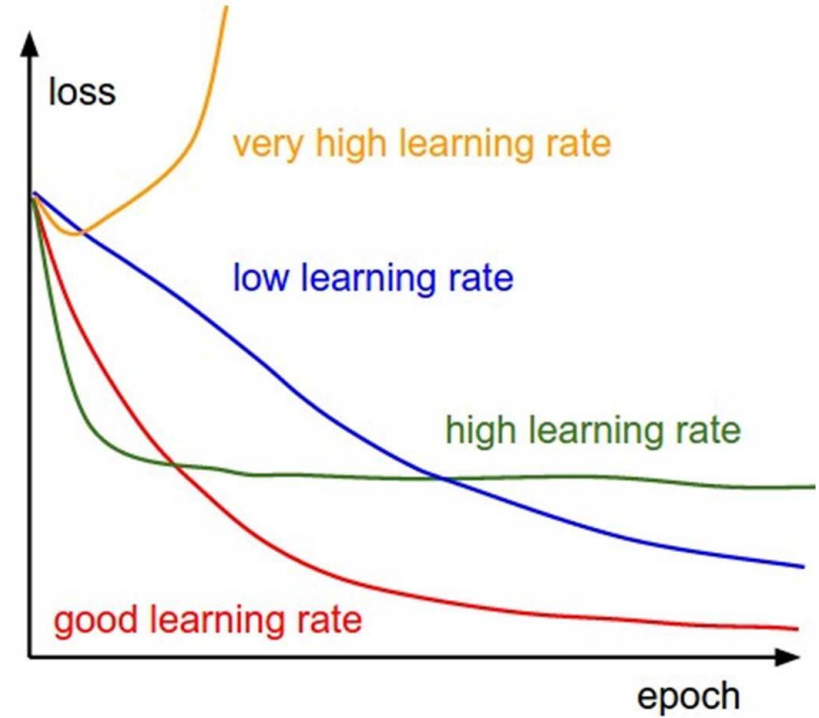
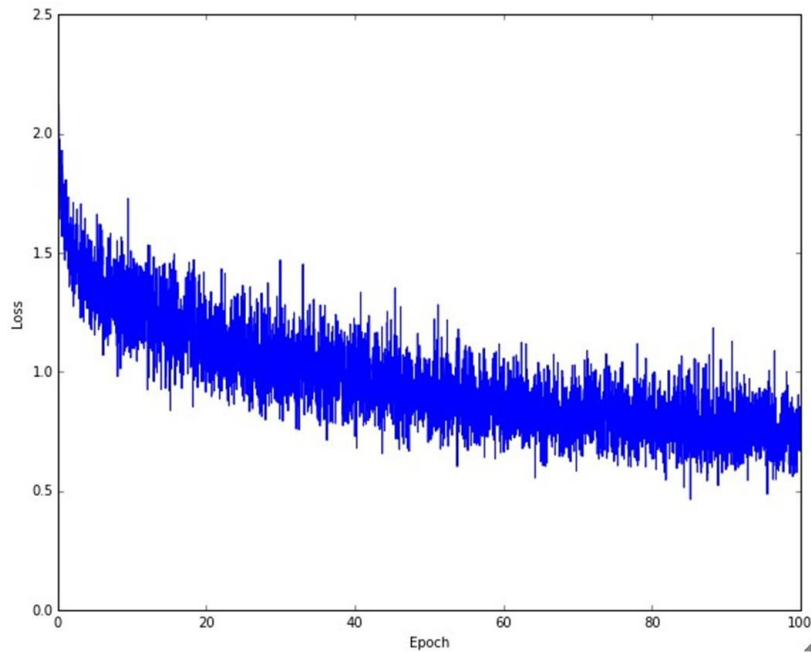
- Common mini-batch sizes are powers of 2 (e.g., 32, 64, 256)
- Most common approach to training NN



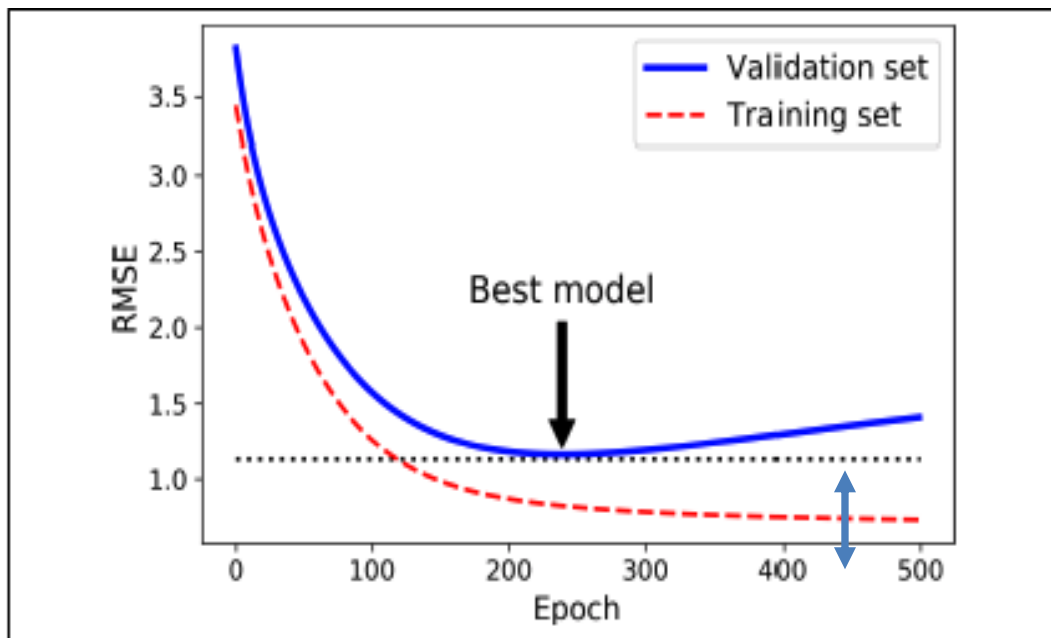
Example of optimization progress while training a neural network.

(Loss over mini-batches goes down over time.)

The effects of step size (or “learning rate



# Combating Overfitting with Early Stopping



big gap = overfitting

**Early stopping** is a form of regularization used to avoid overfitting during the training of a machine learning model.

Figure 4-20. Early stopping regularization

- What if bumpy? (e.g. stoch./mini-batch): keep track of the minimum validation loss encountered during training. When stopping the training, revert to the parameters from the epoch with the minimum validation loss to ensure the best model is selected.
- Set a patience parameter that allows the training to continue for a specified number of iterations after the minimum validation loss is reached. If the validation loss does not improve within this patience period, the training process should be stopped to prevent overfitting.

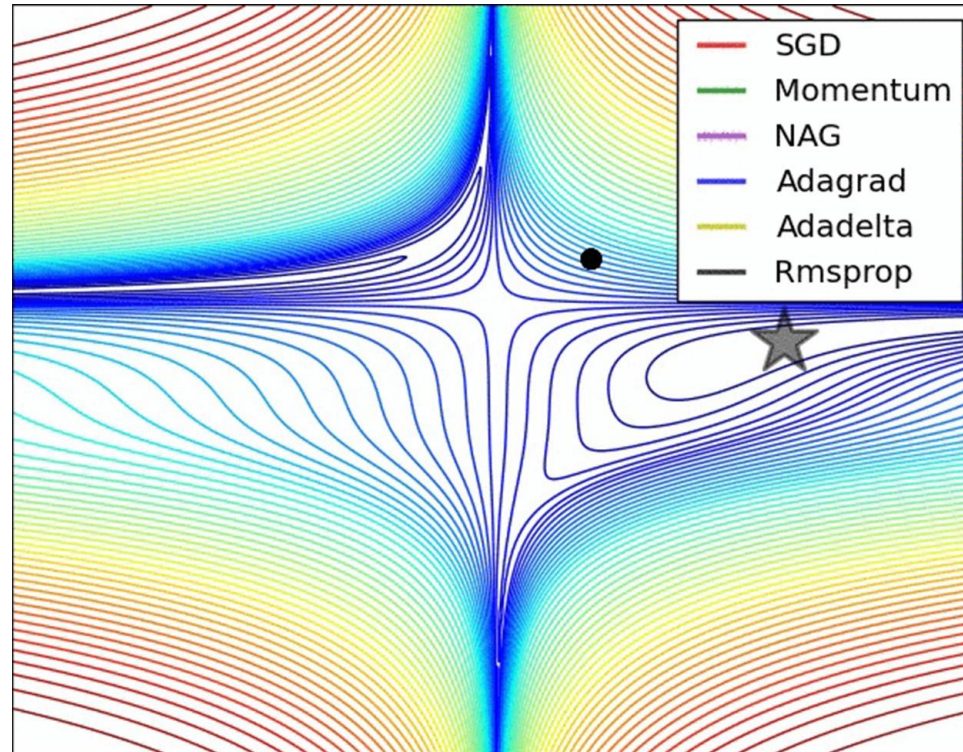
# Weight Update Step

```
# Vanilla Minibatch Gradient Descent

while True:
    data_batch = sample_training_data(data, 256) # sample 256 examples
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
    weights += - step_size * weights_grad # perform parameter update
```

- Can do something fancier than this vanilla update
  - Many algorithms (momentum, Adagrad, RMSProp, Adam, ...)

The effects of  
different  
update  
formulas



(image credits to Alec Radford)

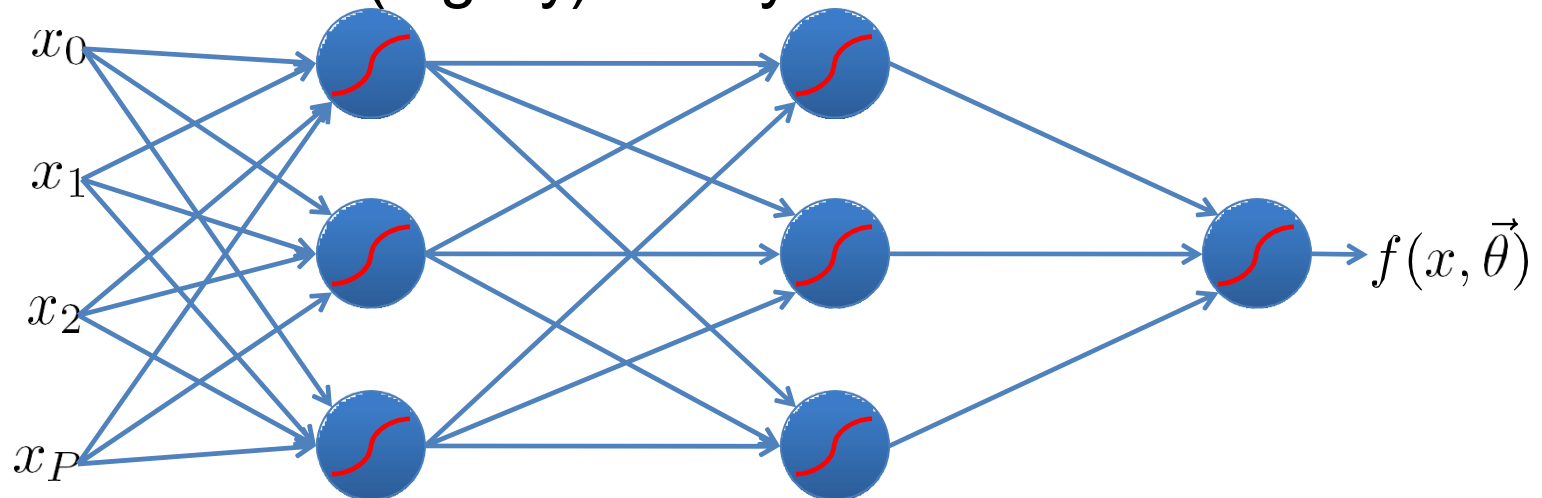


# What do we have so far...

- Components – biological plausibility
  - Neuron / node
  - Synapses / weight
- Perceptrons / Artificial Neurons
  - Threshold Units (boolean functions / classification)
  - Linear Units (regression)
  - Nonlinear Units
- Similarities across units...
  - All can be trained via gradient descent
  - Weights of the connections between nodes determine the behavior of a unit or network

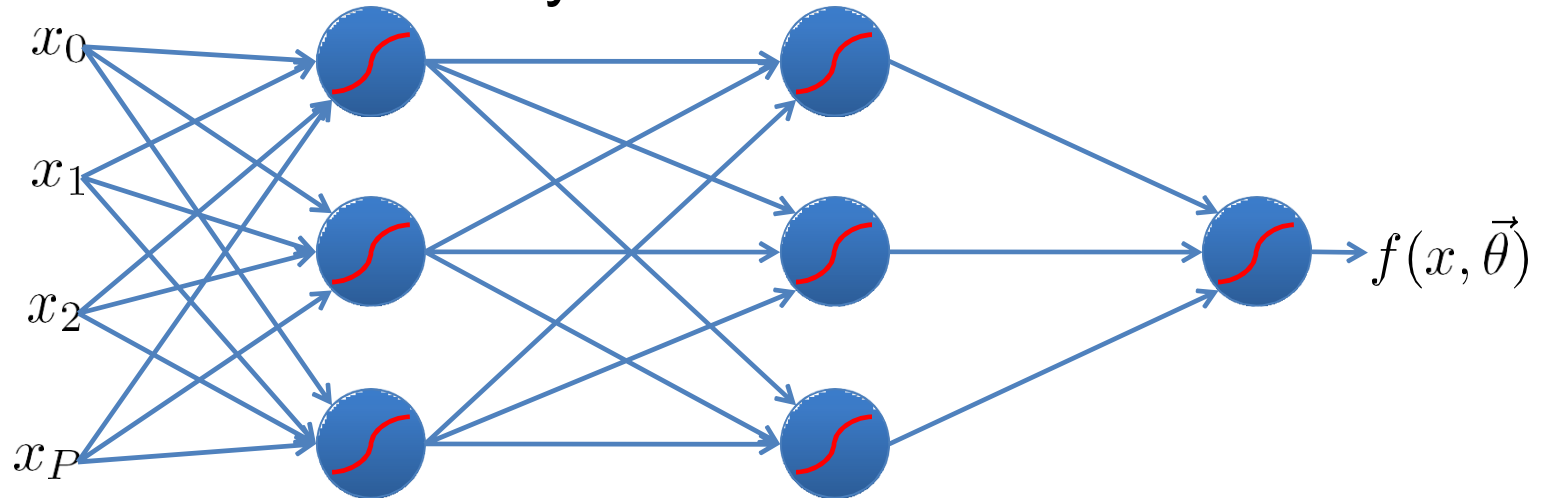
# How Do We Train MLPs?

- Need to learn weights for all of the edges
- PLA/GD works for 1-layer network
  - We will (slightly) modify this for MLP



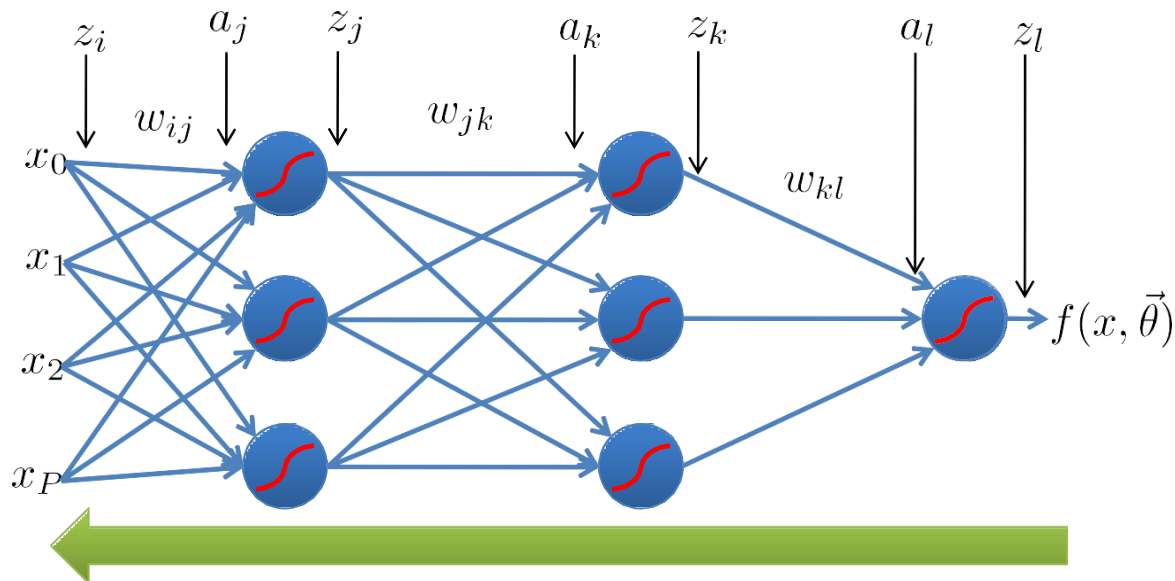
# Error Backpropagation

- Gradient descent on the whole network
- Training will proceed ***backwards***
  - From the last layer to the first



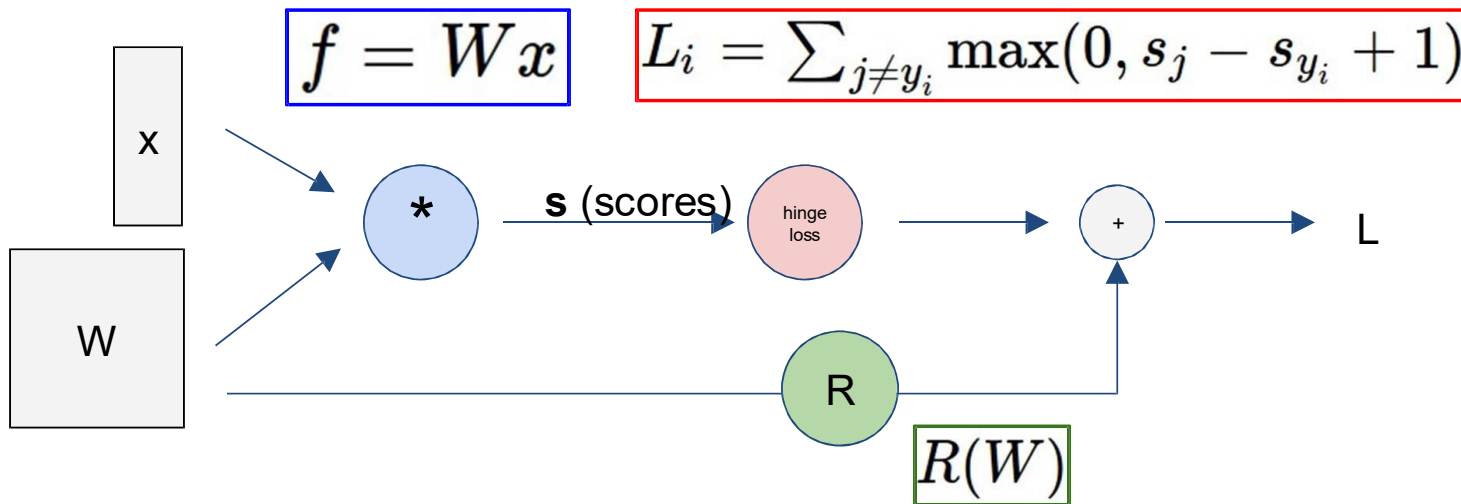
# Error Backpropagation

- Error backprop unravels the multivariate chain rule and solves the gradient for each partial component separately
- The target values for each layer come from the next layer
- This ***feeds the errors back*** along the network

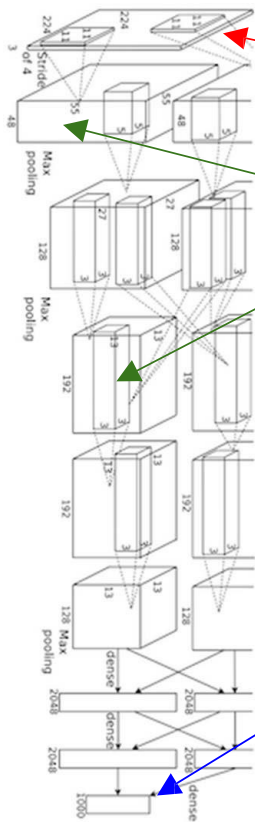


# NN as a Computational Graph

- Consider a neural network as a large computational graph



# Larger Computational Graphs

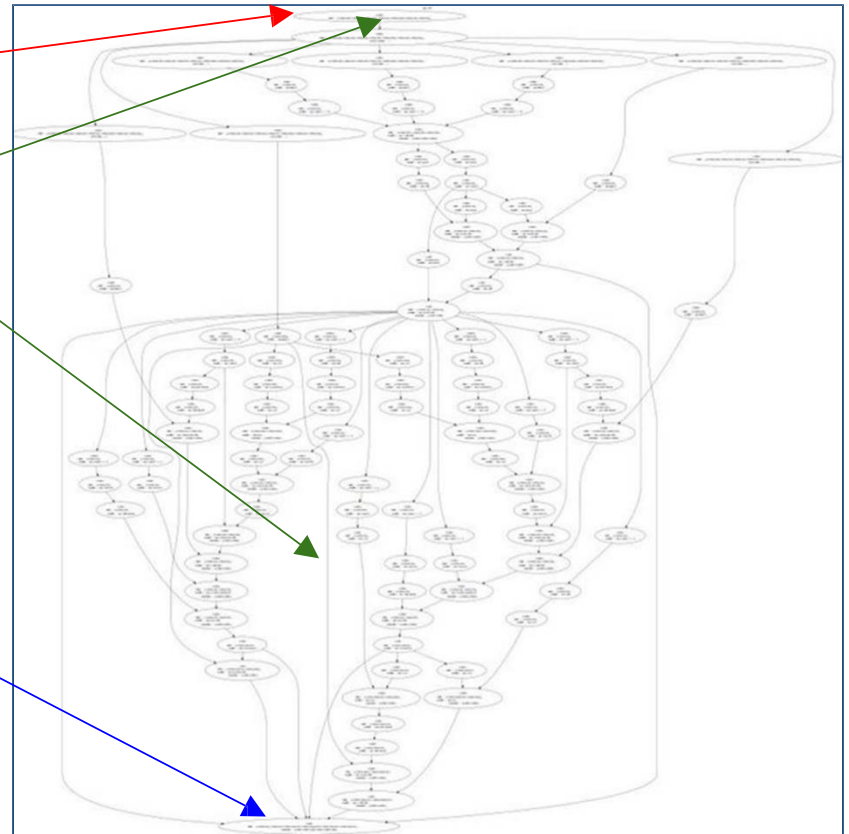


AlexNet

input

weights

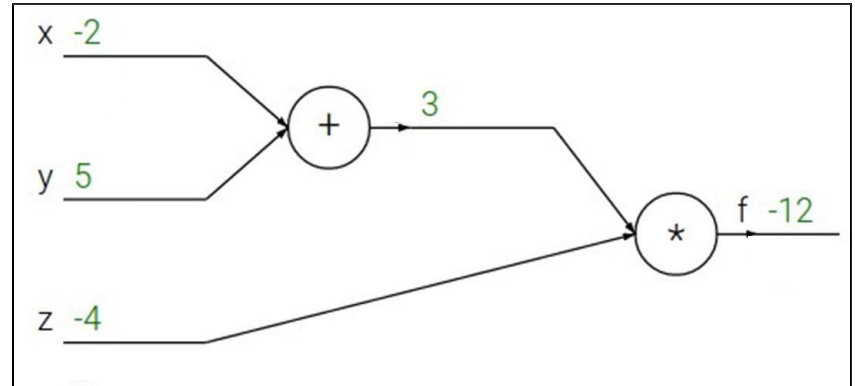
loss



Neural Turing Machine

$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2$ ,  $y = 5$ ,  $z = -4$



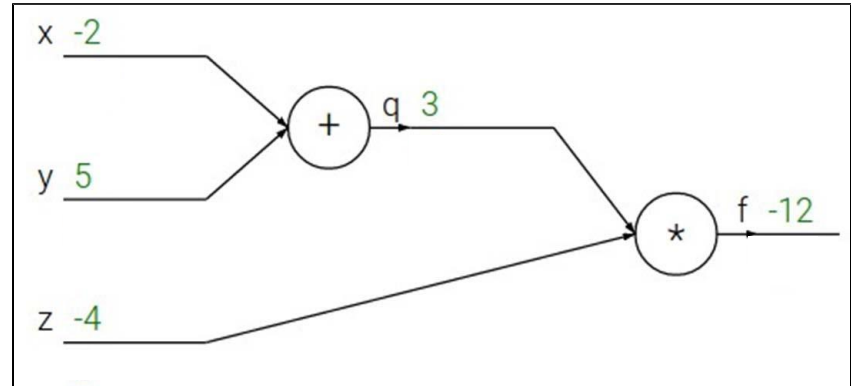
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$





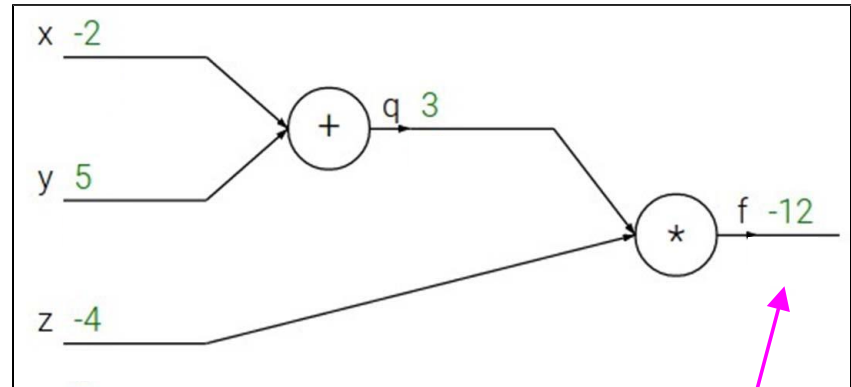
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial f}$$

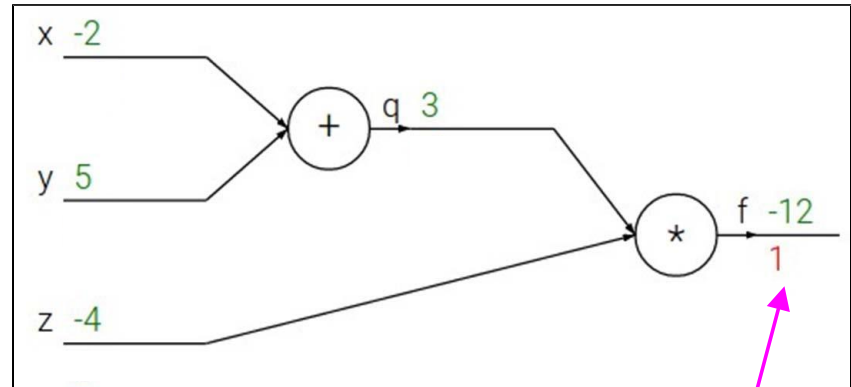
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial f}$$

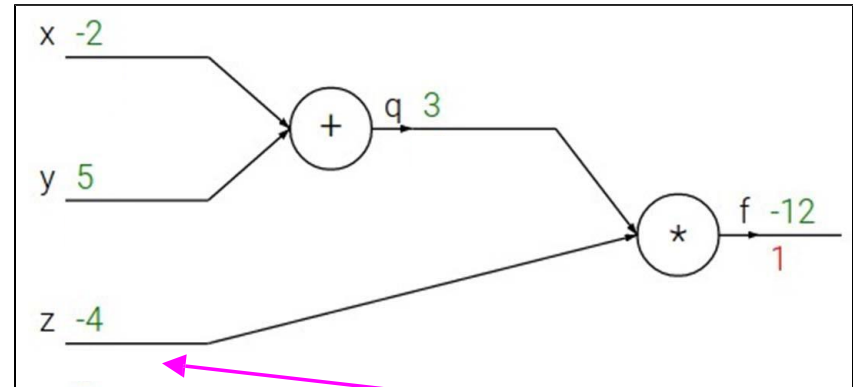
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial z}$$

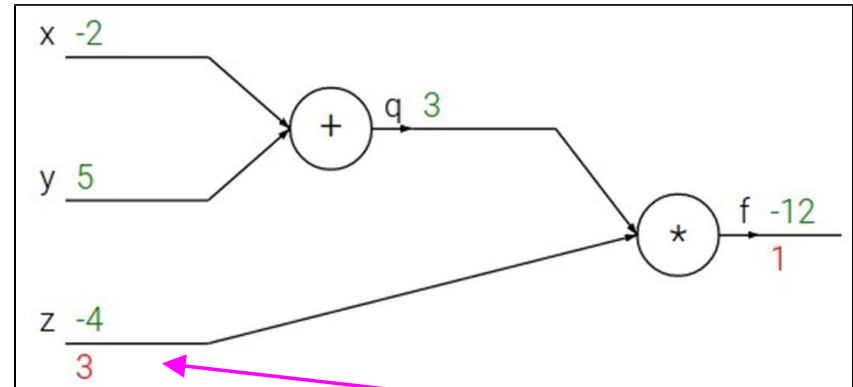
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial z}$$

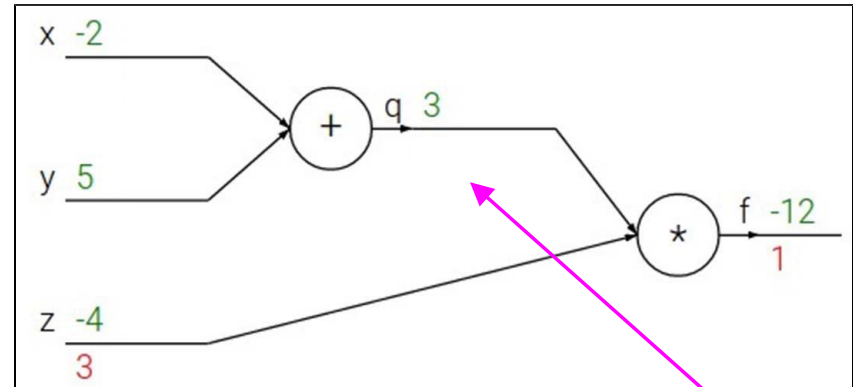
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial q}$$

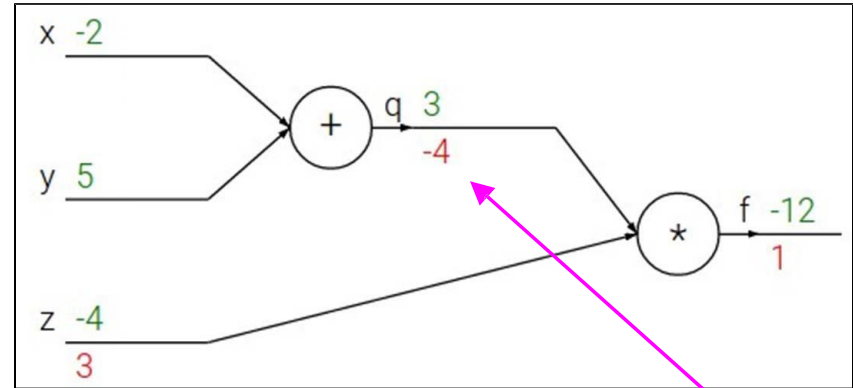
$$f(x, y, z) = (x + y)z$$

e.g.  $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial q}$$

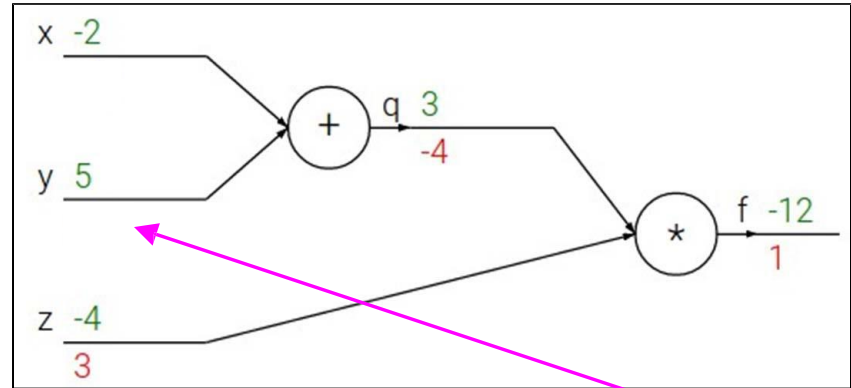
$$f(x, y, z) = (x + y)z$$

$$\text{e.g. } x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial y}$$

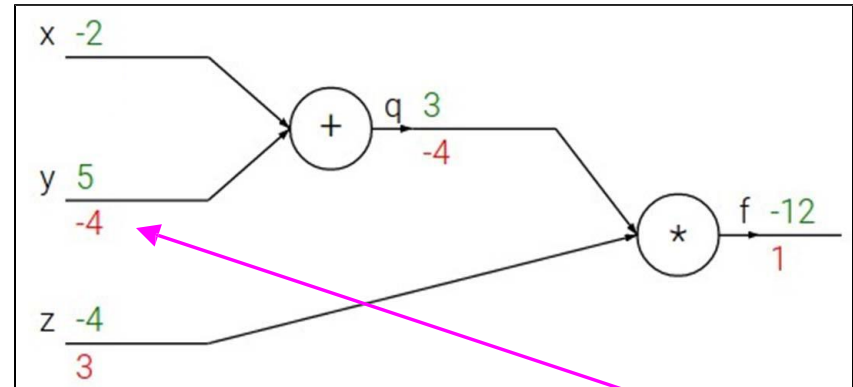
$$f(x, y, z) = (x + y)z$$

$$\text{e.g. } x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

$$\frac{\partial f}{\partial y}$$



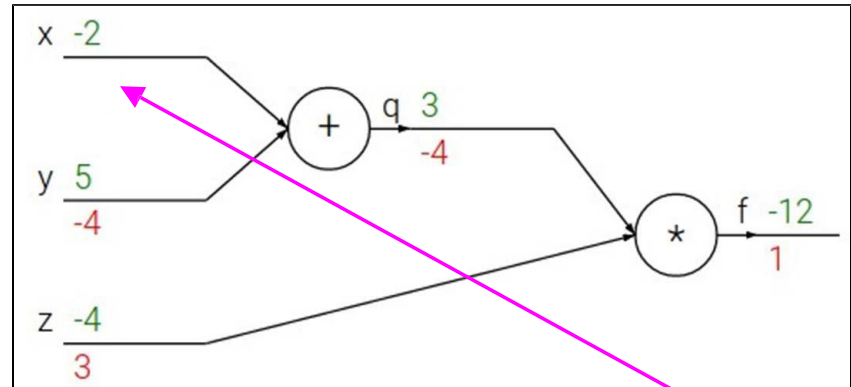
$$f(x, y, z) = (x + y)z$$

$$\text{e.g. } x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial x}$$

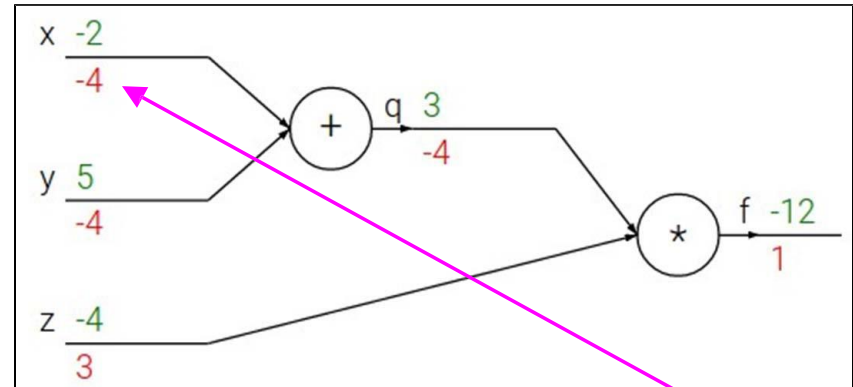
$$f(x, y, z) = (x + y)z$$

$$\text{e.g. } x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

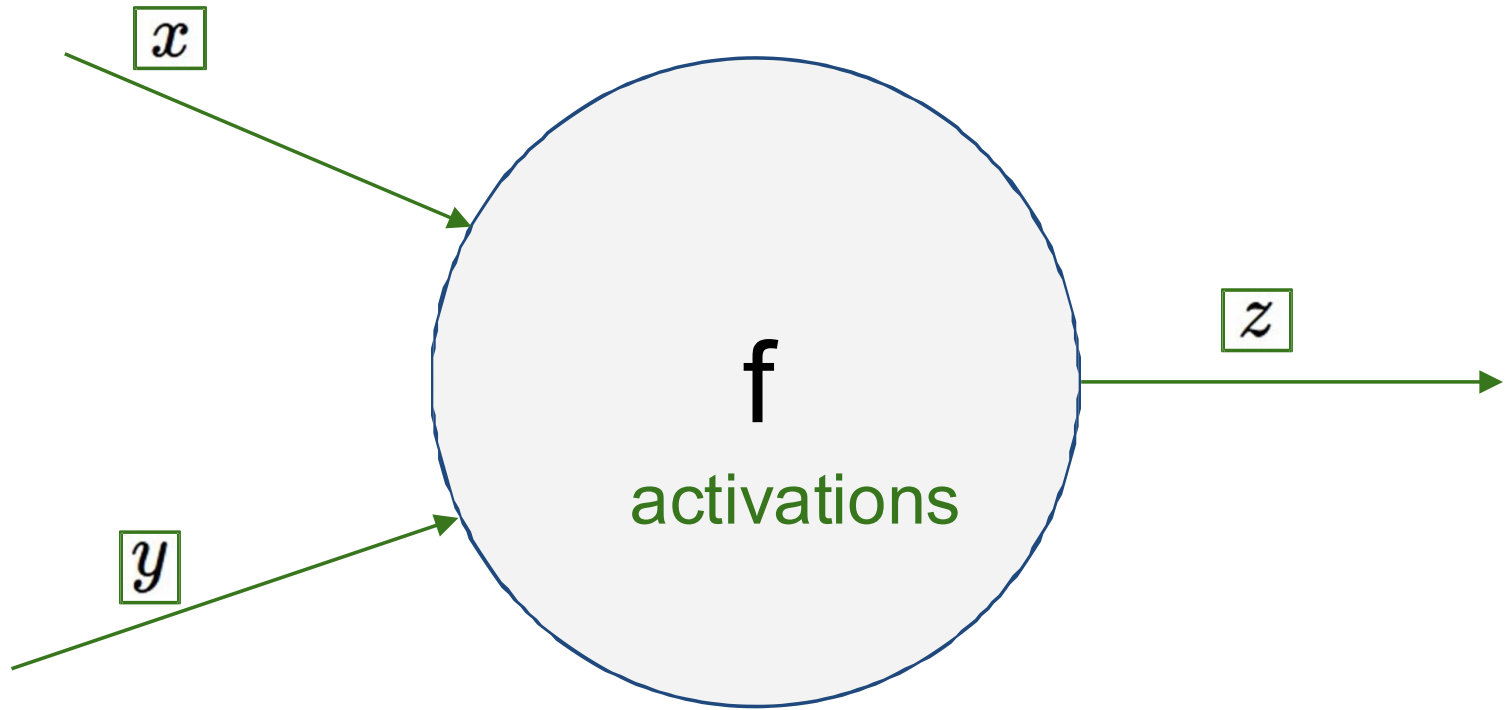
$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$

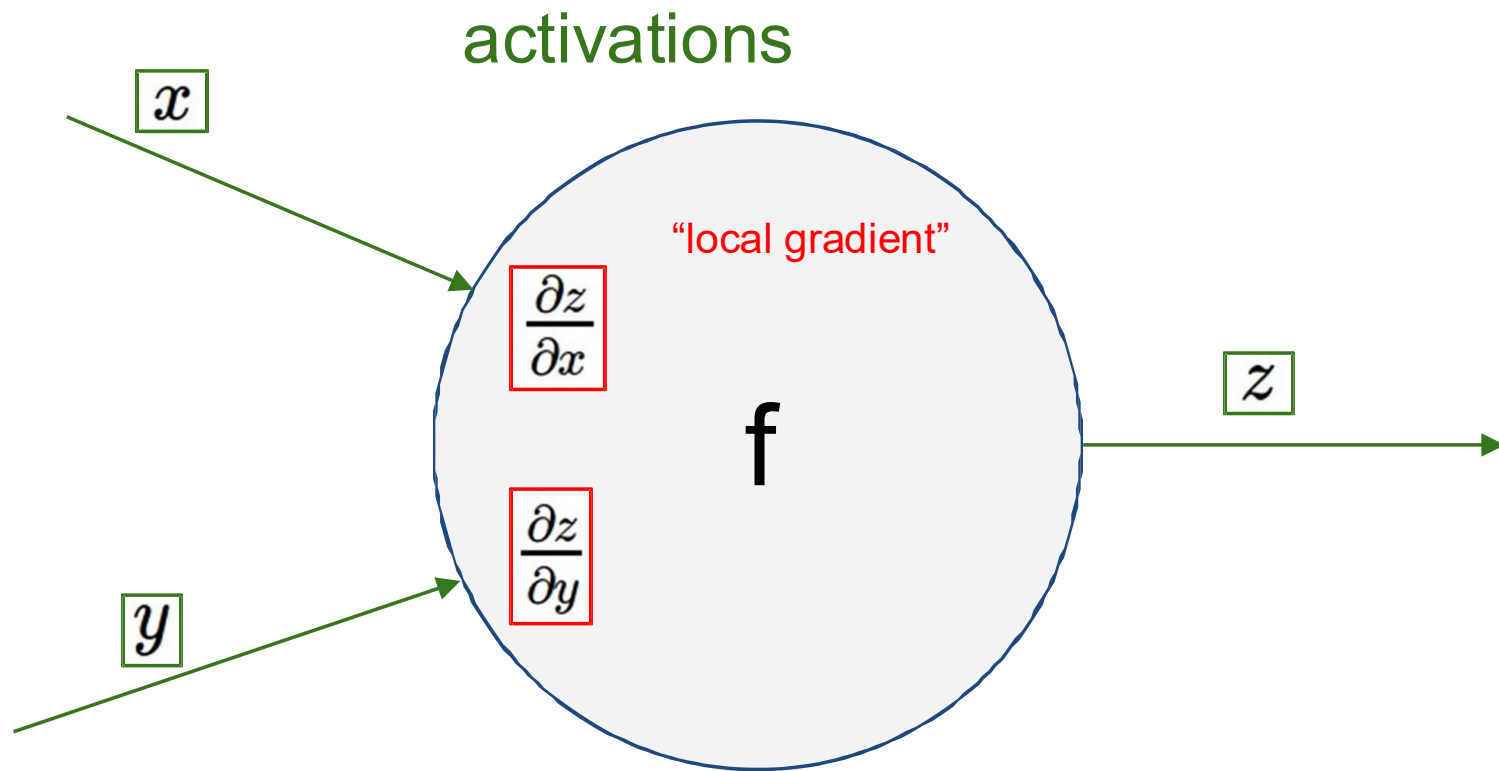


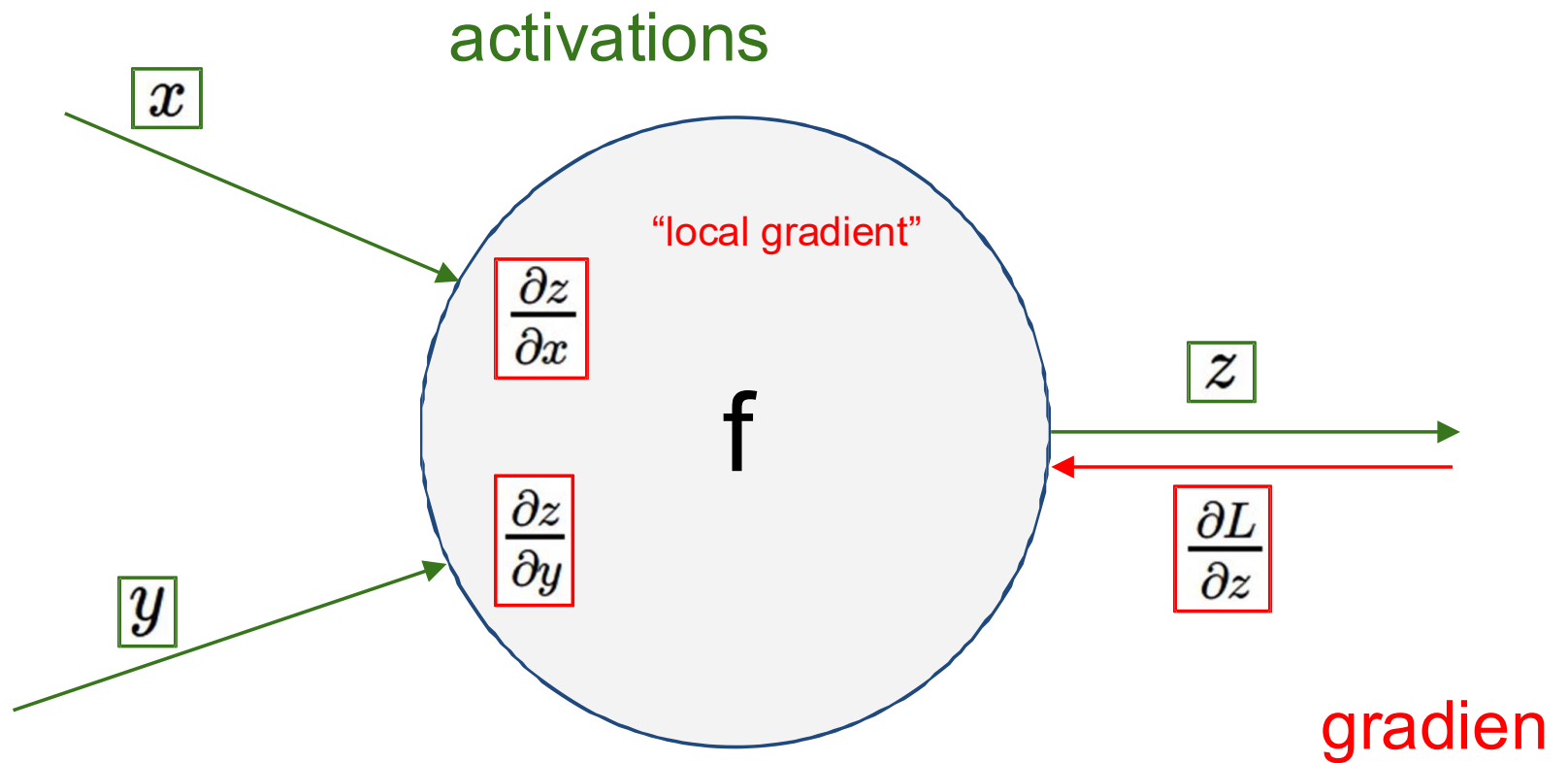
$$\frac{\partial f}{\partial x}$$

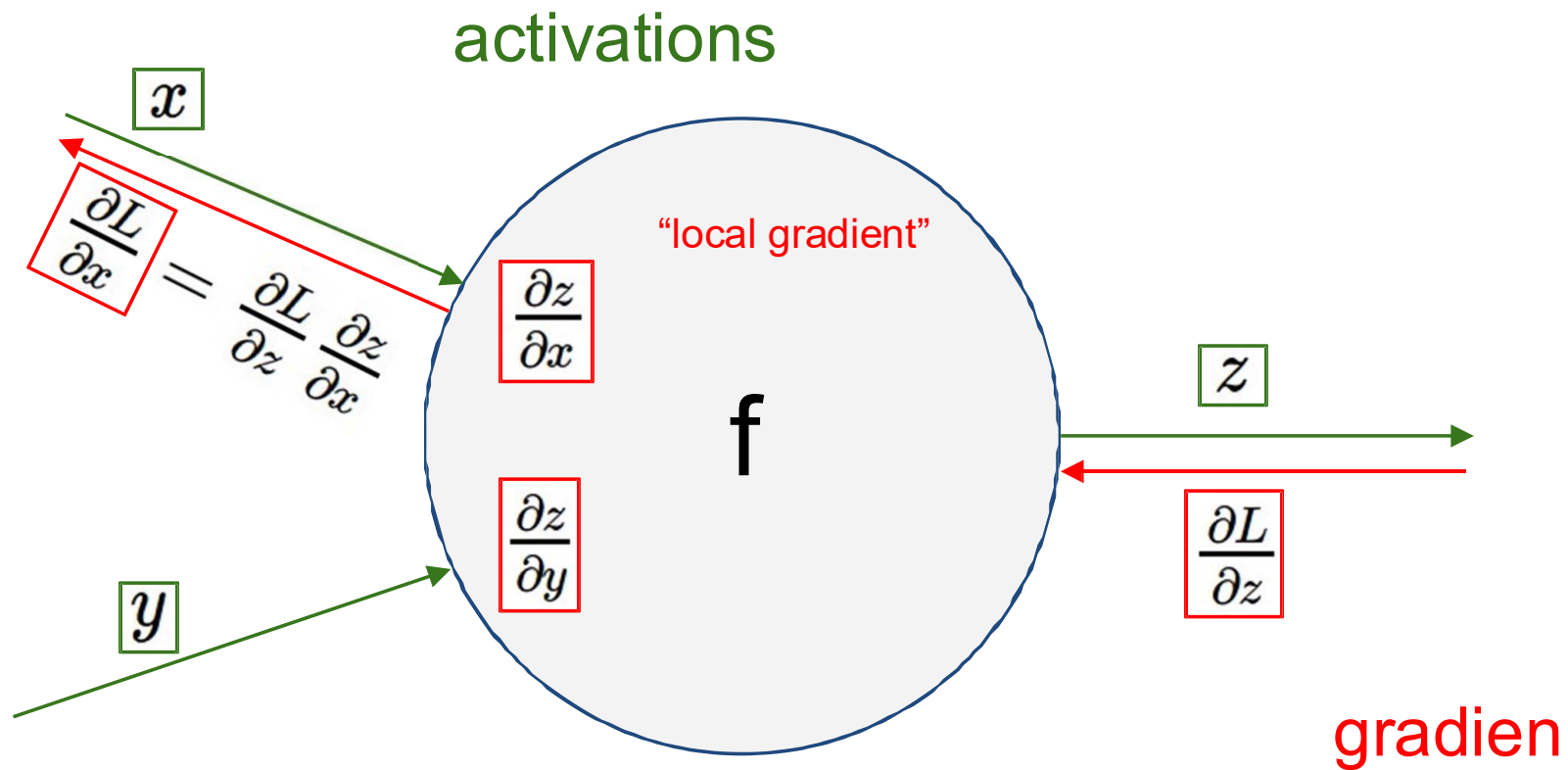
Chain rule:

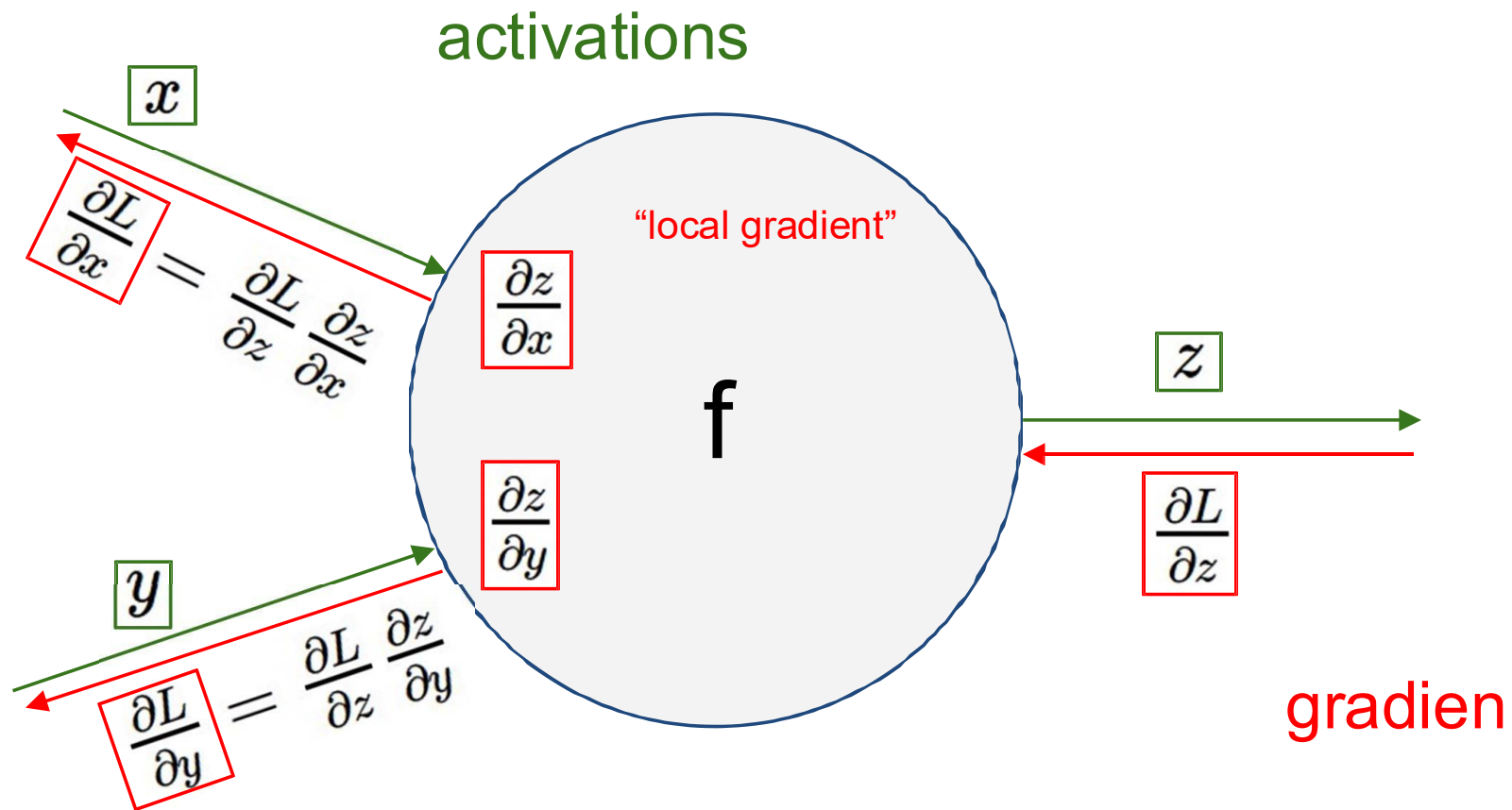
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

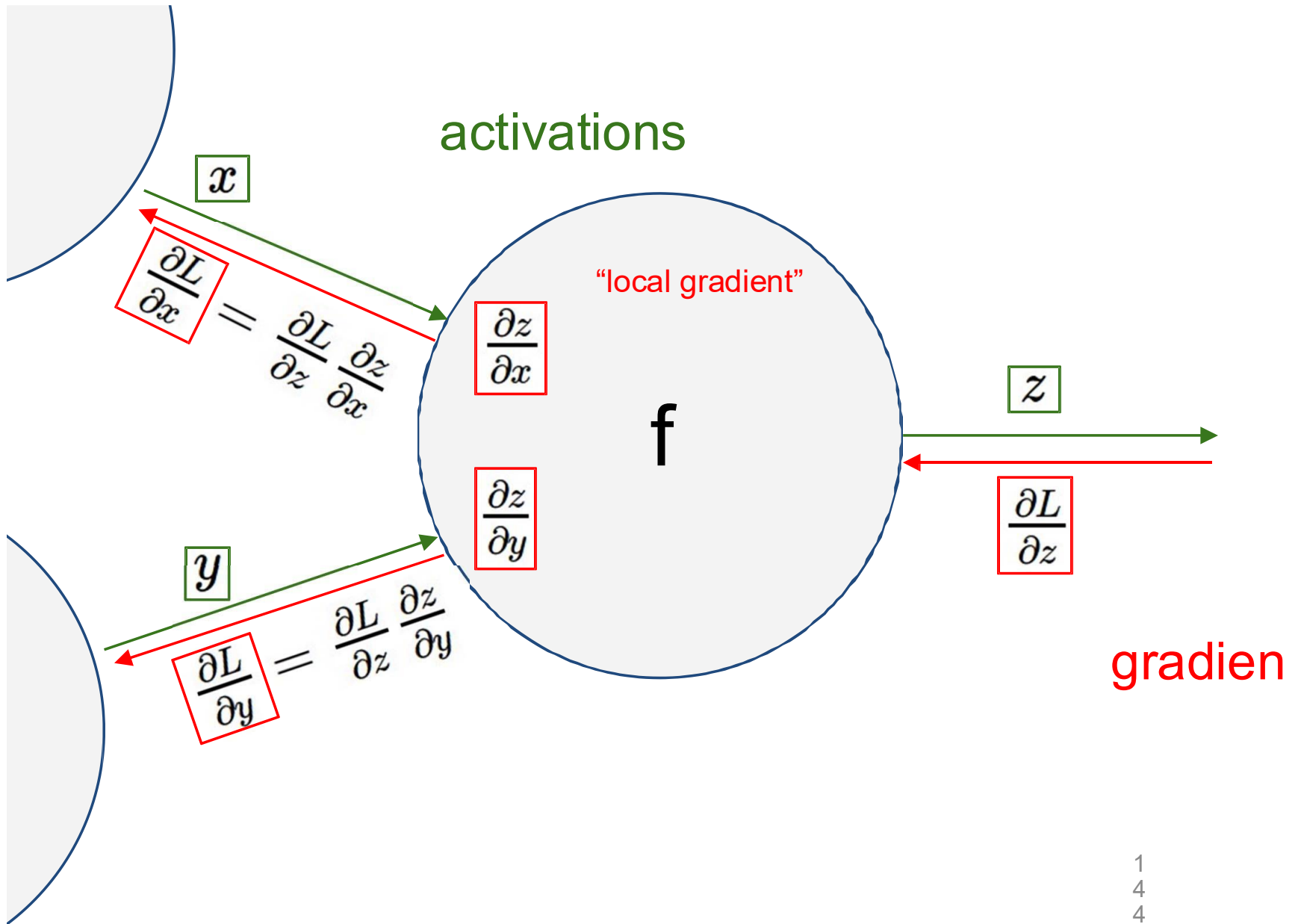






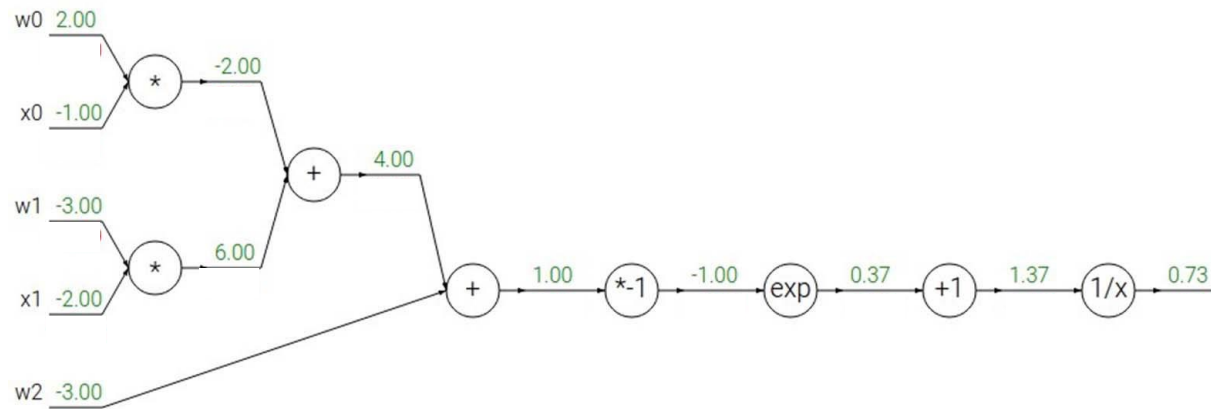




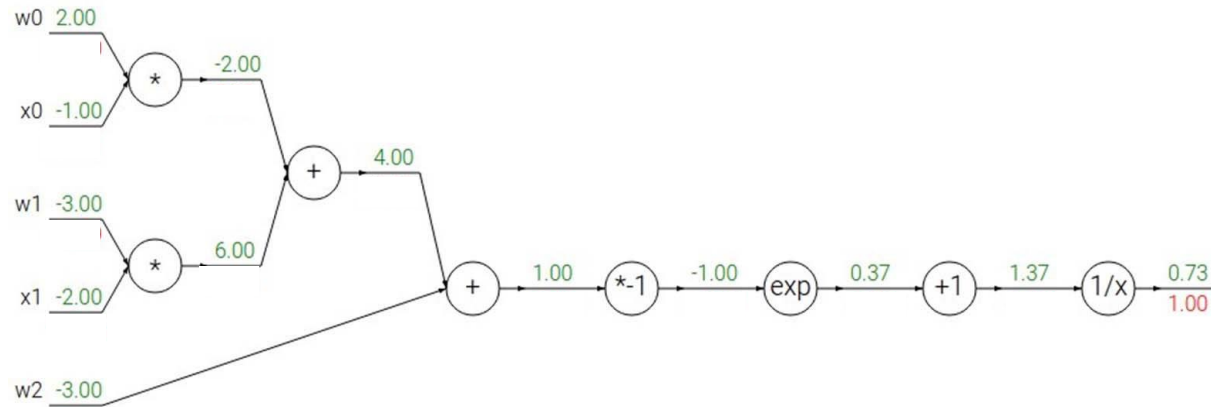




Another example:

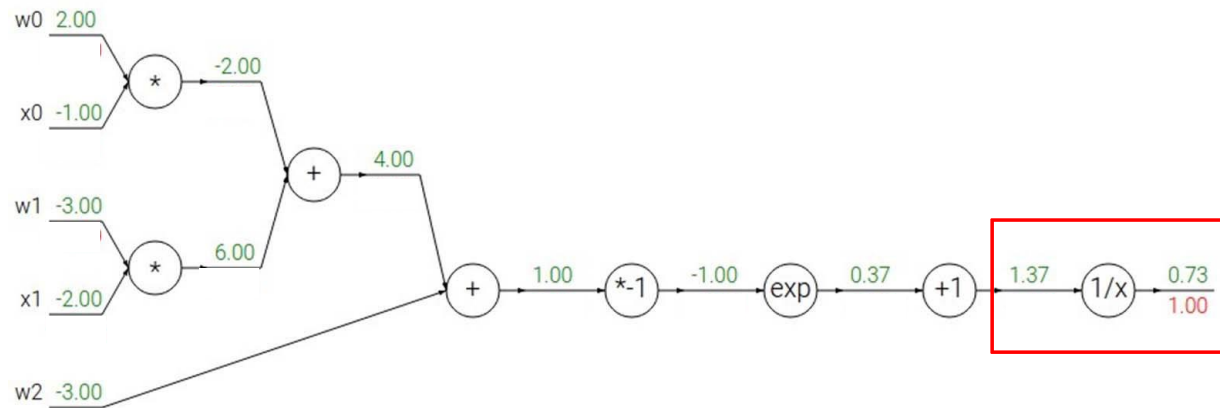


Another example:



|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

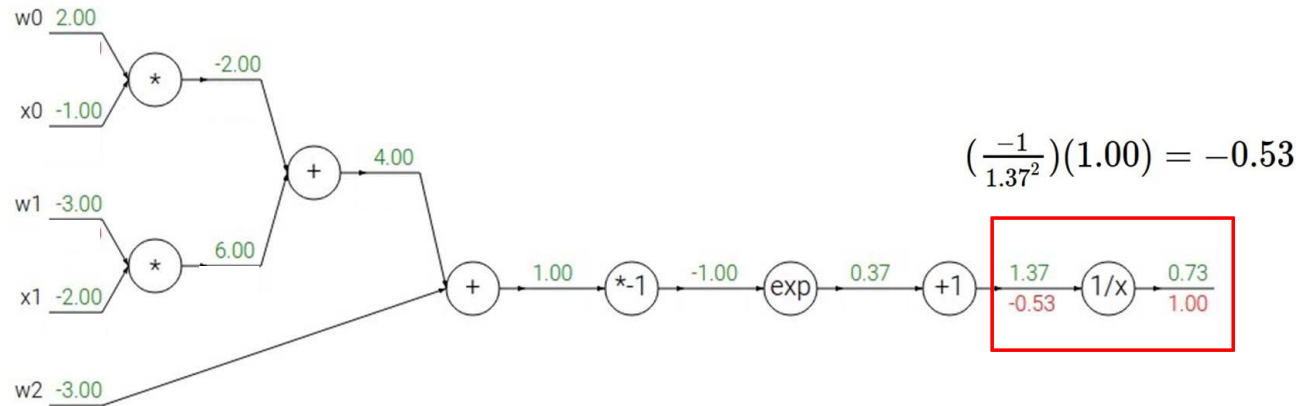
Another example:



$$\begin{array}{lcl}
 f(x) = e^x & \rightarrow & \frac{df}{dx} = e^x \\
 f_a(x) = ax & \rightarrow & \frac{df}{dx} = a
 \end{array}$$

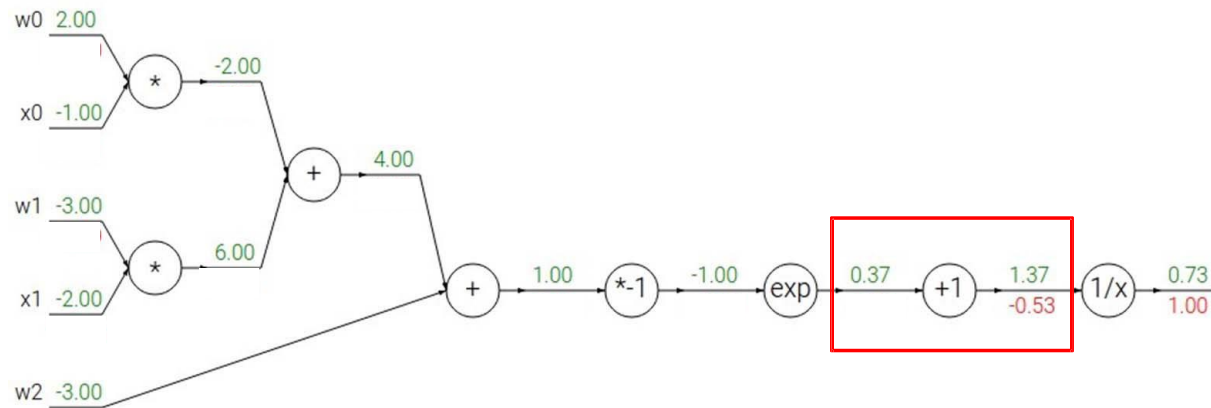
$$\begin{array}{lcl}
 f(x) = \frac{1}{x} & \rightarrow & \frac{df}{dx} = -1/x^2 \\
 f_c(x) = c + x & \rightarrow & \frac{df}{dx} = 1
 \end{array}$$

Another example:



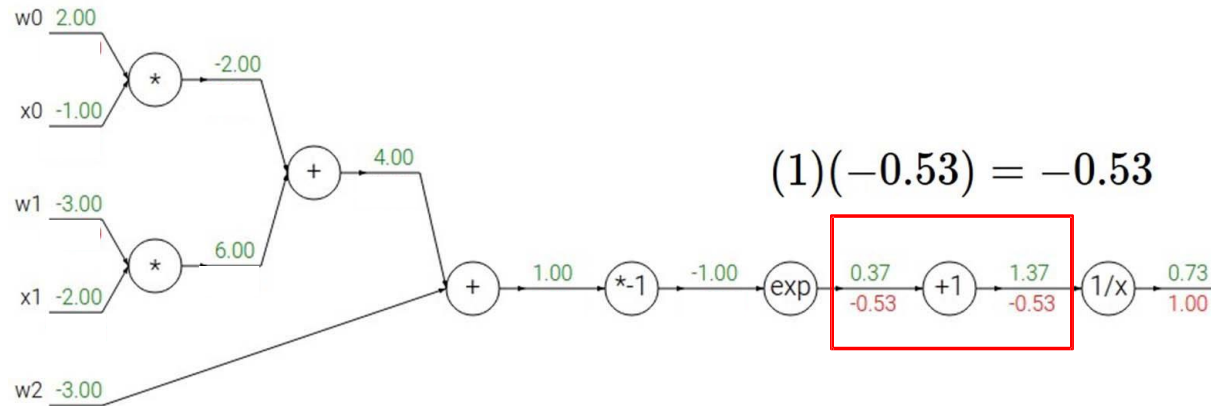
|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

Another example:



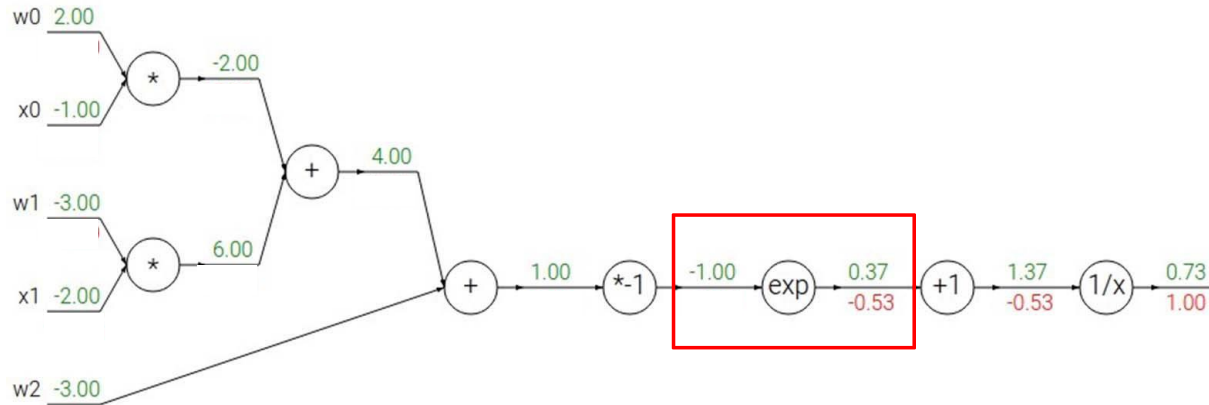
|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

Another example:



|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

Another example:



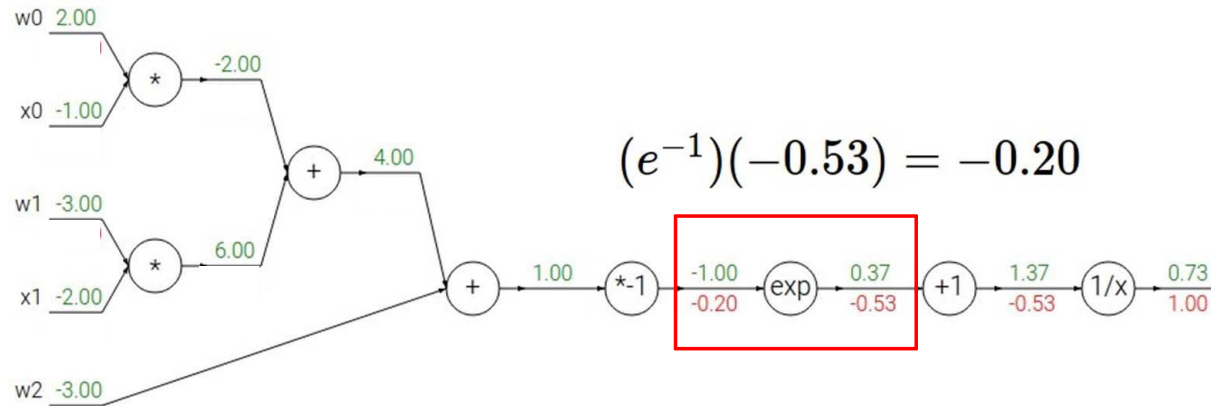
$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Another example:



$$(e^{-1})(-0.53) = -0.20$$

$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

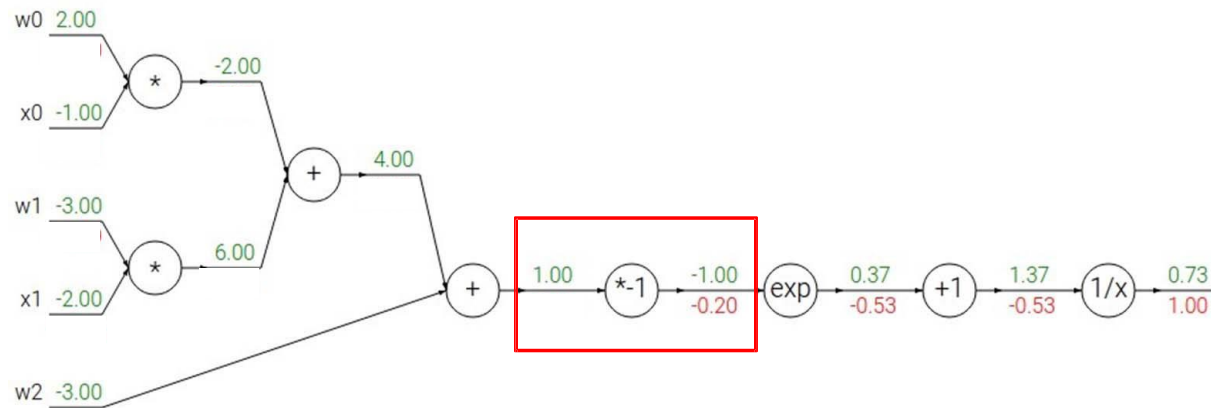
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$



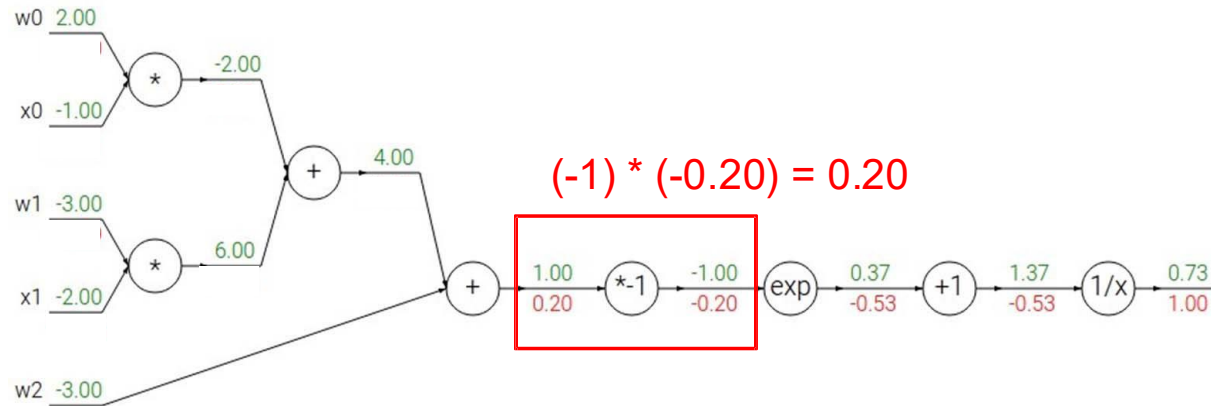
Another example:



|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

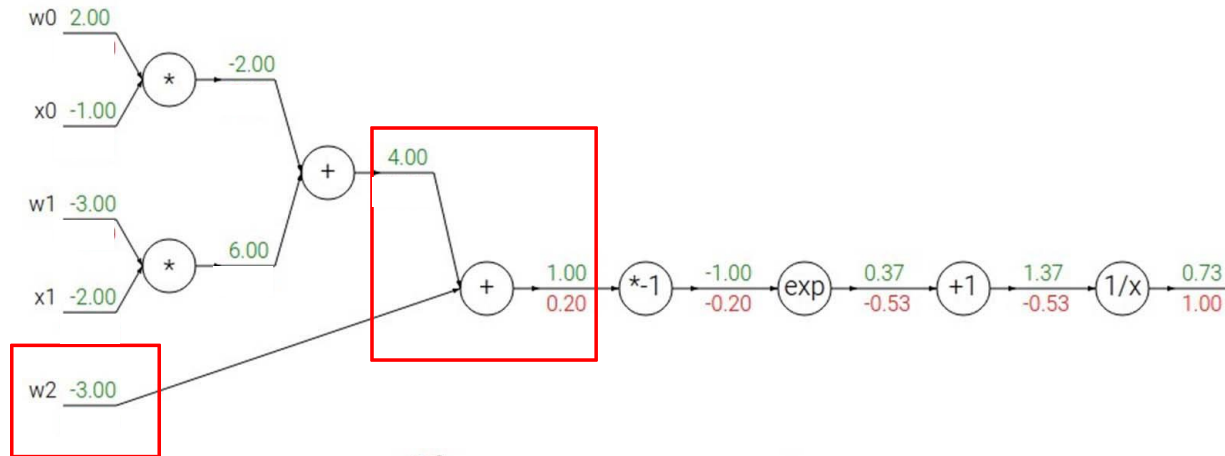
Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

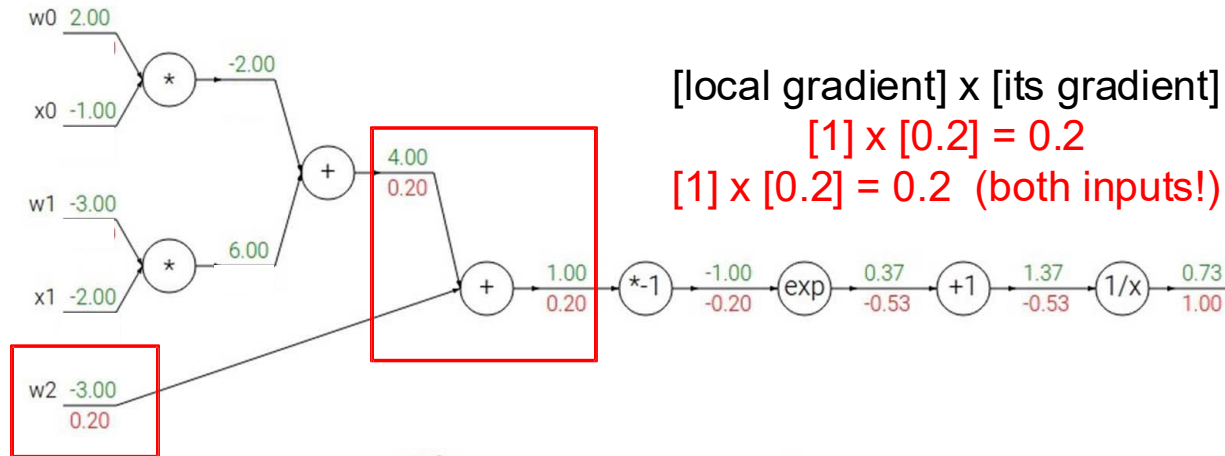
Another example:



|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

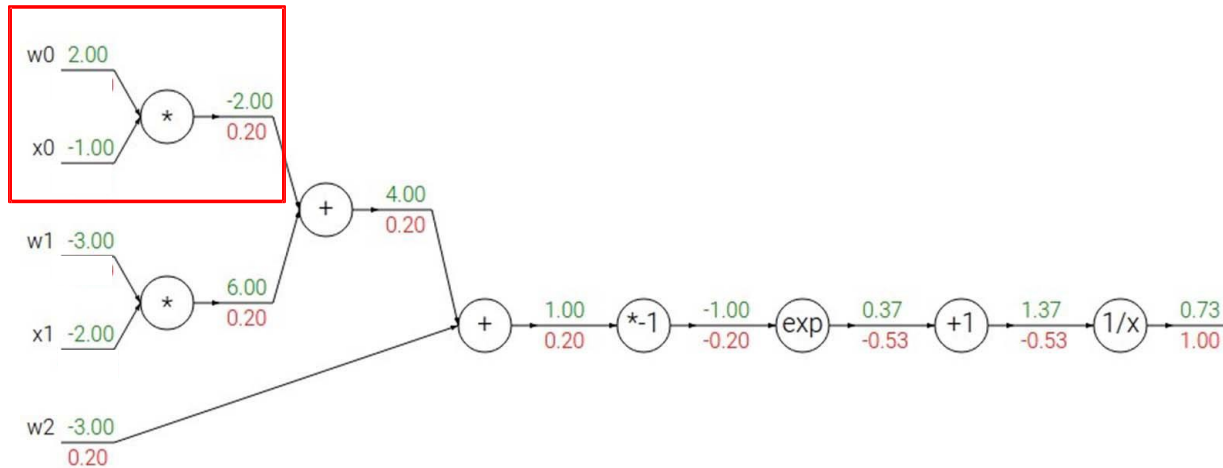
Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

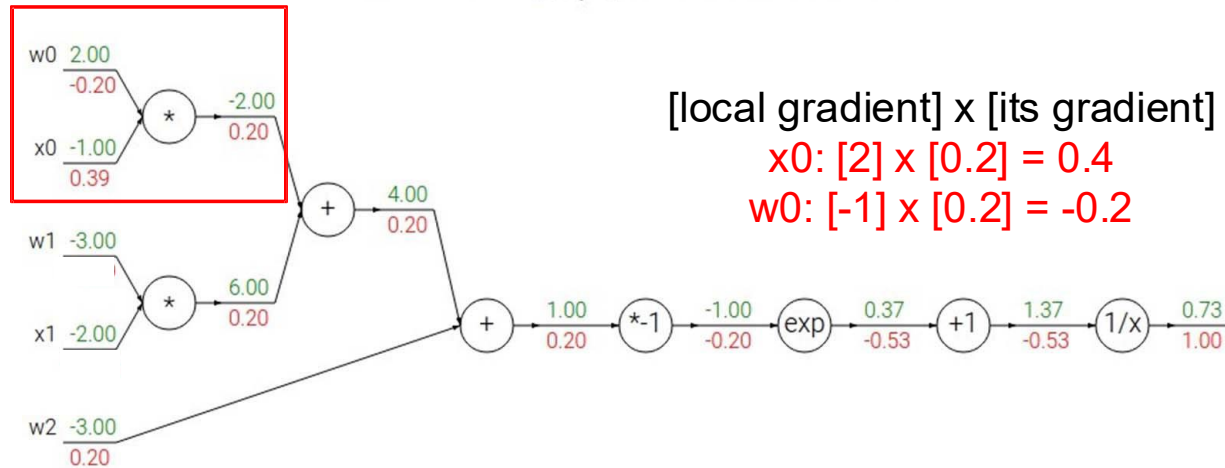
Another example:



|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



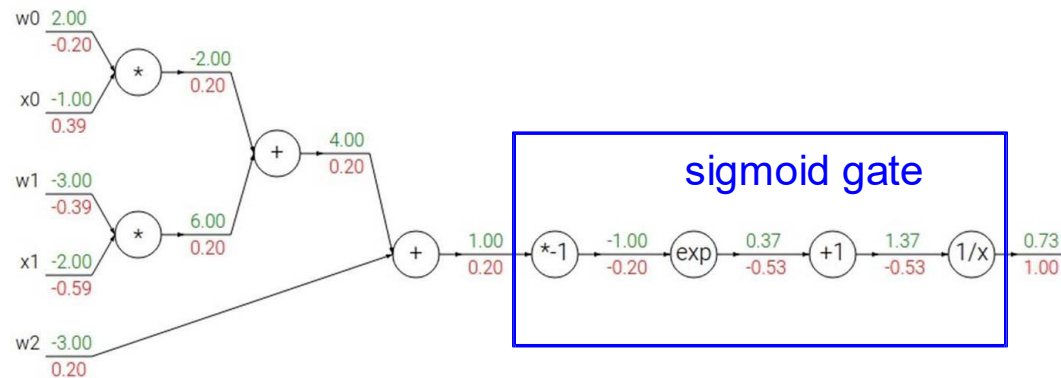
|               |               |                       |  |                      |               |                          |
|---------------|---------------|-----------------------|--|----------------------|---------------|--------------------------|
| $f(x) = e^x$  | $\rightarrow$ | $\frac{df}{dx} = e^x$ |  | $f(x) = \frac{1}{x}$ | $\rightarrow$ | $\frac{df}{dx} = -1/x^2$ |
| $f_a(x) = ax$ | $\rightarrow$ | $\frac{df}{dx} = a$   |  | $f_c(x) = c + x$     | $\rightarrow$ | $\frac{df}{dx} = 1$      |

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

sigmoid function

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left( \frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left( \frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$

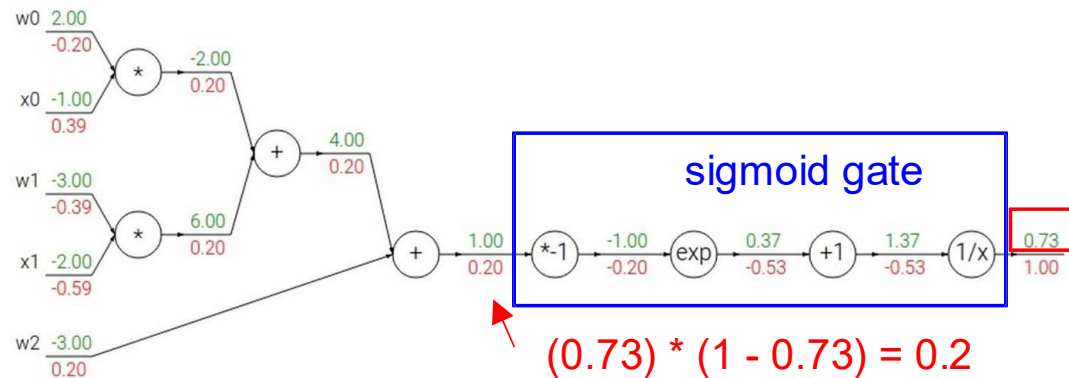


$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

sigmoid function

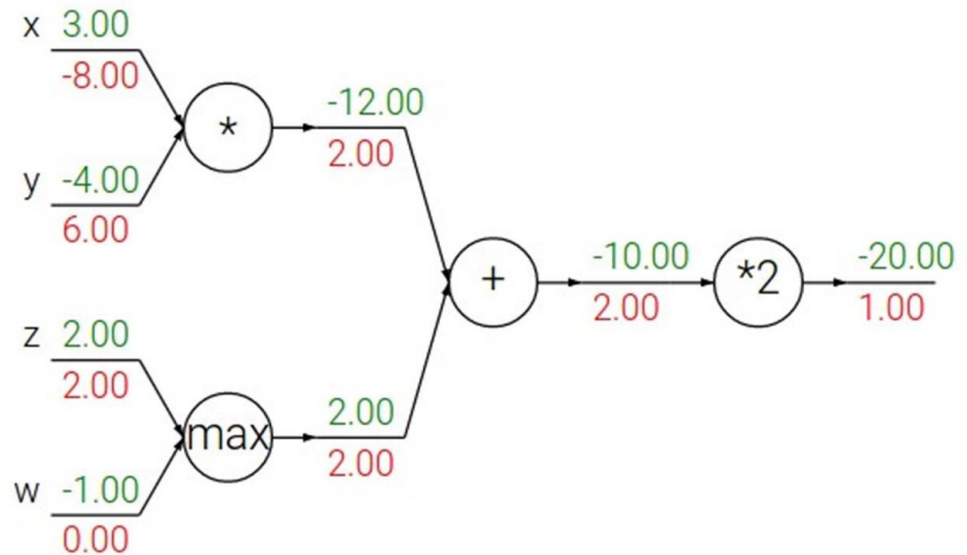
$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left( \frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left( \frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$



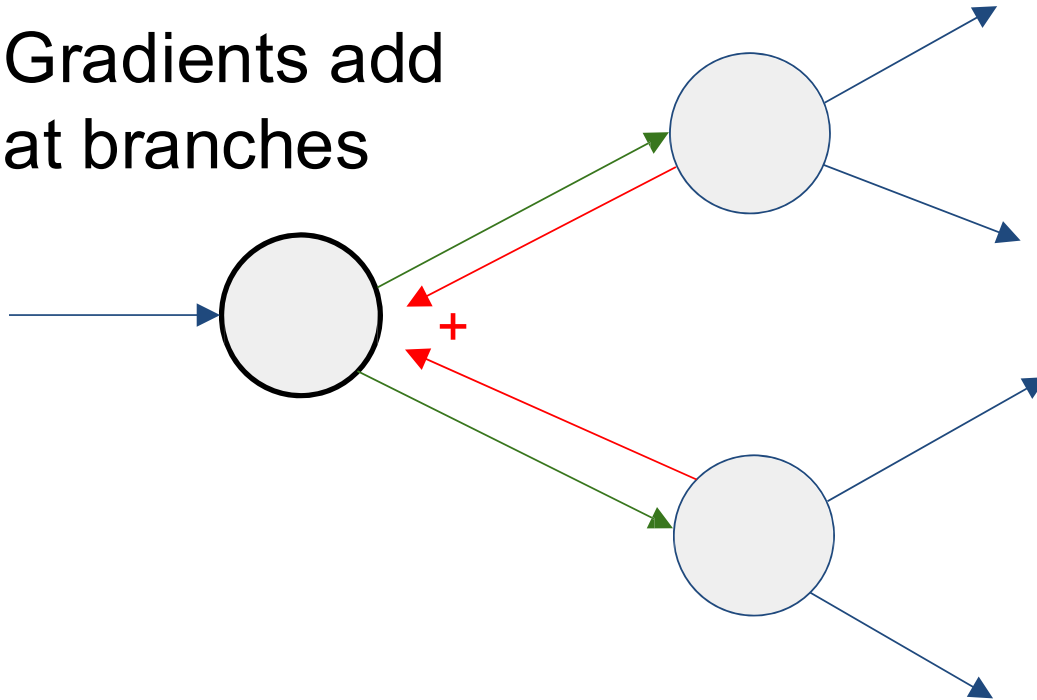


## Patterns in backward flow

**add** gate: gradient distributor  
**max** gate: gradient router  
**mul** gate: gradient... “switcher”?



Gradients add  
at branches



# Backprop Summary

- Modern neural nets are very large
  - No hope of writing down gradient formula by hand for all parameters
- **Backpropagation**: recursive application of the chain rule along a computational graph to compute the gradients of all inputs/parameters/intermediates
- Should maintain a graph structure, where the nodes implement **forward()** and **backward()**
  - **forward**: compute result of an operation and save any intermediates needed for gradient computation in memory
  - **backward**: apply the chain rule to compute the gradient of the loss function with respect to the inputs

# How does this work in practice?

1. Initialize all weights to small, non-zero values
2. Choose a training point (or mini-batch) and **clamp** to input layer
3. Propagate signal **forward** thru network until output is generated
4. Calculate deltas for output layer

# Backprop. in practice (cont'd)

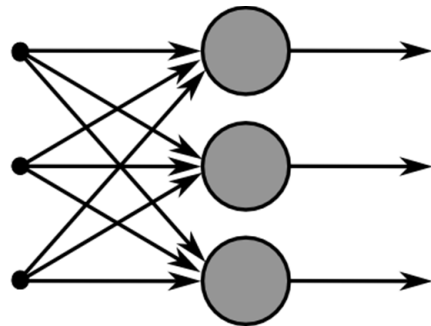
5. Calculate deltas for previous layers by propagating errors backwards until a delta is calculated for every unit
6. Calculate weight updates for every weight
7. Update weights (using some update strategy)
8. Goto step 2

# Notes on Backpropagation

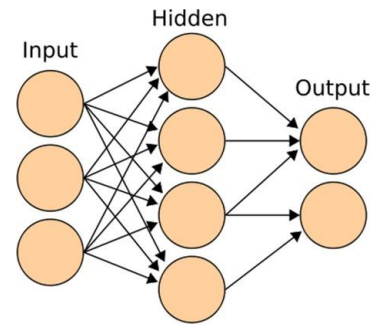
- Learning is incremental
  - Training examples will be presented multiple times
  - Especially with small learning rates
- This is very basic backprop.
  - Other error functions
  - Momentum
  - Different learning rules
- Backprop. will only find local minima
- In practice, it works better with:
  - inputs in the range  $[-1, +1]$
  - small weights

# Why Multilayer?

- What are the hidden layers doing?



One-Layer Perceptron



Two-Layer Perceptron

# Feature / Representation Learning

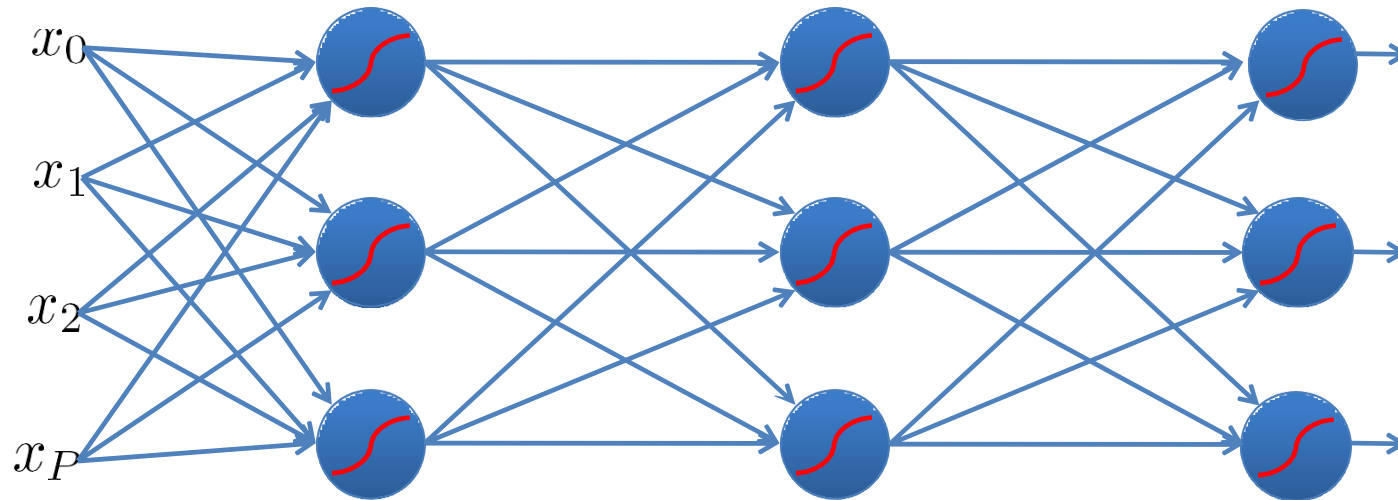
- Each layer maps input to a new feature space
  - You can get away with less clever feature representations using MLP
  - At what cost?
    - Increased VC dimension (also MLP prone to overfitting)
- Nonlinearities can make interpretation of the hidden layers **very** difficult
- Often, MLPs are treated as **black boxes**



# Other Neural Networks

- Backprop generalizes to networks that skip layers
- Many variants of ANN
  - Multiple Outputs
  - Skip Layer Network
  - Recurrent Neural Networks

# Multiple Outputs



Used for N-way classification  
Each output node corresponds to a different class  
Training labels are  $\langle 0, 0, 1, 0, 0, 0 \rangle$   
Testing output: highest value wins!

# Artificial Neural Networks

- MLPs are a network of simple units
  - Lead to complex learners
- Error backpropagation trains an MLP
  - Gradient descent applied to entire network
  - Trains network “one layer at a time”
- These types of MLPs are usually  $\leq 3$  layers
  - Vanishing gradients
  - Too many params

# Introduction to Keras

## – what is it?

An unified high level API (by Chollet) to describe, train, and evaluate deep neural nets. Can work with several computational backends.

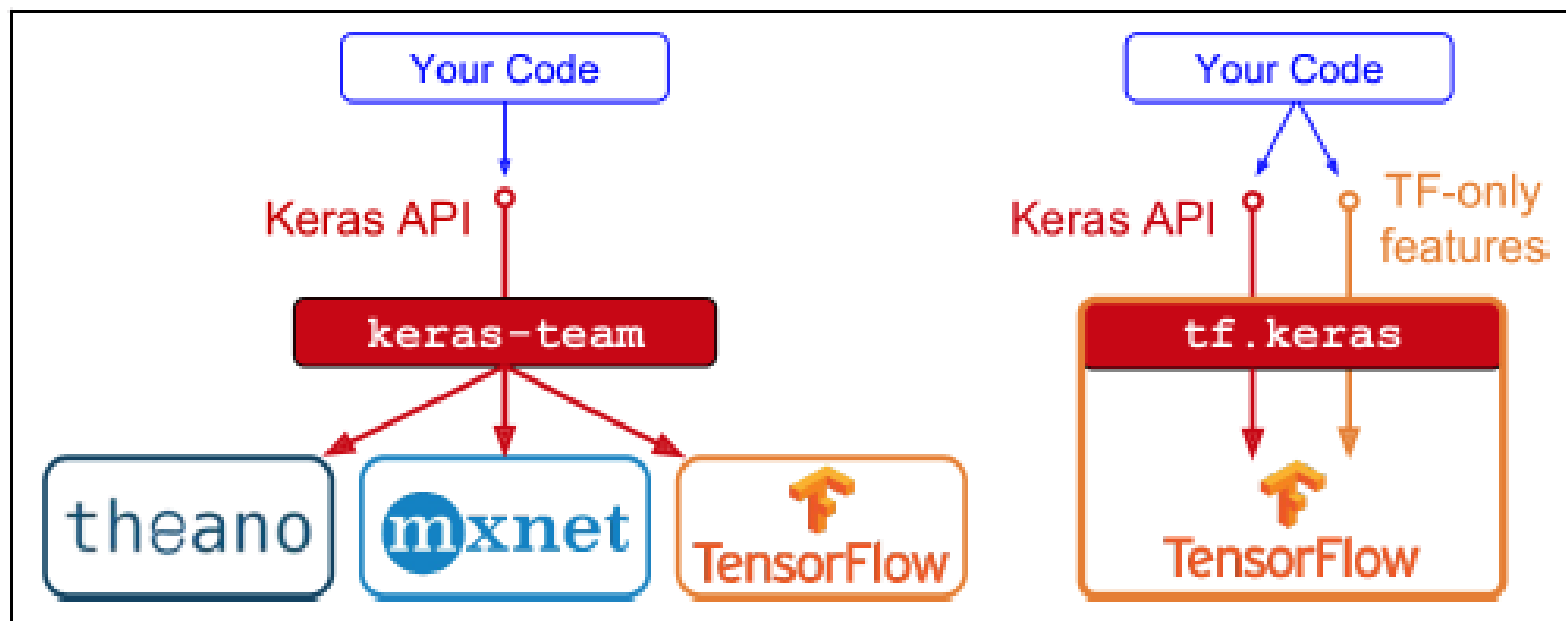


Figure 10-10. Two Keras implementations: *keras-team* (left) and *tf.keras* (right)

# Your First MLP in Keras on TensorFlow

## *step 1: import the library*

```
import tensorflow as tf  from tensorflow
import keras

import matplotlib.pyplot as plt

tf.__version__
keras.__version__
```

## *step 2: import a toy dataset*

```
fashion_mnist = keras.datasets.fashion_mnist
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist.load_data()
```

## *step 3: normalize and scale the toy dataset*

```
X_valid, X_train = X_train_full[:5000] / 255., X_train_full[5000:] / 255. y_valid,
y_train = y_train_full[:5000], y_train_full[5000:]
X_test = X_test / 255.
```

```
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat", "Sandal",
               "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

### *example image*

```
class_names[y_train[0]]  
plt.imshow(X_train[0], cmap="binary")  
plt.axis('off')  
plt.show()
```



# Your First MLP in Keras on TensorFlow

*more example images*

```
n_rows = 4
n_cols = 10
plt.figure(figsize=(n_cols * 1.2, n_rows *
1.2)) for row in range(n_rows):
    for col in range(n_cols):
        index = n_cols * row + col
        plt.subplot(n_rows, n_cols, index + 1)
        plt.imshow(X_train[index], cmap="binary",
interpolation="nearest") plt.axis('off')
        plt.title(class_names[y_train[index]],
fontsize=12) plt.subplots_adjust(wspace=0.2, hspace=0.5)
plt.show()
```





# Your First MLP in Keras on TensorFlow

*step 4: describe the shape of the NN*

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="relu"),
    keras.layers.Dense(100, activation="relu"),
    keras.layers.Dense(10, activation="softmax")
])
```

*print and describe model shape with the following*

```
model.summary()
keras.utils.plot_model(model, "my_fashion_mnist_model.png",
show_shapes=True)
```

| Layer (type)      | Output Shape | Param # |
|-------------------|--------------|---------|
| flatten (Flatten) | (None, 784)  | 0       |
| dense (Dense)     | (None, 300)  | 235,500 |
| dense_1 (Dense)   | (None, 100)  | 30,100  |
| dense_2 (Dense)   | (None, 10)   | 1,010   |

Total params: 266,610 (1.02 MB)

Trainable params: 266,610 (1.02 MB)

Non-trainable params: 0 (0.00 B)

# Your First MLP in Keras on TensorFlow

*step 5: tell it which optimizer, the loss to minimize, and the metrics to measure*

```
model.compile(loss="sparse_categorical_crossentropy",  
              optimizer="sgd",  
              metrics=["accuracy"])
```

*step 6: fit the model, keeping track of validation loss*

```
history = model.fit(X_train, y_train, epochs=30,  
                    validation_data=(X_valid, y_valid))
```

*step 7: plot learning curves*

```
import pandas as pd  
pd.DataFrame(history.history).plot(figsize=(8,  
5)) plt.grid(True)  
plt.gca().set_ylim(0, 1)  
plt.show()
```

*step 8: evaluate the trained model on testing data*

```
model.evaluate(X_test, y_test)
```

Epoch 1/30  
 1719/1719 ————— 2s 810us/step - accuracy: 0.6847 - loss: 1.0173 - val\_accuracy: 0.8162 - val\_loss: 0.5361

Epoch 2/30  
 1719/1719 ————— 1s 759us/step - accuracy: 0.8235 - loss: 0.5093 - val\_accuracy: 0.8414 - val\_loss: 0.4694

Epoch 3/30  
 1719/1719 ————— 1s 770us/step - accuracy: 0.8398 - loss: 0.4583 - val\_accuracy: 0.8432 - val\_loss: 0.4680

Epoch 4/30  
 1719/1719 ————— 1s 767us/step - accuracy: 0.8525 - loss: 0.4198 - val\_accuracy: 0.8600 - val\_loss: 0.4103

Epoch 5/30  
 1719/1719 ————— 1s 754us/step - accuracy: 0.8583 - loss: 0.4045 - val\_accuracy: 0.8566 - val\_loss: 0.4093

Epoch 6/30  
 1719/1719 ————— 1s 755us/step - accuracy: 0.8641 - loss: 0.3854 - val\_accuracy: 0.8726 - val\_loss: 0.3698

Epoch 7/30  
 1719/1719 ————— 1s 762us/step - accuracy: 0.8704 - loss: 0.3680 - val\_accuracy: 0.8714 - val\_loss: 0.3777

Epoch 8/30  
 1719/1719 ————— 1s 739us/step - accuracy: 0.8718 - loss: 0.3600 - val\_accuracy: 0.8782 - val\_loss: 0.3530

Epoch 9/30  
 1719/1719 ————— 1s 746us/step - accuracy: 0.8755 - loss: 0.3498 - val\_accuracy: 0.8750 - val\_loss: 0.3558

accuracy: 0.8836 - loss: 0.3377

